

AI Home Gym – Production System Architecture v1.0

****Audience:**** Kd (solo developer, owner)

****Scope:**** Web app (now) + Mobile app (next) + Gym B2B platform + Consumer subscriptions

****Status:**** This is the master blueprint. Deep-dive specs (exercise engine, gym dashboard, DB schema, billing, mobile, retention) follow as separate parts – see §23.

0. How to use this document

Read §1-§2 first – they explain **why** the redesign looks the way it does. §3-§20 are the target system. §21 is the retention/growth feature set. §22 is the build order with acceptance criteria – follow it top to bottom. Nothing in here requires throwing away your domain logic: your squat rules, calibration logic, quota philosophy, RAG knowledge base, and most frontend pages survive. What gets replaced is the **skeleton** they hang on.

1. Audit of the current codebase

What exists today: three services – `backend-auth` (Node/Express + Mongoose), `backend-ml` (FastAPI + Motor/Mongo + Redis: exercises, workouts, progress, coach RAG, nutrition, running, gamification, pose WebSocket), and `frontend` (React 18 + Vite SPA running MediaPipe in-browser, streaming 15fps keypoints to the server over WebSocket).

1.1 What is genuinely good (keep the logic, port the code)

Asset	Why it's valuable
---	---
Squat / jump-squat / chair-squat rule logic (`form_analyzer.py`)	View-aware, self-calibrating (standing baseline, per-user valgus delta), research-grounded thresholds with documented rationale. This is your real IP. It gets <i>*translated*</i> , not rewritten.

| `RepCounter` config dict (up/down angle + hysteresis + direction) |
Already half-way to the declarative system we need – proof the
config-driven approach works. |
| Quota module (`core/quotas.py`) | Correct fail-open (cheap) /
fail-closed (expensive) philosophy, atomic Redis increment. Survives as-is
conceptually; gets generalized to per-plan limits. |
| Browser-side MediaPipe (`usePoseDetection.js`) | Right call. Throttled
inference, visibility pause, exponential backoff – good engineering. |
| RAG knowledge base content (5 markdown guides) | Content is portable
regardless of vector store. |
| Prod-config boot validation (both backends) | Good habit; carries into
the new API. |
| Frontend pages/UX (Dashboard, ActiveWorkout, Nutrition, Coach, Progress,
WorkoutBuilder...) | The product surface is built. It gets re-pointed at a
new API and a local engine – not rebuilt. |

1.2 The five architectural blockers

****Blocker 1 – Form analysis lives on the server.**** Every workout streams
15fps keypoints over a WebSocket to a Python process holding per-user
`RepCounter` + calibration state ****in process memory**** (`active\_sessions`
dict). Consequences: (a) you can never run two API instances behind a load
balancer without sticky sessions – horizontal scaling is blocked by
design; (b) every rep costs server CPU and mobile data – cost scales with
usage; (c) feedback latency = network round-trip; (d) zero offline
capability – fatal for gyms with bad signal and for mobile; (e) the mobile
app cannot reuse any of this logic without also streaming to the server.

****Blocker 2 – Exercises are hand-written Python functions.****

`\_analyze\_squat`, `\_analyze\_jump\_squat`, `\_analyze\_chair\_squat` = ~38 KB
for 3 variants. At this rate, 59 exercises ≈ 700 KB of bespoke
per-exercise code, each new exercise requires a server deploy, and none of
it runs on-device. This is the single biggest threat to ever shipping the
other 56 exercises.

****Blocker 3 – Two backends, two languages, one shared JWT secret.**** Node
auth + Python everything-else means duplicated middleware, duplicated
deploys, duplicated config, and an HS256 secret shared across services –
all overhead a solo dev pays forever. The split earns nothing: once pose
analysis leaves the server (Blocker 1), there is no Python-only workload

left in the hot path (coach = Groq API call, meal photos = vision API call, routing = ORS API call).

****Blocker 4 – The business layer doesn't exist.**** No gyms, no tenancy, no seats, no plans, no subscriptions, no payments, no entitlements, no admin, no dashboard. Quotas are hardcoded constants identical for every user. The entire revenue model from our previous discussion is unbuilt, and bolting multi-tenancy onto a single-tenant Mongo schema later is far more painful than designing for it now.

****Blocker 5 – MongoDB under relational, money-touching data.****

Users↔gyms↔seats↔subscriptions↔invoices↔quotas↔leaderboards are joins, constraints, and transactions. You want a database that makes double-charging and orphaned seats *impossible*, not merely unlikely.

1.3 Additional findings

- ● ****Committed secrets:**** real values for `GROQ\_API\_KEY`, `GOOGLE\_CLIENT\_SECRET`, `ORS\_API\_KEY`, `JWT\_SECRET`/`ML\_JWT\_SECRET`, `MONGO\_URI` exist in `.env`, `backend-auth/.env`, `backend-ml/.env` inside the shared archive. ****Rotate every one of these before doing anything else****, and purge `.env` from git history (`git filter-repo`). Add secret scanning (Gitleaks) to CI so this can't recur.
- ● ****Effectively zero tests**** (one `test\_stgcn.py`). "No bugs like big tech" is achieved by big tech through automated regression tests, not carefulness. The new architecture makes the critical path (the engine) trivially testable – see §5.4.
- ● ****Server-side ML experiments in prod tree**** (YOLOv8s-pose, ST-GCN checkpoints, 17 MB CTR-GCN ONNX in `frontend/public`, `FormClassifier.js`). Park all of it in `ml-training/`. The rule engine is the product; ML classifiers are a research track, not a dependency.
- ● ****Monolithic page components**** (`ActiveWorkout.jsx` 51 KB, `Settings.jsx` 37 KB) – refactor opportunistically as pages are touched, not as a project.
- ● ****No migrations, no CI/CD, no error tracking, no analytics events.**** All addressed in §16–§19.

****Verdict:**** the *domain logic* is worth keeping; the *skeleton* is a university-project skeleton (it says so in `PROJECT\_PLAN.md`) and must be

replaced before gyms pay you. That is exactly what the rest of this document does.

2. The five architecture decisions

Everything downstream follows from these. Each reverses one blocker.

****D1 – All pose analysis runs on-device, on every platform.**** The camera → MediaPipe → Form Engine loop executes entirely in the browser (web) and on the phone (mobile). The server never sees a keypoint during a workout. Clients send compact ****rep/set summaries**** to a REST sync endpoint when a set ends (batched, retried, idempotent). Result: form checking has ~zero marginal server cost at any scale, feedback is instant (<50 ms loop vs. network RTT), workouts work offline, and the WebSocket + in-memory session store are deleted from the hot path entirely. Anti-cheat moves server-side onto the summaries (\$14).

****D2 – Exercises become data, not code.**** One generic TypeScript ****Form Engine**** interprets declarative ****Exercise Definitions**** (JSON): signals to compute, calibration needs, a rep state machine, fault rules with severities, scoring weights, localized message keys. ~10-12 movement-family **templates** parameterize into all 59 exercises. Definitions are served by the API as a ****versioned bundle**** – you ship a new exercise by publishing a config row, with ****no app-store release and no deploy****. Your existing squat/jump/chair logic becomes the first three definitions, verified for parity against recorded sessions (\$5).

****D3 – One backend: a TypeScript modular monolith.**** Fastify + TypeScript + Zod + Drizzle ORM, with BullMQ (Redis) for background jobs. `backend-auth` and `backend-ml` merge into one deployable with strict internal module boundaries (\$6). One language across API, web, mobile, and the shared engine package means one skill set, shared types end-to-end, and half the operational surface. (This is not "monolith because small" – it's the same shape Shopify, GitHub, and Stripe scaled on. Microservices are an org-chart solution; you are one person.)

****D4 – PostgreSQL is the system of record; Redis and R2 are specialists.**** Postgres (managed, with point-in-time recovery) holds all relational/money

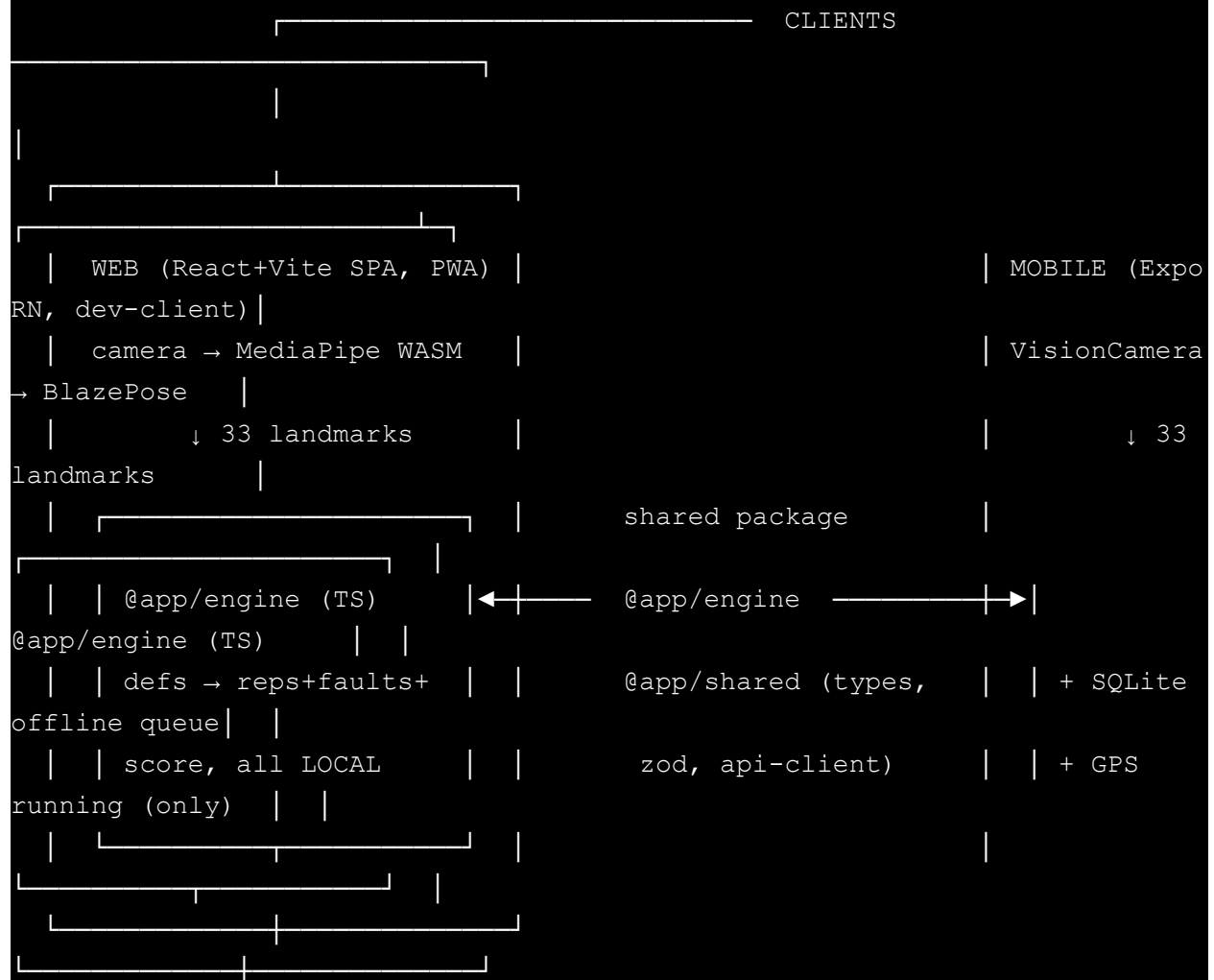
data with real constraints and transactions. Redis: quota counters, rate limits, leaderboard ZSETs, geocode cache, job queue. Cloudflare R2: meal photos (transient), gym logos, avatars, generated share cards and report PDFs. Mongo is retired after a one-time migration (§7.4).

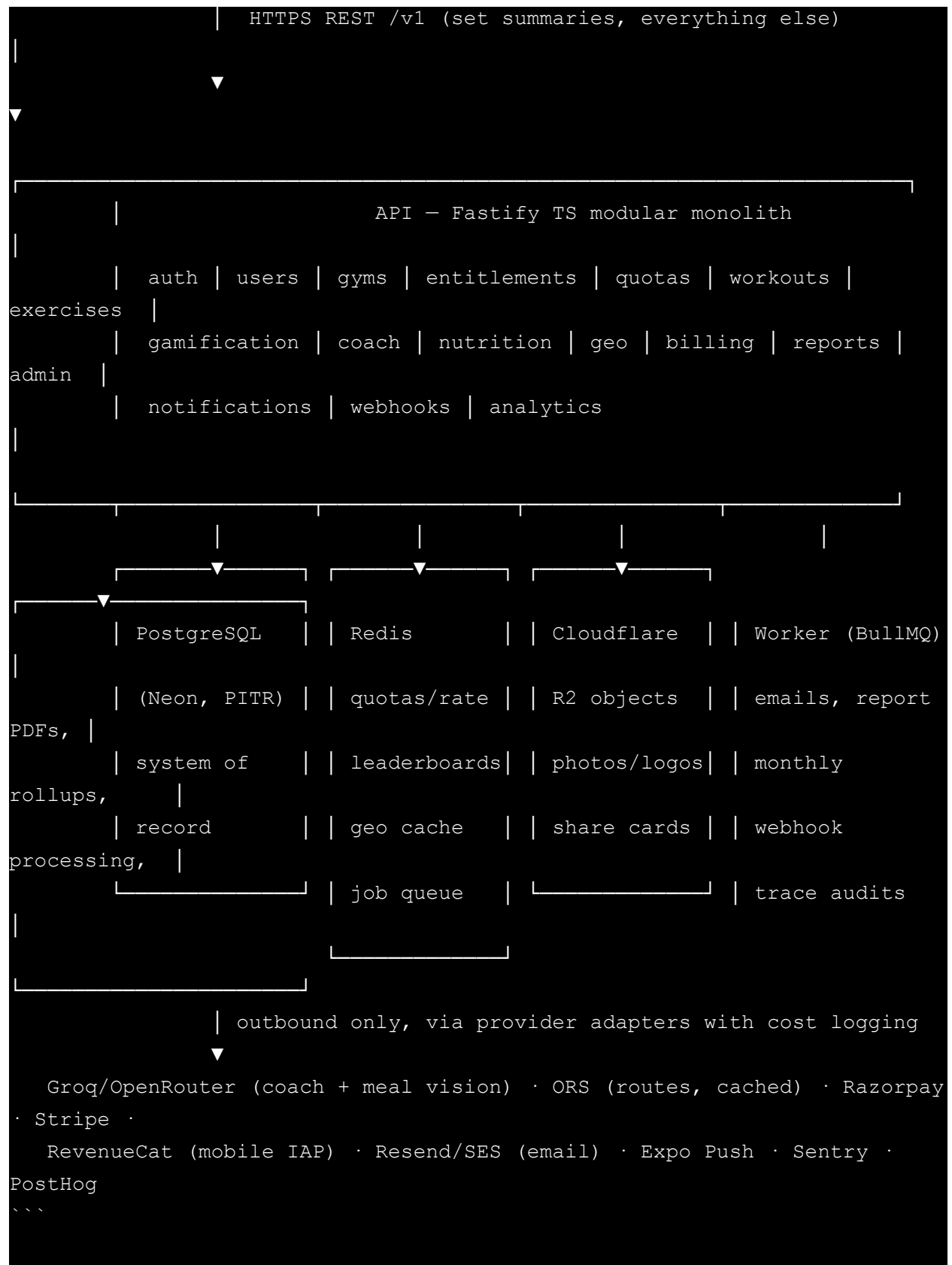
****D5 – Tenancy, plans, and quotas are first-class and config-driven.****

Gyms, seats, roles, and an ****entitlement resolver**** (personal subscription OR active gym membership → effective plan) sit at the core of the schema. Every limit in the product reads from a plan's entitlement JSON – changing a quota or price is a data change, never a deploy (§8–§9).

3. Target system diagram

...





Key property: ****the expensive, latency-sensitive loop (camera→feedback) never crosses the network.**** The server's job shrinks to identity, money, persistence, social features, and metered third-party calls – all cheap, all stateless, all horizontally scalable from day one.

4. Monorepo layout

pnpm workspaces + Turborepo. One repo, one CI, shared types everywhere.

...

```
ai-home-gym/
├─ apps/
│   ├─ api/           # Fastify TS modular monolith ($6)
│   ├─ web/           # Existing Vite SPA, migrated in place ($13)
│   ├─ mobile/        # Expo React Native app ($12)
│   └─ dashboard/     # Gym owner dashboard (Vite SPA or /gym
routes in web) ($11)
├─ packages/
│   ├─ engine/        # THE Form Engine: pure TS, zero deps on
DOM/RN ($5)
│   │   ├─ src/core/  # signals, FSM, fault evaluator, scorer,
calibration
│   │   └─ src/definitions/ # exercise definition JSONs + family
templates
│   │       └─ test/traces/ # recorded keypoint sessions (golden files)
│   └─ shared/        # zod schemas, API types, api-client,
constants
│   └─ config/        # eslint, tsconfig, prettier presets
├─ ml-training/      # research track (YOLO, ST-GCN, CTR-GCN) –
not deployed
├─ infra/            # docker-compose, Dockerfiles, Caddy config,
deploy scripts
└─ .github/workflows/ # CI: typecheck, unit + golden-trace tests,
gitleaks, deploy
...
```

Rules: `packages/engine` imports nothing platform-specific (it must run in browser, Hermes/RN, and Node test runner identically). `apps/\*` never

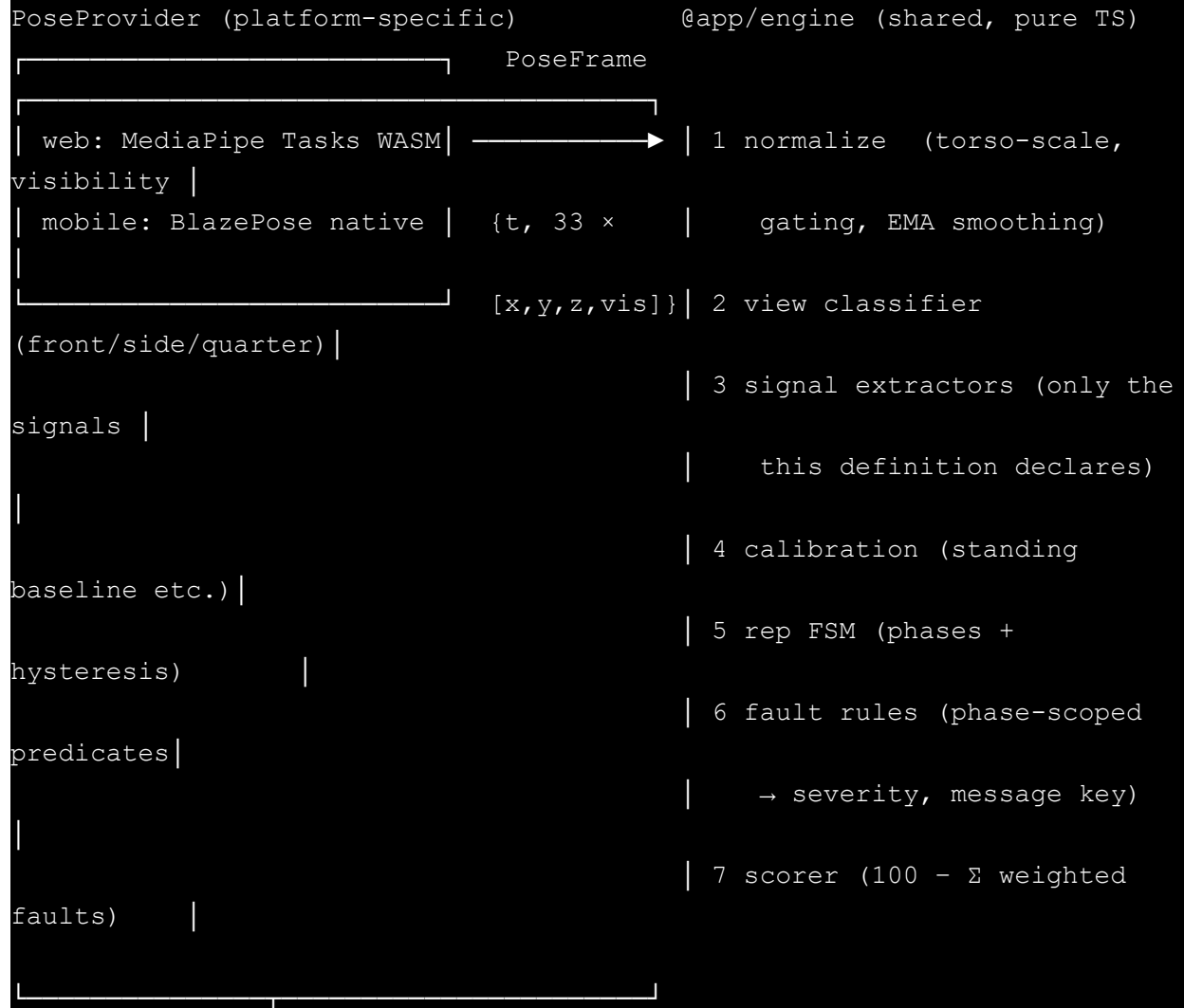
import each other – only packages. All request/response shapes are Zod schemas in `packages/shared`, consumed by both API (validation) and clients (types) – one source of truth, no drift.

5. The Form Engine & Exercise Definition System (EDS)

This is the heart of the product and the answer to "58 exercises to go." Full spec is ****Part 2****; here is the architecture.

5.1 Pipeline (runs on-device, every frame)

...



```
UI events (rep++, live cue, voice) + SetSummary
→ sync ($5.3)
```
```

The canonical `PoseFrame` is BlazePose's 33-landmark layout on **both** platforms – the engine never knows which platform produced it.

### ### 5.2 Exercise Definitions – exercises as data

A definition declares (sketch; full schema in Part 2):

```
```jsonc
{
  "key": "squat", "version": 4, "family": "squat_pattern",
  "views": ["front", "side"], "calibration": ["standing_baseline"],
  "signals": ["knee_angle_L", "knee_angle_R", "trunk_incline",
"valgus_delta", "hip_elevation"],
  "rep": { "metric": "knee_angle_min", "upAt": 160, "downAt": 100,
          "countOn": "up", "minRepMs": 900, "maxRepMs": 12000 },
  "phases": ["standing", "descent", "bottom", "ascent"],
  "faults": [
    { "id": "shallow_depth", "phase": "bottom", "when": "knee_angle_min
> 120",
      "severity": 25, "msg": "fault.squat.depth" },
    { "id": "knee_valgus", "phase": "descent|ascent", "view": "front",
      "when": "valgus_delta > 0.30", "severity": 30, "msg":
"fault.squat.valgus" },
    { "id": "trunk_lean", "phase": "bottom", "view": "side",
      "when": "trunk_incline > 50", "severity": 20, "msg":
"fault.squat.lean" }
  ],
  "scoring": { "base": 100, "floor": 20 }
}
```
```

– **Family templates**: `squat\_pattern`, `hinge`, `lunge`,  
`push\_horizontal`, `push\_vertical`, `pull`, `core\_isometric`  
(plank/wall-sit family – time-based FSM, not rep-based), `core\_dynamic`,  
`jump\_plyo`, `mobility\_stretch` (hold + ROM), `carry\_gait`. Each template  
supplies the FSM shape and default fault set; a concrete exercise is a

template + parameter overrides. 59 exercises  $\approx$  11 templates  $\times$  parameter files – a per-exercise effort of **\*\*hours, not weeks\*\***, once the template exists.

- **\*\*Delivered as a remote bundle\*\***: `GET /v1/exercise-definitions?since=<version>` (ETag-cached, signed). Clients cache the bundle locally; new/updated exercises appear in installed mobile apps **\*\*without a release\*\***. Definitions carry `minEngineVersion` so an old client simply doesn't show an exercise it can't run yet.
- **\*\*Message keys, not strings\*\*** (`fault.squat.depth`)  $\rightarrow$  localization (English/Hindi/Assamese) and voice cues come free (§21).
- Your existing `\_analyze\_squat` / jump / chair logic and the researched thresholds translate 1:1 into the first three definitions + the `standing\_baseline` calibration module. **\*\*Nothing you validated is lost.\*\***

### ### 5.3 What the server receives (the entire workout data contract)

At set end (or on app close), the client POSTs `POST /v1/workouts/sync` with an `Idempotency-Key`:

```
```jsonc
{ "workoutId": "uuid-client-generated", "startedAt": "...", "platform":
"web",
  "engineVersion": "1.3.0", "defsVersion": 41,
  "sets": [{
    "exercise": "squat", "setIndex": 1, "reps": 12, "durationMs": 48000,
    "avgFormScore": 84, "repScores": [90,88,...],
    "faultCounts": { "shallow_depth": 2, "knee_valgus": 1 },
    "tempoMsAvg": 3900, "romStats": { "kneeMinAvgDeg": 96 }, "view":
"side"
  }],
  "traceSample": null    // occasionally a compressed keypoint clip – see
§14
}
```
```

~1-3 KB per workout instead of ~5-10 MB of streamed keypoints. Offline clients queue summaries in local storage/SQLite and sync later; idempotency keys make retries safe. **\*\*This endpoint replaces `pose\_ws.py` entirely.\*\***

### ### 5.4 The golden-trace test harness — how "no bugs" actually happens

Add a dev-mode "record session" toggle that writes raw `PoseFrame` streams to JSONL. Every exercise ships with 3-6 recorded real traces (good form, each major fault, edge cases like partial visibility). CI replays every trace through the engine and asserts exact rep counts, fault sets, and score ranges. Any threshold tweak that would break squat counting fails the build *before* a gym member ever sees it. This harness is also your **\*\*parity gate\*\*** for the Python→TS port: record sessions against the current server analyzer, then require the TS engine to match. Budget ~2 days to build the harness; it pays for itself the first week.

---

## ## 6. Backend — the modular monolith

**\*\*Stack:\*\*** Node 22 · Fastify · TypeScript · Zod (validation + OpenAPI generation) · Drizzle ORM (SQL-first, typed migrations) · BullMQ · Pino structured logs. One Docker image runs in two modes: `api` (HTTP) and `worker` (queues) — same code, different entrypoint.

### ### 6.1 Module map

| Module         | Responsibility                                                                                                                                                             | Notes                                  |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------|
| --- --- ---    |                                                                                                                                                                            |                                        |
| `auth`         | email/password (argon2id), Google OAuth, JWT access (15 min) + rotating refresh tokens (httpOnly cookie on web, SecureStore on mobile), password reset, session revocation | Port of `backend-auth`, minus Mongoose |
| `users`        | profile, preferences, units, locale, onboarding data, account deletion (DPDP)                                                                                              |                                        |
| `gyms`         | gym CRUD, join codes (rotate/expire), <b>**seat enforcement**</b> , staff roles, branding                                                                                  | New                                    |
| `entitlements` | resolves user → effective plan (own sub > gym membership > free); one function the whole codebase calls                                                                    | New; the keystone                      |
| `quotas`       | generalizes your `quotas.py`: limits read from plan entitlements; Redis atomic counters; fail-open/closed per feature (keep your philosophy verbatim)                      | Port + generalize                      |

| `billing` | plans, subscriptions, Razorpay + Stripe + RevenueCat webhook normalization, invoices, dunning, trials | New (\$10) |

| `exercises` | catalog + **\*\*definition bundle publishing/versioning\*\*** | Replaces `exercises.py` + `seed\_exercises.py` |

| `workouts` | `/sync` endpoint, workout/set/rep-aggregate storage, history, PRs | Replaces `workouts.py` + `pose\_ws.py` |

| `gamification` | streaks (+freeze tokens), XP, badges, challenges, leaderboards (Redis ZSET + weekly snapshot) | Port `badges.py`/`challenges.py` logic |

| `coach` | LLM gateway: Groq primary → OpenRouter fallback, prompt versioning, per-call token budget, exact-match answer cache, **\*\*cost logged per call\*\*** | Port; Chroma → `pgvector` in Postgres (one less service; your KB is 5 docs — pgvector is more than enough) |

| `nutrition` | meal photo pipeline: client-compressed upload → R2 presigned → worker → vision model → macros; USDA lookup; photo auto-delete after 30 days | Port `photo\_analyzer.py`/`usda.py` |

| `geo` | route generation via ORS, **\*\*two-layer geocode cache\*\*** (Redis LRU + Postgres persistent, keyed on coords rounded to 3 decimals ≈ 110 m), weather | Port `routing\_provider.py`; cache is mandatory |

| `reports` | monthly gym usage PDF + email, at-risk member computation | New (\$11) |

| `notifications` | Expo push tokens, email (Resend/SES), template registry, per-user daily caps | New |

| `webhooks` | signature verification, `webhook\_events` dedupe table, enqueue-then-ack pattern | New |

| `analytics` | server-side PostHog event emission for money/entitlement events | New |

| `admin` | your internal panel: gyms, cost dashboard, circuit-breaker status, definition publishing, user support | New; can be a simple protected SPA |

### ### 6.2 Conventions that keep the monolith modular

- Each module = `routes.ts` (thin), `service.ts` (logic), `repo.ts` (only file allowed to touch DB), `schemas.ts` (Zod), `events.ts`. Modules call each other **\*\*only\*\*** via service interfaces or domain events on an in-process emitter (bridged to BullMQ for async) — never reach into another module's tables. This discipline is what lets you extract a module into its own service in 2030 without archaeology.

- Every route: Zod-validated input, entitlement check, quota check where metered, rate limit (per-user + per-IP, Redis). OpenAPI spec is generated from the Zod schemas; `packages/shared` exports the typed client from it.
- API versioned at `/v1`; cursor pagination; `Idempotency-Key` honored on `/workouts/sync` and all payment-adjacent POSTs.

---

## ## 7. Data architecture

### ### 7.1 PostgreSQL – system of record (core tables; full DDL in Part 4)

**\*\*Identity & tenancy:\*\*** `users`, `refresh\_tokens`, `gyms`, `gym\_codes` (code, seat-aware, expiry, rotation), `gym\_members` (unique gym+user, status, joined\_at – **\*\*a row = a seat\*\***), `gym\_staff` (owner/manager/trainer).

**\*\*Money:\*\*** `plans` (code, audience consumer|gym, price\_minor, currency, interval, `seat\_cap`, `entitlements JSONB`, active), `subscriptions` (owner\_type user|gym, owner\_id, plan\_id, status trialing|active|past\_due|canceled, trial\_ends\_at, period\_end, provider, provider\_ref), `invoices`, `webhook\_events` (provider+event\_id **\*\*unique\*\*** – dedupe), `api\_cost\_events` (user\_id, gym\_id, feature, provider, units, cost\_micro, at) – powers the circuit breaker (\$9.3) and your margin dashboard.

**\*\*Product:\*\*** `exercises` (catalog: slug, family, difficulty, equipment, muscles, status live|beta|dev), `exercise\_definitions` (exercise\_id, version, `definition JSONB`, min\_engine\_version, published\_at), `workouts`, `workout\_sets` (per-set aggregates incl. `fault\_counts JSONB`, `rep\_scores smallint[]`), `runs` (mobile: distance, duration, polyline, splits), `meal\_logs`, `coach\_threads`/`coach\_messages` (with token+cost columns), `streaks`, `user\_achievements`, `challenges`/`challenge\_participants`, `leaderboard\_snapshots` (weekly persistence of Redis boards), `usage\_daily` (nightly rollup of Redis quota counters – feeds gym reports & analytics), `gym\_usage\_reports`, `push\_tokens`, `audit\_log`, `feature\_flags`, `trace\_samples` (R2 key + verdict, \$14).

Design notes: per-rep detail lives as arrays/JSONB *\*inside\** `workout\_sets` (aggregates are what every screen reads) – no billion-row `rep\_events` table until a real need appears. `workouts` is partitioned by month from day one (cheap now, painful later). Nightly job prunes free-tier history >90 days per plan entitlement.

### ### 7.2 Redis map

`quota:{feature}:{user}:{yyyymmdd}` counters (your existing pattern) · `rl:\*` rate limits · `lb:gym:{id}:{month}:formscore` ZSETs (+ global boards) · `geo:{lat3}:{lng3}` geocode cache · `costs:gym:{id}:{month}` running counter for the circuit breaker · BullMQ queues (`emails`, `reports`, `webhooks`, `rollups`, `traces`). Redis is **\*\*rebuildable\*\*** – nothing lives only in Redis except in-flight counters that also roll up nightly to `usage\_daily`.

### ### 7.3 R2 buckets & lifecycle

`meal-photos/` (client-compressed ≤ 300 KB, EXIF-stripped, presigned PUT, **\*\*auto-delete 30 days\*\***) · `gym-assets/` (logos) · `share-cards/` (generated, 7-day lifecycle) · `reports/` (gym PDFs, 18 months) · `traces/` (sampled keypoint clips, 90 days). Presigned URLs only; bucket never public-listed.

### ### 7.4 Mongo → Postgres migration (one-time)

You have no paying users yet – migrate now while it's a script, not a project: (1) freeze writes, (2) export `users` (keep bcrypt hashes; verify-then-upgrade to argon2id on next login), workouts, gamification state via a Node script mapping ObjectId→UUID with a lookup table, (3) re-seed exercises from the new catalog, (4) run both reads for one week behind a flag if paranoid, (5) decommission Mongo. Chroma is not migrated – re-ingest the 5 knowledge docs into pgvector (minutes).

---

## ## 8. Tenancy, roles & entitlements

**\*\*Roles:\*\*** `user` · `gym\_owner` · `gym\_staff` (trainer/manager) · `admin` (you). A person can hold several (an owner is also a user with workouts).

**\*\*Join flow:\*\*** gym owner signs up → creates gym → gets a join code + QR poster (auto-generated PDF). Member enters code at registration or in Settings → a `gym\_members` row is created **\*\*iff seats remain\*\*** (`SELECT count(\*) ... FOR UPDATE` inside a transaction – the seat cap is enforced by the database, not by hope). Seat full → member sees "ask your gym to upgrade", owner sees an upgrade prompt: your cost ceiling doubles as your upsell mechanism, exactly as designed in our earlier discussion.

**\*\*Entitlement resolver – the one function everything calls:\*\***

```
...
effectivePlan(userId):
 own = active|trialing consumer subscription → its plan
 gym = active gym_members row whose gym has active|trialing subscription
 → gym member plan
 return highest of (own, gym, FREE) # cached in Redis 60s,
busted on billing/membership events
...
```

Every feature gate, quota limit, exercise-library filter, and history window reads `effectivePlan`. Nothing else in the codebase is allowed to reason about billing. When a member leaves a gym (or the gym churns), they silently degrade to `free` and get a win-back prompt for Pro – the B2B→B2C conversion funnel from our earlier discussion, implemented in one code path.

---

## ## 9. Plans, pricing & quotas (config, not code)

### ### 9.1 Consumer plans

|  |                                  |                             |                                                                                                                                                      |                                                                       |        |
|--|----------------------------------|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|--------|
|  |                                  | <b>**Free**</b> (permanent) |                                                                                                                                                      | <b>**Pro**</b> ₹149/mo · ₹999/yr (IN) / \$3.99/mo · \$29.99/yr (intl) |        |
|  | ---                              | ---                         | ---                                                                                                                                                  |                                                                       |        |
|  | Form-checked exercises           |                             | <b>**12 foundational**</b> (one per family: squat, chair squat, lunge, push-up, plank, glute bridge, ...) – <b>**unlimited sessions &amp; reps**</b> |                                                                       | All 59 |
|  | Workout logging, streaks, badges |                             | ✓ unlimited                                                                                                                                          |                                                                       | ✓      |

|                             |                                      |                                           |
|-----------------------------|--------------------------------------|-------------------------------------------|
| History                     | 90 days                              | Forever + advanced analytics, PR timeline |
| AI Coach                    | 5 questions/mo                       | 30/day                                    |
| Meal photo analysis         | 3/mo                                 | 8/day                                     |
| Route generation            | 2/mo                                 | 5/day                                     |
| Programs (multi-week plans) | 1 starter program                    | All                                       |
| Share cards                 | ✓ (small watermark → free marketing) | ✓ no watermark                            |
| Leaderboards                | view + gym boards                    | + global boards                           |

Rationale, per your instruction: everything with ~zero marginal cost (on-device form checking, logging, streaks) is free and **\*\*feels unlimited\*\*** – a free user doing 12 exercises every single day never hits a wall, so no "limit reached too soon" resentment. The three features that cost you real money per use are exactly the three that are metered, with a monthly *\*taste\** so free users experience them before paying. No consumer time-trial – the free tier IS the funnel and your live gym demo (this converts better for unknown solo apps than 7-day trials, per our earlier discussion).

### ### 9.2 Gym plans (tiered flat fee, seat-capped in code)

|          |       |                    |                                                 |
|----------|-------|--------------------|-------------------------------------------------|
| Plan     | Seats | Price              | Notes                                           |
| ---      | ---   | ---                | ---                                             |
| Starter  | ≤ 100 | ₹1,499/mo          | The Jorhat-gym "yes" – most Assam gyms fit here |
| Standard | ≤ 150 | ₹1,999/mo          |                                                 |
| Growth   | ≤ 400 | ₹3,499/mo          |                                                 |
| Scale    | 400+  | ₹4,999/mo → custom |                                                 |

Every gym member gets **\*\*Pro-level entitlements\*\*** (daily quotas above) through the gym's subscription. **\*\*Gym trial: 7 days, full features, self-serve\*\*** (your spec) – trial length is a column on `plans`, and you (admin) can issue extended **\*\*pilot codes\*\*** (30-60 days) for the first hand-sold gyms, reconciling the "free for 2 months for gyms #1-3" strategy without special-casing code. Day-5 trial email + dashboard countdown banner drive conversion. Fair-use policy text (30 coach / 8 photo / 5 route per member per day) ships in the gym agreement – the gym points members at the policy, not at you.

### ### 9.3 Quota engine & cost guardrails

- Limits come from `plans.entitlements` JSON – change a number in the DB, it's live. Your Redis `INCR`+TTL pattern and fail-open (coach) / fail-closed (vision, ORS) split are kept exactly.
- Friendly 429s with reset time; clients show "resets tomorrow" not an error.
- **Every** external API call writes an `api\_cost\_events` row and bumps `costs:gym:{id}:{month}`. Alert to you (email/Telegram) at ₹800/gym/month; soft-degrade (queue meal photos, keep coach on smallest model) only past 3× the gym's fee – you will likely never see it fire, but bankruptcy-by-API-bill is now mathematically impossible.
- Admin cost dashboard: spend by feature/provider/gym vs. revenue – your unit economics live on one screen.

---

## ## 10. Payments architecture

| Surface                                        | Provider                      | Mechanics                                                                                                                                                   |
|------------------------------------------------|-------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Consumer, web, India                           | <b>Razorpay Subscriptions</b> | (UPI Autopay / cards)                                                                                                                                       |
| You already know Razorpay from the travels app |                               |                                                                                                                                                             |
| Consumer, web, international                   | <b>Stripe Billing</b>         | (\$ prices)                                                                                                                                                 |
| Consumer, mobile                               | <b>RevenueCat</b>             | over Apple/Google IAP   Store rules require IAP for consumer digital subs; RevenueCat normalizes receipts + entitlements and posts webhooks like the others |
| Gyms (B2B)                                     | <b>Razorpay Subscriptions</b> | (web dashboard only – never in the mobile app, keeping B2B outside store commission entirely)   e-mandate/UPI Autopay; invoice PDFs auto-emailed            |

Principles: **entitlements are only ever read from our `subscriptions` table** – all three providers converge through the `webhooks` module (verify signature → dedupe on `webhook\_events` → enqueue → worker updates subscription state → busts entitlement cache → emits analytics event). Payment webhooks are the most bug-prone surface in this entire system; the dedupe-and-queue pattern plus idempotent handlers is non-negotiable. Dunning: `past\_due` grace of 5 days with email/push nudges, then downgrade to free (never delete data). Full flows, plan-change proration, and refund policy: **Part 5**.

---

## ## 11. Gym Dashboard v1 (owner-facing web)

Responsive web app (owners will open it on phones – do not build a native dashboard). Screen map – full spec with every widget's data contract is

### **\*\*Part 3\*\*:**

1. **\*\*Overview\*\*** – KPI cards: active members 7d/30d · workouts this week · **\*\*adoption % (active/seats)\*\*** · avg form score; 8-week trend chart; trial/renewal banner; **\*\*at-risk list\*\*** (members inactive 14+ days, with a one-tap "we miss you" push) – this list is the owner's churn weapon and your renewal argument in one widget.
2. **\*\*Members\*\*** – table (name, joined, last active, workouts 30d, avg form score), seat meter (e.g. 87/150), invite via code + QR-poster PDF download, remove member (frees seat).
3. **\*\*Leaderboard\*\*** – monthly form-score & volume boards; **\*\*TV mode\*\*** (fullscreen rotating URL for the gym's lobby screen); printable monthly poster PDF for the notice board. Costs you ~nothing; owners love it; it markets the app inside the gym for you.
4. **\*\*Reports\*\*** – auto-generated monthly PDF (actives, workouts, top members, trend) archived + emailed; this is the "X of your members used it, Y workouts logged" renewal ammunition from our earlier discussion.
5. **\*\*Billing\*\*** – plan & seat tier, trial countdown, invoices, upgrade (self-serve seat-tier bump), payment method.
6. **\*\*Settings\*\*** – gym profile/logo (appears on members' dashboards + share cards), join-code rotation, staff management, notification prefs.

Owner onboarding is a 5-minute wizard: gym name/city → logo → seats tier → payment (or trial) → print poster. **\*\*Time-to-first-member-joined is the metric that predicts gym retention\*\*** – instrument it.

---

## ## 12. Mobile app architecture (Expo React Native)

- **\*\*Build system:\*\*** Expo + **\*\*EAS dev-client / development builds – not Expo Go.\*\*** ⚠️ This changes your current workflow: camera frame processors and pose models are native modules that Expo Go cannot load. You'll install a custom dev build APK on your Android device once, then iterate with the same fast-refresh you have today. EAS Build handles native

compilation in the cloud (no local Android Studio pain); EAS Update gives OTA JS updates post-launch.

- **Pose pipeline:** `react-native-vision-camera` frame processors → BlazePose (via a MediaPipe RN wrapper or `react-native-fast-tf-lite` with the BlazePose model — run a 2-day spike, pick whichever hits the bar) → 33 landmarks → **the same** `@app/engine`. Acceptance bar: ≥20 fps sustained and thermally stable for a 20-minute session on a mid-range Android (test on your device + one ₹12k-class phone). Landmarks stay in worklet-land; only rep/fault events cross the JS bridge.
- **Offline-first:** `expo-sqlite` mirrors workouts/definitions; a sync queue flushes `SetSummary`s (idempotent, so retries are safe) and pulls the latest definition bundle on connectivity. A member in a basement gym with zero signal gets a **full** workout experience — a genuine differentiator in Tier-2/3 India.
- **Running (mobile-only):** `expo-location` background tracking + TaskManager, polyline recorded locally, route-name geocoding on save (cached, quota'd), splits/score computed on device (port `scoring.py` into `@app/engine`). Web keeps route *browsing* of past runs but hides live tracking behind a platform capability flag, replaced by an "available in the app" install prompt (this drives app installs — don't delete the web pages, flag them).
- **Everything else** (auth, coach, nutrition, gamification, dashboard) reuses `packages/shared` API client + React Query — the screens you already built in React translate to RN screen-by-screen.
- Payments: RevenueCat SDK; entitlements still resolved server-side. Push: `expo-notifications`. Crash/error: Sentry RN. Store note: on-device processing is also your App Store privacy story — "video never leaves your device."

## ## 13. Web app — what changes in the existing SPA

Keep React 18 + Vite (no Next.js migration — churn without payoff; add a small static marketing/SEO site later, separately). Changes: (1) `usePoseDetection` stops opening a WebSocket and instead feeds frames to the local `@app/engine` — the hook's public API barely changes, `ActiveWorkout.jsx` mostly doesn't notice; (2) new sync call on set end; (3) new pages: Paywall/Plans, Join-gym-code, Account/Billing; (4) gym dashboard app; (5) PWA manifest + service worker (installable, definition bundle cached offline); (6) delete `FormClassifier.js`/ONNX from the bundle (research track); (7) split the two giant pages when next touched.

---

## ## 14. Anti-cheat & leaderboard integrity (the price of D1)

Moving analysis client-side means a hostile client *could* POST fake summaries. Mitigations, layered:

1. **Server plausibility validation** on every sync: rep cadence within human bounds (from the definition's ``minRepMs`/`maxRepMs``), session duration vs. rep math, score distributions not impossibly perfect, daily volume sanity. Violations → accepted but flagged ``unverified`` (never a user-facing error – false positives must not punish real users).
2. **Sampled trace audits**: clients occasionally attach a compressed 10-second keypoint clip (~50 KB) with a set; a worker replays it through the *server's copy of the same engine* and compares. Mismatch → flag account. Sampling ~1 set per user-week costs pennies and makes systematic cheating detectable.
3. **Scope integrity where it matters**: gym leaderboards are socially self-policing (people know each other); **global** boards and any future prize challenges show verified users only. Personal history is never blocked – someone determined to lie to their own progress chart is welcome to.

This is the same trust model every client-side fitness app (step counters included) lives with; the summaries + sampled audits give you *more* verification than most.

---

## ## 15. Notifications & messaging

- **Push (Expo)** and **email (Resend or SES)** behind one ``notifications`` module with a template registry and per-user **daily cap + quiet hours** – engagement notifications capped at 1/day; nothing erodes retention like notification spam.
- Core sends: workout-streak nudge (only if streak  $\geq 3$  and about to break), weekly recap (stats + form-score delta), PR celebration, trial-day-5 (gyms), payment failed, monthly gym report to owners.

- **WhatsApp** (Business API via a BSP – Gupshup/Interakt) is Phase-2 but architected now: gym owners in Assam live on WhatsApp; monthly report + at-risk alerts via WhatsApp will outperform email dramatically. The `notifications` module treats it as one more channel adapter.

---

## ## 16. Analytics & product metrics

PostHog (cloud free tier) on all three surfaces + server-side for money events. Starter event taxonomy (name → key properties): `signup` (source, gym\_code?), `workout\_started`/`workout\_completed` (exercise count, platform), `exercise\_completed` (slug, reps, avg\_score), `paywall\_viewed` (trigger), `subscribe\_activated` (plan, provider), `coach\_msg\_sent`, `meal\_logged`, `gym\_code\_redeemed`, `gym\_trial\_started`/`gym\_converted`, `share\_card\_created`, `app\_installed\_from\_web\_prompt`. North-star:

**weekly active workout users**; guardrails: D7/D30 retention, free→Pro conversion, gym adoption %, gym 3-month logo retention. Ship the taxonomy with the first release – you cannot retrofit the funnel data you'll need for gym pitches.

## ## 17. Observability & reliability

- **Sentry** on api, worker, web, mobile (release-tagged; engine version attached to every event).

- **Pino structured logs** with request IDs → Better Stack/Axiom free tier; log every quota denial, circuit-breaker event, webhook, and sync rejection with reasons.

- **Uptime checks** (Better Stack) on `/health` (checks DB+Redis) and the definition-bundle endpoint.

- **Backups**: Neon PITR (continuous) + nightly `pg\_dump` to R2; **monthly restore drill into staging** – an untested backup is a rumor. Redis is rebuildable by design (\$7.2); R2 is versioned.

- Targets to hold yourself to: API p95 < 250 ms; sync success ≥ 99.9%; crash-free mobile sessions ≥ 99.5%. When a target is breached two weeks running, that – not vibes – is your signal to spend on infra.

## ## 18. Security & privacy

**\*\*Immediate (before any other work):\*\*** rotate every credential in the committed ``.env`` files (Groq, Google OAuth secret, ORS, JWT secrets, SMTP, Mongo); purge them from git history; add Gitleaks to CI.

Ongoing: argon2id passwords · short-lived JWT + rotating refresh with reuse detection · Zod on every input · per-user and per-IP rate limits (keep your ``trust proxy`` lesson) · helmet/CORS strict · presigned uploads with size/content-type limits · webhook signature verification everywhere · RBAC middleware from ``gym_staff`` roles · `audit_log` on money/admin actions · secrets via platform env (Doppler if it grows).

**\*\*Privacy is a selling point, not a chore:\*\*** video frames never leave the device — only stick-figure landmarks (sampled, consented) and numeric summaries. Put this sentence on the landing page, in the gym agreement, and in the app-store listing. India's **\*\*DPDP Act\*\*** compliance: consent screen at camera first-use, purpose limitation, in-app account deletion (already in ``users`` module), meal-photo auto-deletion (\$7.3), data-export endpoint.

## ## 19. Deployment, environments & cost

|                                 |                                                                                                                      |                                                                                        |  |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|--|
| Piece                           | Choice                                                                                                               | Monthly (early)                                                                        |  |
| ---                             | ---                                                                                                                  | ---                                                                                    |  |
| API + worker                    | 1× Hetzner VPS (CX32-class), Docker Compose: <code>`caddy`</code> (TLS) + <code>`api`</code> + <code>`worker`</code> | ~₹700-1,200                                                                            |  |
| Postgres                        | Neon (managed, PITR, branching for staging)                                                                          | ₹0 → ~₹1,700                                                                           |  |
| Redis                           | Upstash (pay-per-request)                                                                                            | ~₹0-400                                                                                |  |
| Web + dashboard                 | Vercel                                                                                                               | ₹0                                                                                     |  |
| Objects/CDN                     | Cloudflare R2 + CDN                                                                                                  | ~₹0-200                                                                                |  |
| Email / Push / Sentry / PostHog | free tiers                                                                                                           | ₹0                                                                                     |  |
| <b>**Total**</b>                | <b>**~ ₹1,000-3,500/mo**</b>                                                                                         | — profitable from gym #1-2, exactly as the earlier unit-economics discussion concluded |  |

**\*\*Environments:\*\*** ``dev`` (docker-compose: pg+redis local) → ``staging`` (Neon branch + tiny VPS or same VPS second compose project) → ``prod``. **\*\*CI (GitHub Actions):\*\*** typecheck → unit tests → **\*\*golden-trace engine tests\*\*** → Gitleaks → build images → migrate (Drizzle, expand-then-contract only) → deploy staging → manual promote to prod. Mobile: EAS Build + EAS Update channels (staging/prod).

## ## 20. Scale path — what changes, and the trigger for each

| Stage | Users | Change                                                                                                                                                           | Trigger                                     |
|-------|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| ---   | ---   | ---                                                                                                                                                              | ---                                         |
| 0     | 0-2k  | Ship as \$19. Nothing else.                                                                                                                                      | —                                           |
| 1     | ~10k  | Bigger VPS <b>**or**</b> 2× API containers behind Hetzner LB (stateless already — flip a switch); Neon autoscale; move worker to own container                   | p95 > 250 ms sustained or CPU > 70%         |
| 2     | ~50k  | Postgres read replica for dashboards/reports; CDN-cache definition bundle at edge; dedicated worker node                                                         | DB CPU > 60%; report queries visible in p95 |
| 3     | 100k+ | Partition-prune old workout partitions to cold storage; consider extracting `billing` + `coach` modules to services (boundaries already clean); multi-region CDN | Sustained growth + revenue to fund it       |

The honest big-tech secret: because the hot loop is on-device, **\*\*your server load grows with sign-ups, not with reps\*\***. A 100k-user fitness app here is a mid-size CRUD app anywhere else. Do not build Stage 3 at Stage 0.

---

## ## 21. Retention & growth features (ranked — impact ÷ effort)

Existing features all survive; these compound them.  = partially exists today (port it), ★ = new.

| #   | Feature                                                                                                                | Why it retains/attracts                                                                                                                                                | Effort                    |
|-----|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| --- | ---                                                                                                                    | ---                                                                                                                                                                    | ---                       |
| 1   | ★ <b>**Form Score™ + weekly delta**</b>                                                                                | Your proprietary metric — "84 → 88 this week" makes invisible progress visible; no competitor without pose tech can copy it. Anchor of dashboard, recaps, share cards. | S (data already computed) |
| 2   |  <b>**Streaks + freeze tokens**</b> | Streaks retain; freezes (earn 1/week of activity, auto-spend on a miss) prevent the rage-quit when life happens.                                                       | S                         |
| 3   | ★ <b>**Gym leaderboard + lobby TV mode + monthly poster**</b>                                                          | Retention theater inside the gym; markets the app to non-users on the gym floor; the owner's favorite feature.                                                         | S-M                       |

| 4 | ★ **\*\*Challenges\*\*** – personal (30-day squat), gym-wide, and **\*\*gym-vs-gym city leagues\*\*** | Social pressure = retention; gym-vs-gym gives owners bragging rights and gives you a B2B sales hook ("join the Jorhat league"). | M |

| 5 | ★ **\*\*WhatsApp-ready share cards + referral\*\*** (give-a-month/get-a-month) | Auto-generated workout/PR image sized for WhatsApp status – your zero-budget growth channel in India; watermark on free tier. | S-M |

| 6 | ★ **\*\*Programs\*\*** (4-week structured plans built from your 59) | People don't retain on exercises; they retain on *\*plans\** with a visible finish line. Reuses WorkoutBuilder. | M |

| 7 | ✅ **\*\*PR celebrations + badges\*\*** (port `badges.py`) | Cheap dopamine, real effect. Confetti on PR, milestone shares. | S |

| 8 | ★ **\*\*Weekly recap\*\*** (email/push; later WhatsApp) | Re-engagement rhythm; contains Form Score delta + streak status. | S |

| 9 | ★ **\*\*At-risk member alerts for gym owners\*\*** | Retention *\*for the gym\** = renewals *\*for you\**; unique B2B value no consumer app offers. | S (query + notification) |

| 10 | ★ **\*\*Assamese + Hindi localization\*\*** | Fault messages are already keys (\$5.2) → cheap; a real differentiator and demo-day wow in Tier-2/3 India. | M |

| 11 | ✅ **\*\*Voice coach\*\*** (port `voice.js`; cues from message keys) | Hands-free correction mid-set is the "it's actually watching me" moment. | S |

| 12 | ★ **\*\*Trainer mode (Phase 2):\*\*** trainers assign programs, view members' form trends; later, live-view a member's session (this is where a WebSocket *\*earns\** its way back) | Converts trainers from threat to champion – answers the #1 gym objection head-on. | L |

| 13 | ★ **\*\*Offline mode (mobile)\*\*** (\$12) | "Works in any basement gym" – say it in every pitch. | M (in mobile plan) |

| 14 | ★ **\*\*Rest timer + plate calculator + workout notes\*\*** | Small QoL that makes it the *\*only\** app open during a session. | S |

Sequencing: 1, 2, 5, 7, 8 ship with v1 (all Small); 3, 9 ship with the gym dashboard; 4, 6, 10, 11 in the following cycle; 12 after 5+ paying gyms ask for it.

---

## 22. Build sequence (solo-dev realistic; each phase ends demoable)

**\*\*Phase 0 – Foundation (Week 1).\*\*** Rotate secrets; purge history. Monorepo scaffold, CI (typecheck/tests/Gitleaks), Postgres schema v1 migrations, envs, Sentry, PostHog. ✓ *\*Done when: CI green, staging deploys on merge, `/health` is up.\**

**\*\*Phase 1 – The Engine (Weeks 2-4). The keystone phase.\*\*** Build `@app/engine` (normalize → view → signals → FSM → faults → score) + definition schema + **\*\*golden-trace harness\*\***. Record real traces against the current Python analyzer; port squat/jump-squat/chair-squat as definitions; require parity. Swap `usePoseDetection` to the local engine; implement `/v1/workouts/sync`; delete the WS path. ✓ *\*Done when: all traces pass in CI; a full workout works with the API server switched off, then syncs.\**

**\*\*Phase 2 – One backend (Weeks 4-6).\*\*** Auth module (port), users/workouts/progress/gamification modules, entitlement resolver, generalized quotas, Mongo→PG migration script, coach/nutrition/geo ports behind quota gateway with cost logging + geocode cache. Decommission both old backends. ✓ *\*Done when: web app runs 100% on the new API; old services are off; cost dashboard shows real numbers.\**

**\*\*Phase 3 – Money & gyms (Weeks 6-8).\*\*** Plans/subscriptions/webhooks (Razorpay + Stripe), consumer paywall + billing pages; gym onboarding wizard, codes + DB-enforced seats, staff roles; **\*\*Dashboard v1\*\*** (Overview, Members, Billing) + monthly report job + at-risk list; pilot codes. ✓ *\*Done when: a stranger can pay you for Pro, and a gym can self-serve trial-pay with members joining by code.\**

**\*\*Phase 4 – Exercise production line (Weeks 8-10, then ongoing).\*\*** Build the remaining family templates; then it's an authoring pipeline: pick exercise → parameterize template → record 3-6 traces → CI green → publish to bundle. Target **\*\*2-3 validated exercises/week\*\***; reach **\*\*10-12 solid\*\*** (the gym-demo bar from our earlier discussion) before pitching. Leaderboards + TV mode + share cards ship here. ✓ *\*Done when: 12 exercises live via remote bundle, none hand-coded.\**

**\*\*Phase 5 – Mobile (Weeks 10-14).\*\*** EAS dev-client, pose-pipeline spike (2 days, pick winner), screens ported on shared packages, offline sync, running feature, RevenueCat, closed Android test with your first gym's

members. ✓ *\*Done when: a member does a full offline workout on a mid-range Android at  $\geq 20$  fps and it syncs later; running records a real route.\**

**\*\*Phase 6 – Pilots & polish (ongoing).\*\*** 2-3 Jorhat gyms on pilot codes; watch adoption %, weekly recaps on, iterate on Dashboard from owner feedback; keep the exercise line running toward 59.

Order rationale: the engine (P1) de-risks everything – every later phase, the mobile app, and all 56 remaining exercises depend on it; money (P3) comes before exercise volume because our earlier analysis stands: gyms buy on 10 solid exercises, not 59, and you want billing battle-tested before the first renewal.

---

**## 23. What we spec next (one at a time, as you asked)**

| Part | Deliverable                                                                                                                                                                                                                               |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Form Engine & EDS – full spec: definition JSON-schema, signal library, FSM semantics, fault-DSL grammar, all 11 family templates, the complete 59-exercise mapping (template + parameters each), trace harness design, authoring workflow |
| 2    | Gym Dashboard – every screen, widget, query, and empty/edge state                                                                                                                                                                         |
| 3    | Database – full DDL, indexes, partitioning, Mongo→PG migration scripts                                                                                                                                                                    |
| 4    | Billing & entitlements – provider flows, webhook state machines, proration, dunning, edge cases                                                                                                                                           |
| 5    | Mobile – pose-pipeline spike plan, offline sync protocol, store-listing/privacy checklist                                                                                                                                                 |
| 6    | Retention playbook – exact mechanics, copy, and triggers for \$21 features                                                                                                                                                                |
| 7    | Ops runbook – deploy, incident basics, backup drills, launch checklist                                                                                                                                                                    |

Recommended next: **\*\*Part 2\*\*** – it's the keystone and the longest lead time.

*\*– End of Architecture v1.0 –\**

## # AI Home Gym – Part 2: The Form Engine & Exercise Definition System (EDS)

**\*\*Full engineering specification · v2.1 · follows Architecture v1 (\$5, \$22 Phase 1, \$23 Part 2)\*\***

**\*\*Audience:\*\*** Kd (solo developer, owner)

**\*\*Status:\*\*** This is the keystone spec. Every later deliverable – the mobile app, all remaining exercises, offline mode, anti-cheat, and the gym demo – depends on what is defined here. It is written to be followed without needing the v1 doc open, but it never contradicts v1.

**\*\*Changelog v2.1\*\*** (evidence-verified reconciliation): catalog corrected to the definitive **\*\*58 exercises\*\*** from `scripts/seed\_exercises.py` – media-alias ghosts removed (`soldierpress`, `hamstring` were Shoulder Press / Hamstring Stretch media, confirmed by decoding GIF frames), five genuinely missing entries added (Warrior II, Bridge Pose, Arm Circles, Mountain Pose, Brisk Walking), `tracking: "pose"|"timer"` field added to the schema, Appendix B converted to a resolved evidence log, \$9.1 gains the formal video-derived threshold protocol, \$8.2 gains the dead-ONNX cleanup item. Companion document: **\*\*Part 2B – The Trust Layer\*\*** (calories, meal-photo portions, recommendations, predictions).

---

## ## 0. What this document locks in place

Architecture v1 made two decisions this document now specifies completely:

**\*\*D1\*\*** – all pose analysis runs on-device (browser and phone), inside one shared, pure TypeScript package `@app/engine`. The server never sees keypoints during a workout; it receives only compact set summaries.

**\*\*D2\*\*** – exercises are data, not code. One generic engine interprets declarative Exercise Definitions. Shipping exercise #14 or #59 is publishing a config, not writing a Python function or deploying anything.

I have verified this spec line-by-line against your actual code. Every threshold your squat, jump-squat and chair-squat analyzers use today (`form\_analyzer.py`, `angles.py`, `rep\_counter.py`, the

`\_StandingCalibration` and chair-depth calibration in `pose\_ws.py`) appears in this document explicitly, mapped to the field of the new system that carries it. **\*\*Nothing you validated with real footage is lost or changed – it is translated.\*\*** Where this spec proposes numbers for the 50+ exercises that don't exist yet, they are clearly marked *\*starting values, to be tuned via recorded traces\** – never presented as validated.

One correction to v1 while we're here (small, but you should rotate these too): beyond the secrets v1 listed, the archive also contains real values for `VITE\_LOCATIONIQ\_KEY` (this one ships **\*\*inside the public browser bundle\*\*** – anyone can read it from your JS today), `RAPIDAPI\_KEY`, and `USDA\_API\_KEY`. Rotate all three along with the rest, and move geocoding behind the server `geo` module exactly as v1 §6.1 already plans.

---

## ## 1. Scope, non-goals, and the six invariants

### ### 1.1 In scope

The engine package (`packages/engine`), its input/output contracts, the eight-stage per-frame pipeline, the signal library, the calibration modules, the rep and hold state machines, the fault DSL, the scoring model, the Exercise Definition JSON schema, the 12 family templates, the complete mapping of your real catalog onto those templates, the golden-trace test harness, the Python→TypeScript parity protocol, and the authoring/publishing workflow that gets you from 3 exercises to 59.

### ### 1.2 Non-goals

No UI specification (the existing `ActiveWorkout` surface survives; it consumes engine events). No ML classifiers – the CTR-GCN/ST-GCN/YOLO work stays parked in `ml-training/` as a research track exactly as v1 §1.3 ruled; the rule engine is the product. No server code – the sync endpoint's contract is restated here only so the engine's output is unambiguous.

### ### 1.3 The six invariants (violating any of these is a build failure, not a style issue)

**\*\*I1 – Purity.\*\*** `@app/engine` imports nothing platform-specific: no DOM, no React, no React Native, no Node APIs, no timers, no `Date.now()`. Time enters only through frame timestamps. This is what lets the identical code run in the browser, in Hermes on a phone, and in the Node test runner.

**\*\*I2 – Determinism.\*\*** Same frame sequence in → same events out, bit-for-bit, on every platform, every run. No randomness, no wall-clock reads, no locale-dependent formatting. Determinism is what makes golden traces (§7) and server-side replay audits (v1 §14) possible at all.

**\*\*I3 – One canonical skeleton.\*\*** The engine consumes exactly the BlazePose 33-landmark layout, on every platform. If a future mobile pose provider emits a different layout, the *\*adapter\** converts it before the engine ever sees it; the engine never branches on platform.

**\*\*I4 – Versioned everything.\*\*** Every session records `engineVersion` (semver of the package) and `defsVersion` (bundle number) – both already appear in the v1 §5.3 sync payload. A definition declares `minEngineVersion`; a client whose engine is older simply doesn't offer that exercise. No silent behavioral drift, ever.

**\*\*I5 – Performance budget.\*\*** Per-frame evaluation (stages 2-8, i.e. everything after pose inference itself) must run in **\*\*≤ 3 ms p95 on a mid-range Android (₹12k class)\*\*** and allocate no growing memory across a 30-minute session. Pose inference (MediaPipe/BlazePose) is outside this budget – it dominates and is the pose provider's cost. The budget is enforced by a benchmark test in CI (§7.6), not by hope.

**\*\*I6 – Fail soft, never silently wrong.\*\*** Missing landmarks, NaN, dropped frames, fps collapse – every degraded input maps to a defined behavior in this spec (usually: hold state, don't count, surface a visibility hint). An exception escaping the engine is a P0 bug; an incorrectly counted rep because inputs were garbage is also a bug. The correct response to garbage is *\*no decision\**, exactly the principle your current `RepCounter.update()` already applies when the angle is `None`.

---

**## 2. Canonical data contracts**

### ### 2.1 `PoseFrame` – the engine's only input during a set

One frame from the pose provider:

```
```jsonc
{
  "t": 123456.7,           // monotonic ms since session start (float). NOT
                           wall-clock.
  "kp": [ [x, y, z, vis], /* x 33 */ ]
}
```
```

Landmark semantics (identical to what MediaPipe Tasks gives you today, and what your `KP` map in `angles.py` indexes):

| Field        | Range / convention                                                                                                                                                                                                                                         |
|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| `x`, `y`     | Normalized image coordinates in [0, 1]. Origin top-left, <b>**y increases downward**</b> (your elevation code already depends on this: "risen above standing" = y decreased).                                                                              |
| `z`          | BlazePose relative depth, roughly hip-origin, same scale as x. Used sparingly (§3.4) – it is the noisiest channel.                                                                                                                                         |
| `vis`        | Visibility/presence score [0, 1]. The engine's visibility gate (§3.2) uses your existing threshold of <b>**0.3**</b> .                                                                                                                                     |
| Index layout | BlazePose 33: 0 nose ... 11/12 shoulders, 13/14 elbows, 15/16 wrists, 23/24 hips, 25/26 knees, 27/28 ankles, 29/30 heels, 31/32 foot index. The engine ships this index map as a frozen constant; it is the same table your `angles.py` `KP` dict encodes. |

Frame-rate contract: the engine accepts **\*\*8–40 fps\*\*** and adapts (all temporal logic below is specified in milliseconds, then converted to frames using measured inter-frame deltas – never hardcoded frame counts at an assumed fps). The target remains ~15 fps analysis / ~30 fps overlay, exactly your current web throttling; those numbers now live in the *\*pose provider adapters\**, not in the engine.

### ### 2.2 Mirroring and left/right policy (this bites every camera app – pin it now)

- "left\_\*" always means the **\*\*person's anatomical left\*\***, which is MediaPipe's convention. It usually appears on the *\*right side of the image\** for a front-facing camera.
- Front cameras typically render a mirrored *\*preview\**. The rule: **\*\*mirror only the drawn overlay, never the coordinates.\*\*** The engine always receives un-mirrored provider coordinates. (Your current pipeline already behaves this way implicitly; making it a stated rule prevents the classic mobile bug where someone mirrors the landmark array to "fix" the overlay and silently flips every left/right and valgus sign.)
- Signals that live in image-space (valgus x-positions) keep your documented convention: signs are in image space, and the *\*baseline-relative delta\** is what carries meaning – which is precisely why your ``standing_baseline`` calibration design is correct and survives unchanged.

### ### 2.3 Session inputs

A session is created with: the resolved **\*\*ExerciseDefinition\*\*** (already parsed and lint-validated, §4), optional **\*\*carry-over calibration\*\*** from a previous set of the same exercise in the same workout (so set 2 doesn't recalibrate – your ``active_sessions`` reconnect-persistence behavior, reproduced locally), and an optional **\*\*user profile hint\*\*** block (reserved: height, mobility limitations – v2 territory, ignored by engine v1).

### ### 2.4 Outputs – three event levels plus one document

**\*\*FrameResult\*\*** (every analyzed frame; feeds the overlay and live UI):  
``phase``, ``repCount``, ``isActive``, ``view``, ``visibilityOk``, ``liveCue`` (nullable message key), ``signals`` (the small set the definition declares, for debug/telemetry overlays), ``calibrationState`` (``pending`` | ``ready``).

**\*\*RepEvent\*\*** (at the moment a rep is credited): ``repIndex``, ``score`` (0-100), ``faults`` (array of fault ids observed in that rep cycle), ``durationMs``, ``phaseTimings`` (ms in descent/bottom/ascent), ``romExtreme`` (e.g. min knee angle reached), ``view``.

**\*\*HoldTick / HoldEvent\*\*** (isometric family only): accumulated qualifying ms, in-band or out-of-band, ended-hold summary.

**\*\*SetSummary\*\*** (at set end – this is *\*the only thing that leaves the device\**, byte-compatible with v1 \$5.3):

```
```jsonc
{
  "exercise": "squat", "setIndex": 1, "reps": 12, "durationMs": 48000,
  "avgFormScore": 84, "repScores": [90, 88, 76, ...],
  "faultCounts": { "shallow_depth": 2, "knee_valgus": 1 },
  "tempoMsAvg": 3900,
  "romStats": { "metricMinAvg": 96 },
  "view": "side",
  "holdMs": null, // isometrics: qualifying hold time
  "calibration": { "usedStandingBaseline": true, "chairDepthTarget": null
},
  "engineVersion": "1.0.0", "definitionVersion": 5
}
```
```

Design note: per-rep scores stay as a small array inside the set (matches v1 \$7.1's "no billion-row rep\_events table"); everything a screen or a gym report needs is derivable from SetSummary.

---

### ## 3. The pipeline – eight stages, fully specified

Order per frame: **\*\*ingest** → **condition** → **view** → **signals** → **calibration** → **FSM** → **faults** → **score/emit.\*\*** Each stage below states its algorithm, its parameters with defaults, and its failure behavior. Parameters marked ⚙ are overridable per-definition; everything else is engine-global.

#### ### 3.1 Ingest & validation

Port of your `\_valid\_keypoints` verbatim: exactly 33 entries, each with ≥4 finite numbers (NaN check `v !== v`). Additionally: `t` must be finite and strictly greater than the previous frame's `t` (out-of-order frames are dropped, counted in a session diagnostic). An invalid frame is **\*\*dropped silently\*\*** – state holds, nothing counts, no event. Three consecutive invalid frames flip `visibilityOk=false` on FrameResult so the UI can show

"step back into frame" (your existing "cannot see your legs clearly" behavior, generalized).

### ### 3.2 Conditioning: smoothing + visibility gating

- **Angle smoothing:** rolling mean over a **7-frame window** – your ``RepCounter.angle_buffer`` value, kept. Implemented as a per-signal ring buffer; window is time-aware (falls back to fewer samples at low fps rather than stretching latency).
- **Two smoothing channels, deliberately:** FSM/fault evaluation reads *smoothed* signals; the **elevation signals used for airborne detection read raw** values with only a 3-frame spike filter. Reason: a 7-frame mean at 15 fps is ~470 ms of lag, and a jump-squat flight phase is often 300–500 ms – heavy smoothing would erase the very event you're detecting. Your current server code effectively gets away with this because elevation is computed per-frame from geometry; the engine makes the raw-path explicit so nobody "optimizes" it away later.
- **Visibility gating with hysteresis:** a landmark is *usable* when ``vis``  $\geq 0.30$  (your threshold) and becomes *unusable* only when ``vis``  $< 0.15$  – the 0.15 gap stops flicker at the boundary from toggling signals on/off every frame. A signal whose required landmarks are unusable returns ``null`` for the frame; consumers of ``null`` follow §3.6's no-decision rule.

### ### 3.3 View classifier

Direct port of your ``detect_view`` – it is measured against labeled clips and the clusters don't overlap, so it stays:

- Reference height = `|ankle_y - shoulder_y|` (distance-invariant normalizer).
- ``shoulder_width_norm`` and ``hip_width_norm`` = x-spans divided by that height.
- **side** if both  $< 0.15$  · **front** if shoulder  $> 0.20$  or hip  $> 0.15$  · else **unknown**. Requires shoulders, hips, one ankle visible; otherwise **unknown**.

Two engine-level additions:

1. **Hysteresis:** the reported view changes only after the new classification persists **10 consecutive frames** (~0.7 s at 15 fps).

Prevents mid-rep view flapping when someone rotates slightly, which would otherwise toggle view-scoped faults (valgus) on and off within one rep.

2. **\*\*View policy per definition\*\*** (🌀): ``views: ["front","side"]`` plus ``preferredView`` and ``requireView`` flags. If a definition *requires* a view (e.g. valgus coaching genuinely needs front), the UI gets a ``liveCue: "cue.turn_to_front"`` instead of silently skipping checks. ``quarter`` (~45°) is not a v1 output class – your measured clusters are binary + unknown; a quarter stance lands in ``unknown`` and simply gets the view-agnostic checks, which is safe. (Recorded as a v1.1 candidate, not scope creep now.)

### ### 3.4 Signal library v1

A *signal* is a named, typed, per-frame scalar with a declared formula, view validity, smoothing channel, and calibration dependency. Definitions declare which signals they need; the engine computes **\*\*only those\*\*** (perf budget I5). The v1 library – 22 signals – covers every exercise in §6. Formulas are exact where ported from your code.

| #     | Signal                                                                      | Formula / source                                                                                                                     | Valid views     | Needs calibration | Notes                                                                 |
|-------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|-----------------|-------------------|-----------------------------------------------------------------------|
| 1-6   | <code>`knee_L/R`</code> , <code>`hip_L/R`</code> , <code>`elbow_L/R`</code> | Interior angle at joint via your 3-point <code>`calculate_angle`</code> (arccos of normalized dot product, degrees)                  | any             | –                 | The workhorses. <code>`hip`</code> = shoulder-hip-knee.               |
| 7-8   | <code>`shoulder_L/R`</code>                                                 | Interior angle elbow-shoulder-hip                                                                                                    | any             | –                 | Drives raises, presses, jacks.                                        |
| 9     | <code>`knee_avg`</code>                                                     | Mean of visible knee angles (1 or 2)                                                                                                 | any             | –                 | Your <code>`_analyze_squat`</code> uses exactly this.                 |
| 10    | <code>`trunk_incline`</code>                                                | Angle of shoulder-hip vector from vertical, degrees; prefers left pair, falls back right – your <code>`incline_from_vertical`</code> | any (best side) | –                 | 0 = upright.                                                          |
| 11    | <code>`shin_incline`</code>                                                 | Knee-ankle vector from vertical, same helper                                                                                         | side            | –                 | Used in <i>relative</i> lean checks (trunk vs shin).                  |
| 12-13 | <code>`valgus_L/R`</code>                                                   | <code>`(knee_x - ankle_x) / hip_width`</code> – your validated-notebook formula, image-space sign                                    | front           | –                 | Raw stance value.                                                     |
| 14-15 | <code>`valgus_delta_L/R`</code>                                             | Raw minus <b>**standing-baseline**</b> value                                                                                         | front           | standing_baseline | Negative = inward drift. The only valgus numbers rules may reference. |

```

16	`hip_elevation`	`(baseline_hip_y - hip_y) / torso_height`, **raw channel**	any	standing_baseline	Positive = airborne rising. Your jump-squat signal, normalized exactly as captured by `_StandingCalibration`.		
17	`ankle_elevation`	Same construction on ankles, raw channel	any	standing_baseline	Confirms true flight (both feet leave).		
18	`body_line`	Angle at hip between shoulder→hip and hip→ankle vectors; 180 = straight body	side	-	Plank/push-up sag-or-pike. Your current plank rule (`left_hip` 160-180) is this signal by another name.		
19	`elbow_under_should`	`	elbow_x - shoulder_x	/ torso_height`	side	-	Plank stacking cue.
20	`cadence`	Cycle events per minute from the FSM ($3.6 cadence mode)	any	-	Derived, not geometric.		
21	`stillness`	Windowed (700 ms) mean displacement of hip+shoulder midpoints, torso-normalized	any	-	Hold quality for isometrics/stretch holds; also the "person is set up and ready" detector.		
22	`symmetry_knee` / `symmetry_elbow`	`	L - R	` of the smoothed pair, degrees	front	-	Uneven press/curl/squat cue; only evaluated when both sides usable.

```

Reliability annotations ship with the library (and drive the risk notes in \$6): `z` is used by **\*\*no\*\*** v1 signal (too noisy – rotation-type exercises are handled via x-span oscillation instead, see russian twist in \$6); prone/supine poses (superman, cobra, bridges) reduce landmark confidence, so their definitions require the side view and lean on hip/knee/shoulder angles that survive it.

Adding a signal later = adding one pure function + its row here + trace coverage. Signals are versioned with the engine (I4), not with definitions.

### ### 3.5 Calibration modules

Calibration exists so checks are *relative to this person's own body and setup* rather than absolute cutoffs – the single smartest property of your current squat system, generalized. Three modules in v1:

**\*\*C1 `standing\_baseline` – port, unchanged semantics.\*\*** Arms automatically when smoothed knee  $\geq$  **\*\*160°\*\*** (`STANDING\_KNEE\_MIN`); captures after **\*\*8**

**consecutive** qualifying frames; buffers reset on any non-qualifying frame  
– your exact `\_StandingCalibration` state machine. Captures: `valgus\_L/R`  
baselines (front view), `hip\_y`, `ankle\_y`, `shoulder\_y`, and  
`torso\_height = |shoulder\_y - hip\_y|`. Lifetime: the exercise session;  
**\*\*carried across sets\*\*** of the same exercise in the same workout (\$2.3),  
matching today's reconnect-surviving server sessions. Invalidations that  
force re-capture: view flips front↔side, or subject-lost for > 3 s (the  
person who walks back into frame may be standing elsewhere).

**\*\*C2 `adaptive\_target` – port of chair-depth calibration.\*\*** Observes the  
metric extreme (min knee angle) across the **\*\*first 2 completed reps\*\***,  
sets `session\_target` = mean of those extremes, clamped to a  
definition-declared range (⚙ chair squat: clamp [80°, 120°], fallback  
**\*\*100°\*\*** = your `CHAIR\_FALLBACK\_TARGET`). Until it fires, scoring uses the  
fallback – precisely today's early-rep behavior. Declared generically so  
any "target-depth, not max-depth" exercise (step-up height, wall-sit  
angle) can reuse it.

**\*\*C3 `floor\_reference` – new, tiny.\*\*** For supine/prone starts (glute  
bridge, superman, leg raises): captures resting `hip\_y` / `shoulder\_y`  
during the first **\*\*1 s of stillness\*\*** (signal 21 below threshold). Gives  
bridges and leg raises the same "relative to your own setup" quality –  
camera height and mat position stop mattering.

UI contract: `calibrationState: "pending"` on FrameResult until required  
modules are ready; the UI shows the "stand still facing the camera" beat  
your users already experience implicitly. Faults that depend on an unready  
calibration are **\*\*not evaluated\*\*** (no decision on missing data, I6) –  
depth and rep counting still work, exactly as today.

### ### 3.6 Rep & hold state machines

One generic FSM, four modes (⚙ `rep.mode`). All modes share the  
anti-garbage guards, which are straight ports of your `RepCounter`  
hardening:

- Null metric this frame → **\*\*hold state, count nothing, mark  
`isActive:false`\*\*** (your None-angle rule).
- Debounce: state must persist **\*\*≥ 3 frames down / ≥ 2 frames up\*\***  
(time-normalized at low fps).

- **Minimum rep interval 450 ms** engine-wide, and per-definition ``minRepMs` / `maxRepMs`` (⚙) on top – the same numbers the server later uses for plausibility validation (v1 \$14), so honest clients can't trip anti-cheat.

- **Bilateral gate** (⚙, on by default for two-leg knee metrics): the *other* knee must be  $\leq 150^\circ$  when both are visible – your one-legged-phantom-squat rejector, kept, including its fall-back-to-single-knee-when-occluded behavior.

**Mode A – `alternating\_threshold`** (squats, push-ups, curls, presses...). Metric + ``upAt`/`downAt`` hysteresis + ``countOn``: "up" | "down". Phase model derived for free: ``top`` ( $\geq \text{upAt}$ ), ``descent`` (falling between), ``bottom`` (rising-edge of local minimum, captured as the ROM-extreme event), ``ascent``. Faults may scope to phases; "at bottom" faults evaluate once per rep at the captured extreme (this is how depth scoring becomes per-rep instead of per-frame noise).

**Mode B – `hold`** (plank, wall sit, side plank, stretch holds). Enter qualifying hold when metric inside ``holdBand`` **and** ``stillness`` below threshold for ``enterMs`` (default 700 ms); accumulate qualifying ms; exit when out-of-band for ``exitMs`` (default 500 ms – brief wobbles don't reset a plank, which is today's biggest hold-UX complaint pattern in every competitor). Emits `HoldTicks` (UI timer) and per-hold summaries. "Reps" for isometrics = number of qualifying holds; the headline number is ``holdMs``.

**Mode C – `alternating\_sides`** (lunges, Bulgarian split squat, step-ups, cycling crunch). Two Mode-A sub-machines, one per side; frame routes to the side whose metric is *more flexed*; total = L + R with per-side counts kept (the UI can show 6+6). A side-switch mid-descent voids that partial rep (no double-count exploit, no confusion when someone alternates legs freely).

**Mode D – `cadence`** (jumping jacks, high knees, mountain climbers, skipping...). Counts *cycles* of a periodic signal: rising-edge crossings of the signal's own rolling midline, accepted only when peak-to-trough amplitude  $\geq \text{`minAmplitude`}$  (⚙) and instantaneous cadence within ``[minCadence, maxCadence]`` (⚙). Robust to the ROM sloppiness that's *normal* in conditioning moves, and cheap. Reports cadence (signal 20) for pace cues.

### ### 3.7 Fault rules – the DSL

A fault rule is data. Full shape:

```
```jsonc
{
  "id": "knee_valgus",
  "view": "front",                      // optional; omit = any view
  "phase": ["descent", "ascent"],       // optional; omit = any phase
  "when": "valgus_delta_min < -0.15",  // condition (grammar below)
  "sustainMs": 0,                       // condition must hold this long (0
= instant)
  "perRep": true,                       // fire at most once per rep cycle
  "severity": 20,                       // score deduction weight (§3.8)
  "severe": "valgus_delta_min < -0.30", // optional: forces rep
'incorrect' regardless of score
  "msg": "fault.squat.valgus",          // message key – localization &
voice for free
  "cueCooldownMs": 4000                 // don't re-speak/re-show within
this window
}
```
```

**\*\*Condition grammar\*\*** (kept deliberately small – every operator below is required by some exercise in §6, and nothing else is included):  
comparisons ``<` <= > >= ==` between a signal reference and a number;  
boolean `&&`, `||`, parentheses; signal refs may take per-rep aggregates  
`_min`, `_max`, `_avg` (evaluated at rep completion, e.g. `knee_avg_min > 130` = your shallow-depth check) or calibration refs (target, from C2, e.g. `knee_avg_min > target + 25` = your chair shallow-margin check).  
That's the whole grammar. It is parsed **once at definition load** into an expression tree – zero per-frame parsing (I5). The linter (§9.2) rejects unknown signals, view-invalid signals (valgus in a side-only definition), and unsatisfiable ranges at authoring time, not at runtime in a gym.`

**\*\*Runtime semantics.\*\*** Frame-scoped rules evaluate on smoothed signals each frame within their phase/view scope, honoring ``sustainMs`` (a lean must persist ~300 ms to fault – single-frame noise never speaks). Rep-scoped rules (`_min/_max/_avg`, ``perRep``) evaluate exactly once at rep completion. Fired faults join the current rep's fault set; ``severe``

conditions additionally mark the rep incorrect – the generalization of your `severe\_faults` list (trunk > 60° and valgus < -0.30 both port as `severe` expressions).

**\*\*Coaching output policy – port of your best UX decision.\*\*** At most **\*\*2 corrections\*\*** surfaced at a time, deduplicated, ordered by severity – exactly your `deduped[:2]`. Voice gets only the single worst, throttled by `cueCooldownMs`. Message *\*keys\**, not strings: the engine emits `fault.squat.depth`; the client's locale table renders English/Hindi/Assamese text and TTS. Appendix A seeds the key catalog from your current correction strings so nothing regresses.

### ### 3.8 Scoring

Two-layer model that reproduces your squat scorer exactly and generalizes it:

**\*\*Layer 1 – component curves.\*\*** A definition declares scoring components, each mapping a signal (or rep-aggregate) through a **\*\*piecewise-linear band curve\*\*** – the declarative form of your `\_score\_depth` / `\_score\_trunk\_lean` / `\_score\_valgus` functions. Example, squat depth, which is *\*numerically identical\** to `\_score\_depth`:

```
```jsonc
{ "component": "depth", "input": "knee_avg_min",
  "curve": [ [100, 100], [130, 30], [155, 0] ],    // ≤100°→100 pts ...
  "inactiveAbove": 155 }                          // standing = no depth
opinion (your None)
```
```

Trunk lean ports as `[[45,100],[75,0]]` (100 down to 0 across your 30° falloff); valgus as a curve on `|valgus\_delta\_min|`:  
`[[0.15,100],[0.30,0]]`, inactive for outward drift. Chair depth uses C2's `target`: full credit ≤ `target+10`, linear to 40 at `target+25`, then your low-credit tail – the three chair constants (`TOLERANCE 10`, `SHALLOW\_MARGIN 25`, standing cutoff 155) become curve parameters.

**\*\*Layer 2 – aggregation.\*\*** Per-rep score = **\*\*mean of active components\*\*** (your rule), **\*\*80 neutral\*\*** when none are active (your

standing-between-reps rule), floored at a definition `floor` (default 0; squat keeps behavior via curves). `formCorrect` per rep = score  $\geq$  **\*\*70\*\*** and no `severe` fault – verbatim. Set `avgFormScore` = mean of rep scores. Session **\*\*Form Score™\*\*** (the v1 \$21 retention anchor) = exposure-weighted mean across the workout – computed by the engine so web, mobile, and the server's audit replays can never disagree about the headline number.

Isometrics score differently (they have no reps): hold score = `% of hold time in-band` (0-100) with a stillness bonus band (up to +10, capped 100); definition declares the band. Wall-sit example in §6.

### ### 3.9 Emission & the set lifecycle

`startSet(definition, carryOverCalibration?) → feedFrame(PoseFrame) → endSet() : SetSummary`. `endSet` is idempotent; an abandoned set (app killed) is reconstructed from the last emitted RepEvents by the client shell – the engine additionally exposes `snapshot()` (plain-JSON state) every 5 s to the shell for exactly this, replacing the server-side session store's crash-resilience role.

---

## ## 4. The Exercise Definition schema

Field-by-field. One definition = one exercise = one JSON document, validated by the linter (§9.2) before it can be published. (Schema formalized as JSON-Schema/Zod in `packages/shared` – the table below is the human contract.)

| Field                                                    | Type / constraints      | Meaning                                                                                                               |
|----------------------------------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------------------|
| ---                                                      | ---                     | ---                                                                                                                   |
| `key`                                                    | slug, unique, immutable | e.g. `chair_squat`. Aliases list absorbs your existing plural keys (`chair_squats`) so old clients/URLs keep working. |
| `version`                                                | int, monotonic          | Bumped on any behavioral change. Appears in every SetSummary.                                                         |
| `minEngineVersion`                                       | semver                  | Client gate (I4).                                                                                                     |
| `family`                                                 | enum of §5              | Template this definition instantiates.                                                                                |
| `name`, `muscles`, `equipment`, `difficulty`, `mediaRef` | catalog metadata        | Carried for the library UI; not read by the engine.                                                                   |

```

| `views` | array: front/side | Allowed views. |
| `preferredView`, `requireView` | slug / bool | §3.3 policy;
`requireView` triggers the turn-cue. |
| `calibration` | array of C1/C2/C3 refs + params | e.g. `[{ "module":
"adaptive_target", "clamp": [80,120], "fallback": 100, "observeReps": 2
}]`. |
| `signals` | array of §3.4 names | Everything the engine must compute.
Linter cross-checks every rule/curve reference against this list. |
| `rep` | object | `mode` (A-D §3.6) + mode params: `metric`, `upAt`,
`downAt`, `countOn`, `minRepMs`, `maxRepMs`, `bilateralGate`, `holdBand`,
`enterMs`, `exitMs`, `cadenceSignal`, `minAmplitude`, `minCadence`,
`maxCadence`, `perSide`. |
| `phases` | array | Names for Mode A phases (UI + fault scoping). |
| `faults` | array of §3.7 rules | The coaching brain. |
| `scoring` | object | Components with curves (§3.8), `floor`, `neutral`
(default 80), `correctAt` (default 70), isometric band if Mode B. |
| `setup` | object | Pre-set guidance: `cameraHint` key ("place phone
sideways, 3 m away, full body visible"), `positionCheck` (a boolean signal
condition that must hold before the set arms – e.g. body horizontal for
planks). Kills the #1 real-world failure mode: wrong camera placement. |
| `safety` | object | `contraindicationNote` key + `maxRecommendedReps`
(UI nudge only, never a hard block). |
| `status` | draft / beta / live | Publishing state (§9.3). |

```

**\*\*Worked example – `squat` v5, expressing today's Python behavior exactly\*\*** (this is the parity target for §7.5; abbreviated to the load-bearing parts):

```

```jsonc
{
  "key": "squat", "version": 5, "minEngineVersion": "1.0.0", "family":
"squat_pattern",
  "tracking": "pose",
  "views": ["front", "side"], "preferredView": "side", "requireView":
false,
  "calibration": [{ "module": "standing_baseline" }],
  "signals": ["knee_avg", "knee_L", "knee_R", "trunk_incline",
    "valgus_delta_L", "valgus_delta_R"],
  "rep": { "mode": "alternating_threshold", "metric": "knee_avg",
    "upAt": 160, "downAt": 100, "countOn": "up",

```

```

        "minRepMs": 900, "maxRepMs": 12000, "bilateralGate": 150 },
    "phases": ["standing", "descent", "bottom", "ascent"],
    "faults": [
        { "id": "shallow_depth", "when": "knee_avg_min > 130", "perRep": true,
          "severity": 25, "msg": "fault.squat.depth" },
        { "id": "trunk_lean", "when": "trunk_incline > 45", "sustainMs": 300,
          "severe": "trunk_incline > 60", "severity": 20, "msg":
"fault.squat.lean" },
        { "id": "knee_valgus", "view": "front", "phase": ["descent","ascent"],
          "when": "valgus_delta_min < -0.15", "severe": "valgus_delta_min <
-0.30",
          "perRep": true, "severity": 20, "msg": "fault.squat.valgus" }
    ],
    "scoring": {
        "components": [
            { "component": "depth", "input": "knee_avg_min",
              "curve": [[100,100],[130,30],[155,0]], "inactiveAbove": 155 },
            { "component": "trunk", "input": "trunk_incline",
              "curve": [[45,100],[75,0]] },
            { "component": "valgus", "input": "abs(valgus_delta_min)", "view":
"front",
              "curve": [[0.15,100],[0.30,0]], "inactiveWhenPositiveDrift": true
            }
        ],
        "neutral": 80, "correctAt": 70
    },
    "setup": { "cameraHint": "setup.squat.camera" },
    "status": "live"
}
...

```

****`tracking` (added v1.1):**** ` "pose" | "timer"`, default ` "pose"`. Your seed already contains one non-camera entry (Brisk Walking, `ai\_supported: false`), so the schema must represent it honestly rather than pretending every catalog row is pose-tracked. ` "timer"` entries carry no `signals`/`fsm`/`faults`/`scoring` blocks – the linter *rejects* them if present – and the app renders duration logging instead of the camera flow. This is also the future home of GPS-tracked entries on mobile (` "gps"` reserved, not implemented). One enum field now prevents a class of "why is the camera opening for walking?" bugs later.

5. The 12 family templates

A template = a pre-filled definition (FSM mode, phase model, default fault set, required signals, calibration, view policy) exposing a small parameter surface. Authoring exercise #23 means picking its template and filling parameters – hours, not weeks. v1 sketched ~11 families; building the real mapping against your actual catalog (§6) refined that to ****12**** – the deltas: bridges split from hinge (supine FSM + C3, not standing), holds split from mobility (a plank and a hamstring stretch share an FSM but not a fault philosophy), single-joint isolations unified (curl/extension/raise/calf-raise are one template with the metric swapped), and a dedicated cadence family for conditioning moves.

#	Template	FSM mode	Default metric	Core default faults	Calib.	Views
F1	`squat_pattern`	A	`knee_avg` 160/100	shallow depth, trunk lean, valgus (front)	C1	front+side
F2	`hinge`	A	`trunk_incline` (hip-driven: down = incline rises while knees stay > 140)	knees-bend-too-much (turns it into a squat), ROM short, jerk (tempo min)	C1	**side req.**
F3	`bridge_supine`	A (inverted: countOn "up" = hips peak)	`hip_elevation` vs C3 floor	incomplete lockout (hip angle < 160 at top), feet drift, hold-at-top missing	C3	side req.
F4	`lunge_split`	C (per-side)	front `knee` 160/100	front knee past toe (shin incline > 25 side-view), torso lean, shallow back knee	C1	side pref.
F5	`push_horizontal`	A	`elbow_avg` 160/90	half reps (elbow_min > 110), hip sag (body_line < 160), hip pike (> 195 equiv.), flared symmetry	–	side req.
F6	`push_vertical`	A	`elbow_avg` 160/90	incomplete lockout, asymmetric press (symmetry_elbow > 15), trunk lean-back > 20	C1	front pref.
F7	`pull`	A	`elbow_avg` 160/70	partial ROM, kip (hip_elevation oscillation), chin-short	C1	front

```

| F8 | `isolation_single_joint` | A | configured joint (elbow / shoulder /
heel-elev.) | partial ROM both ends, swing (trunk_incline oscillation >
10), speed | C1 for elevation variants | front/side per exercise |
| F9 | `core_dynamic` | A or C | `hip_avg` or knee-to-chest proxy | ROM
short, momentum (maxCadence), lower-back arch (body_line on leg raises) |
C3 | side req. |
| F10 | `hold_isometric` | B | exercise band (body_line 160-180 plank;
knee 80-100 wall-sit) | out-of-band (implicit), sag/pike cues, elbow
stacking | C1/C3 | side (plank), front (wall-sit ok) |
| F11 | `cadence_conditioning` | D | exercise periodic signal | amplitude
too small ("knees higher"), cadence out of band | C1 | front pref. |
| F12 | `mobility_hold_flow` | B (or D for cat-cow) | pose-specific band,
generous | none scored harshly – coaching tone is encouragement + hold
timer; score = hold time attained | – | per pose |

```

Template defaults are starting values; every concrete exercise tunes via traces (§7). F12 deliberately has a different *product* posture: stretches are guided-and-timed, not graded – grading a hamstring stretch's ROM without knowing the user's history reads as punishment, and these are your retention-friendly rest-day content (v1 §21 #14 territory).

6. The complete catalog mapping – every exercise, its template, its starting parameters, its risks

Ground truth (verified, v1.1): the canonical catalog is `scripts/seed\_exercises.py`, which seeds **exactly 58 exercises** into Mongo – matching your dashboard count. The earlier "57" was an artifact of counting media *filenames*: several misspelled GIF basenames (`hillraise` → Heel Raises, `soldierpress2.gif` → Shoulder Press's second demo, `hamstring2.gif` → Hamstring Stretch's second demo, `truckjump` → Tuck Jumps) are media aliases inside `frontend/src/utils/exerciseMedia.js`, not separate exercises – confirmed by decoding the GIF frames themselves. Two seeded exercises (**Mountain Pose**, **Brisk Walking**) are hidden by the frontend's `REMOVED\_EXERCISES` set, so the visible library is 56; both remain in this mapping because they exist in the system of record and need an explicit migration decision (§6, Tier 3). `arnold\_shoulder\_press` has a rep-counter config but is **not** in the 58 – it's noted as a ready-made expansion candidate, not counted.

****Tiers:**** T1 = the 12-exercise gym-demo set (v1 Phase 4 bar) – chosen for landmark reliability, camera simplicity, and coverage of the movements a skeptical trainer will actually test. T2 = next 17. T3 = remainder.  = validated logic exists today and ports with its numbers.

****Tier 1 – the demo dozen****

Exercise	Family	Rep metric & thresholds	Primary faults (starting values)	View / Calib	Risk notes
squat 	F1	knee_avg 160/100, count up	depth >130 · lean >45 (severe 60) · valgus -0.15/-0.30	F+S / C1	None – parity target.
chair squat 	F1	knee 155/110	shallow > target+25 · lean: >50		
and trunk-shin >20 F+S / C1+C2 None – parity target.					
jump squat 	F1+airborne	knee 160/130	no-flight (hip_elev <0.15 in cycle) · stiff landing (knee >130 at touchdown) · shallow load (<40° flex) · lean 50(+shin 15)/55 front	F+S / C1	Raw-channel elevation (\$3.2) is mandatory.
push-up	F5	elbow 160/90 (your config)	half-rep elbow_min>110 · hip sag body_line<160 (your 160-180 rule) · pike	S req / –	Wrist landmarks jitter on the floor – metric uses elbows only.
wall push-up	F5	elbow 155/95	half-rep · hips-to-wall (trunk_incline < 10 means standing too upright)	S / –	
Beginner-critical for gym demos; easiest camera setup in the catalog.					
lunge	F4	front knee 160/100 (your config)	front-knee-past-toe shin>25 · torso lean >20 · back-knee shallow	S pref / C1	
Side-detection of <i>*which*</i> leg is forward comes free from Mode C routing.					
glute bridge	F3	hip_elevation vs floor, top = lockout	no lockout (hip angle <160 at peak) · partial height (<60% of calibrated best)	S req / C3	Supine landmark confidence drops – require side, mark first release beta.
plank	F10	hold: body_line 160-180 (your band)	sag <160 · pike >posture-max · elbow not under shoulder	S req / –	Your existing rule set maps 1:1 to the hold band.
wall sit	F10	hold: knee 80-100	too-high knee>110 sustained · heels lifting (ankle_elev)	F or S / C1	Very reliable; great demo.
bicep curl	F8 (elbow)	elbow 150/60 (your config)	partial ROM (min>70 or max<140, your 50-160 band) · swing trunk osc >10	F / C1	Dumbbell occlusion of wrist is fine – elbow drives everything.

| shoulder press | F6 | elbow 160/90 (your config) | lockout short
(max<160, your 80-170 band) · asym >15° · lean-back >20 | F / C1 | |
| high knees | F11 | cadence on knee_L/R alternation | amplitude: knee
must pass hip height ("higher!") · cadence <100/min | F / C1 | Amplitude
via hip-relative knee_y - visibility-robust. |

****Tier 2 - next 17**** (compressed: metric → faults → view · risk)

| Exercise | Family | Spec seed |

|---|---|---|

| deadlift (RDL pattern) | F2 | hip-hinge: trunk_incline rises to 60-90
while knee stays >140 → faults: knees-collapse-to-squat (knee<120
mid-rep), ROM short, tempo-jerk · ****S req**** · Honest limitation, stated
in-app: spine **rounding** is not directly measurable from 33 landmarks - we
coach hinge **pattern**, and say so. Never oversell; trainers respect the
honesty in a demo. |

| hip thrust | F3 | as glute bridge with bench setup hint; lockout hip
≥170 · S · bench occludes hips at bottom - positionCheck gates start. |

| bulgarian split squat | F4 | per-side, front knee 160/95; balance fault
via stillness spikes · S · rear-foot elevation makes ankle vis low -
metric ignores rear ankle. |

| step up | F4 | per-side knee 160/100 + hip_elevation gain; fault:
push-off-with-floor-leg (bilateral gate inverted) · S · needs C2-style
height target - reuse `adaptive\_target` on hip_elevation. |

| calf raises | F8 (heel-elev) | heel/ankle_elevation vs C1, up/down on
elevation band | partial raise, bounce (maxCadence) · S · smallest ROM in
catalog - raw channel + generous bands; beta until traces confirm. |

| crunch | F9 | shoulder-lift proxy (shoulder_y drop vs C3) or hip angle
140/110 · faults: neck-pull unobservable (excluded - don't fake it),
momentum cadence cap · S. |

| leg raises | F9 | hip angle 170/95 lying · lower-back arch via body_line
during descent · S / C3. |

| mountain climber | F11 | cadence on alternating knee-to-chest;
plank-line fault from F10 rules concurrently · S · the one exercise
combining two templates' fault sets - supported: faults are just rules. |

| jumping jack | F11 | cadence on wrist-above-head + stance width
oscillation; amplitude faults ("arms all the way up") · F. |

| superman | F9 (prone) | chest+thigh elevation vs C3 floor, hold-ish reps
· S req · prone = worst-case landmark confidence; ****beta****, generous
thresholds. |

| side plank | F10 | body_line lateral (shoulder-hip-ankle) 160-180,
per-side holds · S · elbow-under-shoulder check reused. |
| tricep dips | F6 | elbow 160/80; shoulder-shrug fault (shoulder_y rise)
· S · bench occlusion – camera hint critical. |
| lateral raises | F8 (shoulder) | shoulder abduction 20/80-90; fault:
shrug, swing, over-raise >100 · F. |
| russian twist | F9 (rotation) | oscillation of shoulder-line x-span
(side-to-side), Mode D counting; fault: feet-down (if variant), tempo · F
· rotation via x-span oscillation, ****not z**** (I-noisy) – twist **depth** is
approximate; framed as tempo/count coach. |
| tricep extension | F8 (elbow, overhead) | elbow 160/70 overhead;
elbow-flare via symmetry+elbow_x drift · S. |
| lunge jump | F4+D hybrid | per-side landing detection + flight (hip_elev
raw) · S · uses jump-squat's airborne module verbatim. |
| skipping rope | F11 | cadence on ankle/hip elevation micro-oscillation;
rope invisible – count jumps, coach rhythm only, never "form" · F · set
expectations in-app copy. |

****Tier 3 – remaining 29**** (one line each; 28 pose-tracked through the same
pipeline, 1 timer-only)

pull-up (F7 – hardest camera story in the catalog: bar height forces
low-angle camera; kip detection via hip_elevation oscillation; ship late,
beta, with explicit camera hint) · bench press (F5 side-view low camera;
bar path not visible – elbow ROM + tempo only, stated honestly) · pike
push-up (F6) · resistance band pull (F7, wrist-x span 0.4→1.1
torso-widths) · burpees (F11 composite cycle: floor→jump detection chain)
· plank jack (F11 + F10 concurrent) · skater jump (F11 lateral hip_x
oscillation) · tuck jump **(media alias `truckjump` – confirmed from GIF
frames)** (F11 + airborne + knee-to-chest amplitude) · jogging in place
(F11) · flutter kick (F9 cadence on alternating ankle_y, small amplitude –
beta) · bicycle crunch (F9 per-side) · seated leg raise (F9) · heel raise
**(media alias `hillraise` – GIF frames show unsupported standing heel
raise; distinct catalog entry from Calf Raises, which is the supported
variant)** (F8, elevation vs C1, generous bands, beta until traces) · ****arm
circles**** (F11/F8 hybrid: wrist circular oscillation via wrist_y + wrist_x
phase, Mode D cadence; amplitude fault "full circles" via wrist-path span;
F view – trivially reliable, good warm-up demo) · ****mountain pose**** (F12
posture hold: stillness score + trunk_incline < 5° + arms-at-side;
currently hidden by frontend `REMOVED\_EXERCISES` – **migration decision:**

reinstate as a 30s "posture reset" between mobility flows, or drop from the Postgres seed; both are one-line changes) · ****brisk walking**** (``tracking: "timer"`` – the catalog's one non-camera entry, already ``ai_supported: false`` in your seed; no pose definition; at migration either reclassify under the running/steps module (mobile GPS) or timer-only logging on web – do ****not**** delete: gym walking-warmup programs reference it) · ****mobility thirteen**** (F12): cat-cow (Mode D slow cycles on trunk flexion oscillation), child's pose, cobra, downward dog (body inverted-V: hip angle 60-100 band), ****bridge pose**** (supine backbend **hold** – same landmark story as glute bridge but F12 hold semantics, not F3 reps: hip_elevation ≥ calibrated band sustained; S req / C3; beta with superman), hamstring stretch **(its second demo GIF is the misnamed `hamstring2.gif` – one entry, not two)**, hip-flexor stretch, seated forward bend, shoulder stretch, world's greatest stretch (3-position flow = sequenced holds – the only **sequenced** definition; template supports ordered hold stages), tree pose (stillness-scored balance hold – great share-card moment), warrior I (front knee 100-130 + arms-overhead band), ****warrior II**** (front knee 100-130 + arms-horizontal band via wrist_y ≈ shoulder_y ±0.05 torso-heights + stance width > 1.4 shoulder-widths; F view – the width signal makes it **front**-view unlike Warrior I) – all: generous bands, timer-first UX, no harsh scoring.

Expansion candidate (not in the 58): ``arnold_shoulder_press`` – a validated rep-counter config (160/90) already exists with no catalog entry or media; adding it later is one seed row + one definition inheriting shoulder press with rotation unscored.

Coverage check: ****12 + 17 + 29 = 58 – every seeded exercise mapped, zero open slots.**** (Shoulder Press's strict variant and the "hamstring curl" that earlier drafts listed were media-alias ghosts, now removed; Warrior II, Bridge Pose, Arm Circles, Mountain Pose, and Brisk Walking were the genuinely missing five.) Live visible today = 56 (58 – the two ``REMOVED_EXERCISES``).

7. The golden-trace harness – how "no bugs" is actually engineered

This is the quality system. Budget ~2 days to build; it protects every exercise forever after.

7.1 Trace file format

One recorded session = one file: a JSON header + JSONL frames (streams cheaply, diffs cleanly in git):

```
```jsonc
// header line
{ "traceVersion": 1, "exercise": "squat", "recordedWith": { "engine":
"1.0.0", "defs": 41 },
 "device": "poco-x5", "platform": "web", "fps": 15.2, "view": "side",
 "label": "clean_10_reps",
 "expected": { "reps": 10, "faultsExact": {}, "scoreRange": [82, 95],
 "formCorrectAll": true } }
// then one PoseFrame per line
{ "t": 0.0, "kp": [...] }
{ "t": 66.9, "kp": [...] }
```
```

``expected`` is authored by **you watching the video once** and locking the truth – after that, CI is the reviewer. Raw video is never stored (privacy invariant); the trace is stick-figure data only, ~1-2 MB per 10-minute session uncompressed, ~150 KB gzipped in the repo.

7.2 Recording mode

A dev-flag in the web app (and later mobile) adds a "record trace" toggle to ActiveWorkout: it tees the exact PoseFrames the engine consumes into a downloadable file. Because it captures **engine input**, a trace recorded on web replays identically on mobile CI – that's I2 and I3 doing their job.

7.3 Required fixture matrix – per exercise, before it may leave ``beta``

At least ****6 traces****: (1) clean set, 8-12 reps · (2-3) one trace per **major** fault, deliberately performed · (4) occlusion/interruption – walk partially out of frame mid-set, return · (5) wrong-view attempt (front-only exercise done side-on) – expected: cue fires, nothing miscounts · (6) speed extremes – rushed reps and grinding-slow reps. Isometrics swap (6) for a wobble-and-recover hold. The matrix is a checklist in the authoring workflow (§9.1), not folklore.

7.4 Assertions & tolerances

Rep count: ****exact****. Fault multiset: ****exact**** for scripted-fault traces; clean traces assert **empty**. Scores: within the trace's declared ``scoreRange`` (± 5 -point authoring slack absorbs smoothing-window edge effects). Hold time: ± 700 ms. Phase sequences: optionally asserted for FSM-sensitive traces. A failing assertion names the first divergent frame — debugging is "open frame 412", not archaeology.

7.5 The parity protocol (Python → TypeScript port gate)

Before ``pose_ws.py`` is decommissioned: (1) run the **current** system in a capture mode that logs, per frame, both the incoming keypoints and the server's full response; (2) record **** ≥ 6 squat, 4 jump-squat, 4 chair-squat**** real sessions this way, across front and side views, including your known real footage cases (the chair-squat clips bottoming at $\sim 62^\circ$ and $\sim 71\text{--}90^\circ$ that your comments cite); (3) these become traces whose ``expected`` values are **the Python outputs**; (4) the TS engine must match rep counts exactly and scores within ± 3 before the port is called done. This converts "I think the port is faithful" into a green checkmark.

7.6 CI wiring & the performance gate

Every push: lint definitions → unit tests (signal formulas against hand-computed geometry fixtures) → ****replay all traces**** → perf bench (replay a 10-minute trace in a tight loop; assert p95 frame cost ≤ 3 ms scaled to the CI baseline machine, and stable heap) → fuzz pass (random NaN injection, landmark dropout bursts, fps jitter 8-40 on a clean trace; assert: no throw, no count **increase**, recovery after signal returns). A threshold tweak that would break squat counting fails the build before any gym member ever sees it — this, concretely, is the "no bug" machinery.

8. Porting plan — nothing validated gets lost

8.1 Constant-preservation table (the audit trail for the port PR)

| | Today (file · constant) | Value | Lands in | |
|--|-------------------------|-------|----------|--|
|--|-------------------------|-------|----------|--|

```

|---|---|---|
| rep_counter squat up/down | 160 / 100 | squat `rep.upAt/downAt` |
| rep_counter jump squat down | 130 | jump_squat `rep.downAt` (+ rationale
comment carried into def description) |
rep_counter chair up/down	155 / 110	chair_squat `rep`
smoothing window	7 frames	engine conditioning (§3.2)
min down / up frames	3 / 2	FSM debounce defaults
bilateral engage angle	150°	`bilateralGate` default
min rep interval	0.45 s	engine floor; defs add `minRepMs: 900`
SQUAT_STANDING_KNEE_MIN / SHALLOW / PARALLEL	155 / 130 / 100	depth
curve `[[100,100],[130,30],[155,0]]`		
SQUAT_TRUNK_LEAN_MAX (+severe)	45 (+15→60)	trunk fault
`when`/`severe` + curve `[[45,100],[75,0]]`		
VALGUS flag / severe	0.15 / 0.30	valgus fault + curve
neutral score / correct bar / corrections cap	80 / ≥70 / 2	scoring
`neutral`, `correctAt`; coaching policy		
_StandingCalibration frames / knee min	8 / 160°	C1 params
CHAIR fallback / tolerance / shallow margin / standing	100 / 10 / 25 /	
155	C2 fallback + chair curves	
CHAIR trunk absolute / shin delta	50 / 20	chair lean fault (relative
check)		
JUMP airborne elevation / stiff landing / loading depth / lean S+F		
0.15 / 130 / 140 / 50+15, 55	jump_squat airborne module + faults	
visibility threshold	0.3	conditioning gate (+0.15 hysteresis floor,
new)		
view thresholds	side <0.15/<0.15 · front >0.20 or >0.15	view
classifier (§3.3)		
FORM_RULES bands (push-up, lunge, plank, curls, presses)	as coded	
T1/T2 definitions' starting fault bands (§6) |

```

8.2 Sequence (fits v1 Phase 1, weeks 2-4)

Week 2: engine skeleton – contracts, conditioning, view, signals 1-17, C1, Mode A, DSL parser, scorer; harness built; parity traces recorded **first** (while the Python path still runs). Week 3: squat/jump/chair definitions to parity green; C2, airborne module; web `usePoseDetection` swapped to local engine behind a flag (the hook's public API barely changes – `ActiveWorkout` mostly doesn't notice); `/v1/workouts/sync` live. Week 4: Modes B-D, C3, signals 18-22; T1's nine new exercises drafted to `beta`;

flag flipped, WS path deleted. Exit = v1 Phase 1's acceptance line: full workout with the API server switched off, then a clean sync.

****Cleanup item (verified dead code, v1.1):****

`frontend/src/ai/FormClassifier.js` and the 17 MB
`public/models/ctr\_gcn\_clean\_ensemble\_quant.onnx` it loads are imported by
****nothing**** in the app – a remnant of the ML-classifier path you correctly abandoned for rule-based (the right call: no server inference cost, no dataset-limitation ceiling, deterministic and auditable). Delete both plus the `onnxruntime-web` dependency in Week 2. It costs nothing at runtime today but is 17 MB of deploy weight, a misleading signal to future-you, and a dependency with its own security-update surface.

9. Definition lifecycle: authoring → publishing → rollback

9.1 Authoring workflow (the production line)

Pick exercise → copy family template → fill parameters (\$6 row is the seed) → derive/verify thresholds via the protocol below → self-record the \$7.3 fixture matrix → adjust until traces green locally → PR (definitions are code-reviewed data; the diff *is* the behavioral change) → CI green → status `beta`.

****The threshold-derivation protocol (v1.1 – your existing method, formalized and upgraded).**** The method that produced the validated squat/chair/jump thresholds – extract joint angles from good-form videos in Colab, cross-check against published exercise research – is the right method; MediaPipe-measured numbers beat textbook numbers because they include MediaPipe's own systematic biases. Keep it, with four upgrades that fix its blind spots:

1. ****3–5 good-form videos, not 2**** – different body proportions (limb ratios, torso length) shift measured angles by 5–15° for the *same* quality of movement. Two videos give you the center of the distribution; they cannot give you its width. From 3–5 sources, take the ****p10–p90 band**** of the target metric at each phase (bottom, lockout, etc.), not a single value. The pass threshold sits at the *permissive* edge of the band; the coaching-cue threshold sits at the strict edge. Where a

subject's own standing baseline matters more than the population (valgus, chair depth), the C1/C2 calibration modules absorb the variance – that's what they're for.

2. ****2-3 deliberate-fault videos per fault rule**** – record yourself intentionally shallow, intentionally leaning, intentionally caving the knees. A fault threshold set only from good reps is a guess about where "bad" begins; a threshold set with true fault positives is a measurement. Every fault rule in a definition must have at least one trace where it fires and one where it correctly stays silent (this is already the §7.3 fixture requirement – the fault videos are those fixtures).

3. ****Research pass is corroboration, not source**** – the desk-research step (published ROM norms, coaching standards) stays, but its role is to **sanity-check** the video-derived band, flag safety-relevant limits (e.g., knee-past-toe conventions), and supply coaching language. When research and your measured band disagree, trust the measurement and write one line in the definition PR explaining the delta – that line is gold six months later.

4. ****Numbers land in the definition JSON, never in code**** – the Colab notebook's output is a filled §4 definition (plus its band evidence pasted into the PR description). The same session's videos become the golden traces. One recording session per exercise therefore yields the thresholds **and** the regression fixtures – that's the whole production line for the 2-3/week cadence in §10.

9.2 The linter (runs in CI and on publish)

Rejects: unknown signal references; rules/components referencing signals absent from `signals`; view-scoped rules in definitions that exclude that view; curves not monotonic in x; `upAt ≤ downAt`; hold bands inverted; missing `msg` keys or keys absent from the locale table; `severe` without a base fault; missing camera hint; fixture matrix incomplete for a `live` promotion. Every rejection message says which field and why – authoring stays a one-person-friendly loop.

9.3 Publishing & staged rollout

Definitions live in Postgres (`exercise\_definitions`, v1 §7.1) and ship as the ****signed, ETag-cached bundle**** (v1 §5.2): `beta` definitions are included only for flagged users (you + consenting pilot-gym members – one feature flag); `live` for everyone. Promotion `beta → live` requires:

fixture matrix complete, ≥ 1 week of beta telemetry with fault-rate and reps-per-set distributions inside expected bands (a fault firing on 95% of reps means the threshold is wrong, not that everyone squats badly – the telemetry check catches exactly this). Rollback = repoint the bundle to the previous version row; clients pick it up on next fetch; no deploy, no app-store anything, in either direction.

9.4 Post-publish telemetry to watch (per definition, on your admin panel)

Fault-rate per fault id (distribution, not average) · reps-per-set distribution vs. family norm · % sessions ending < 60 s (rage-quit proxy – usually a setup/camera problem) · view distribution vs. `preferredView` · score histogram shape. These five catch every bad-threshold incident class I can anticipate, cheaply.

10. Production cadence, and definitions of done

****Cadence:**** with templates and harness in place, budget ****2–3 exercises/week**** sustainably (each $\approx$ half a day of parameterization + one recording session + tuning). T1's nine new exercises land inside v1 Phase 4's window; T2 completes ~ 5 weeks later; the full 59 well inside a quarter – while the gym pilots run on T1, which is the point: gyms buy on 12 solid, not 59 (unchanged conclusion from the business discussion).

****Definition of done – engine v1:**** all \$1.3 invariants enforced in CI · parity traces green (\$7.5) · perf gate green on the mid-Android profile · fuzz pass green · web app runs a full offline workout through the local engine · sync payload byte-matches \$2.4.

****Definition of done – any exercise:**** linter green · fixture matrix complete and green · beta telemetry week clean · camera-hint copy + all message keys in EN (HI/AS keys may lag but must exist) · \$6 risk note either resolved or surfaced as honest in-app copy.

Appendix A – Message-key seed (from your current strings, so nothing regresses)

```
`fault.squat.depth` "Squat lower – aim to get your hips level with your knees" · `fault.squat.lean` "Keep your chest up – you're leaning too far forward" · `fault.squat.valgus` "Push your knees out – don't let them cave inward" · `fault.body.visibility` "Cannot see your legs clearly – step back so your full body is in frame" · `fault.pushup.depth` "Lower your chest closer to the floor" · `fault.pushup.line` "Keep your body in a straight line" · `fault.pushup.sag` "Don't let your hips sag" · `fault.lunge.frontknee` "Bend your front knee to 90 degrees" · `fault.lunge.backknee` "Bend your back knee toward the floor" · `fault.lunge.torso` "Keep your torso upright" · `fault.plank.hips` "Raise your hips – keep body straight" / "Don't let your hips drop" · `fault.plank.elbows` "Keep elbows directly under shoulders" · `fault.curl.rom` "Full range of motion – curl all the way up" · `fault.press.lockout` "Press all the way up" · plus new `cue.turn_to_front`, `cue.turn_to_side`, `setup.<exercise>.camera` family. Hindi/Assamese columns are a translation task, not an engineering one – schedule with a native speaker before T1 goes to pilot gyms (it's a demo-day differentiator, v1 $21 #10).
```

Appendix B – Reconciliation log (RESOLVED, v1.1 – evidence-verified against your repo)

All six questions closed. Method: decoded frames from every ambiguous GIF, cross-referenced ``frontend/src/utils/exerciseMedia.js`` (the name-media map), and treated ``scripts/seed_exercises.py`` (58 entries, unique-name index) as the canonical catalog.

1. ``hillraise`` → ****Heel Raises**** – real catalog entry (seed line ~1424). GIF frames show an unsupported standing heel raise; ``exerciseMedia.js`` maps the files to the ``'heel raises'`` key. Distinct from ****Calf Raises**** (the supported/holding-on variant per its own GIF). **Not** hill sprints – no running content in the media; hill/incline running belongs to the GPS module if you ever want it.
2. ``hamstring2.gif`` → the ****second demo GIF of Hamstring Stretch****, not a separate hamstring-curl exercise (``'hamstring stretch': [..., 'hamstringstretch1.gif', 'hamstring2.gif']``). The "hamstring curl" row in earlier drafts was a ghost – removed.

3. `soldierpress` → ****media alias of Shoulder Press**** (`'shoulder press': ['soldierpress.jpg', 'shoulderpress1.gif', 'soldierpress2.gif']`). GIF confirms a standing overhead press. The separate "military press" row was a ghost – removed; the strict-form lean-back fault ($\leq 10^\circ$) is now simply part of the single Shoulder Press definition.

4. `truckjump` → ****Tuck Jumps****, real entry, confirmed from GIF frames (knees-to-chest airborne). Misspelling only.

5. Count: ****58, not 59 or 57.**** The seed script is unambiguous (58 inserts, unique index on `name`). Your dashboard figure was correct. The five entries earlier drafts missed: Warrior II, Bridge Pose, Arm Circles, Mountain Pose, Brisk Walking – all now mapped in §6. The frontend hides Mountain Pose + Brisk Walking via `REMOVED\_EXERCISES`, hence 56 visible. Nothing is missing; no exercises need inventing.

6. T1 demo-dozen: stands as specified in §6 (no objection raised). Swappable at any time – T1 membership is a tag on the definition, not engine structure.

****Two product decisions surfaced by the reconciliation (make them during Part 4's migration, both one-liners):**** (a) Mountain Pose – reinstate as a posture-reset hold or drop from the Postgres seed; (b) Brisk Walking – reclassify `tracking: "timer"` on web and fold into the running module on mobile, or timer-only forever.

**– End of Part 2. Next deliverables, one at a time as agreed (v1 §23):
Part 3 Gym Dashboard screen-by-screen · Part 4 Database DDL & migration ·
Part 5 Billing & webhooks · Part 6 Mobile deep spec · Part 7 Retention
playbook · Part 8 Ops runbook.**

AI Home Gym – Part 2B: The Trust Layer

****Calories · Meal-Photo Nutrition & the Portion Problem · Recommendations
· Predictions · The display standard that makes users believe your
numbers****

****Full engineering specification · v1.0 · companion to Part 2, follows
Architecture v1****

****Audience:**** Kd (solo developer, owner)

0. Why this document exists

You named the risk precisely: *"user will lose trust if the results are vague, not actual."* This document is the engineering answer. It audits every numeric surface in the app that claims to know something about the user's body or behavior – calories burned, meal nutrition, recommendations, predictions – against your actual code, gives a keep/extend/replace verdict for each, and specifies the redesigns.

One thesis governs everything here, and it is worth internalizing because every design decision below follows from it:

> ****Users forgive honest imprecision. They never forgive fake precision.****

"About 180–230 kcal" that is right every time builds trust. "212.4 kcal" that changes to 187.1 on refresh destroys it – even if 212.4 was, on average, closer. A fitness app cannot be a metabolic lab, and pretending to be one is the fastest way to look like a toy. The apps users trust for years (and the trainers and clinicians who recommend apps to **their** users) are the ones whose numbers are stable, explainable, and honestly bounded. That is a buildable engineering property, not a marketing wish – and for your B2B segments it is the sales feature: a physio or trainer will demo your app to a client, and the first overclaimed number ends the deal.

****Verdict summary (each detailed in its own section):****

| Surface | Code audited | Verdict |
|----------------------|--|--|
| Calories burned | <code>`ai/fitness/calories.py`</code> | **Keep the method – it's genuinely good. Extend coverage** (49 of 58 exercises currently fall to a fallback that misprices them) and fix the display. |
| Meal photo nutrition | <code>`ai/nutrition/photo_analyzer.py`,`routers/nutrition.py`</code> | **Redesign the pipeline.** The vision model currently both identifies <i>*and*</i> invents the numbers; portion size (your bowl problem) is unsolvable in that architecture. Same cost after redesign: still one vision call. |
| Recommendations | <code>`routers/recommendations.py`</code> | **Keep the architecture** (deterministic content-based scoring – right choice at your |

scale). ****Add**** the two missing layers: performance-awareness and recovery spacing. ****Cancel**** the planned "Neural CF" upgrade. |
| Predictions | `ai/predictions/forecaster.py` | ****Replace Prophet + per-request XGBoost**** with transparent statistics. As built, this module is the single biggest trust risk in the app – and the replacement is **more** accurate at your data sizes, not less. |

1. The four principles (engineering doctrine – every section below applies them)

****P1 – Separate perception from arithmetic.**** AI models (vision, pose) **identify** things: "that is dal, in a medium katori, about $\frac{3}{4}$ full." Deterministic code **computes** things: grams × per-gram nutrients = kcal. The moment a language model is allowed to do the arithmetic, you get numbers that are unreproducible, unexplainable, and unauditable. This is the exact principle your form engine already embodies (MediaPipe perceives landmarks; your rules compute verdicts) – Part 2B extends it to nutrition.

****P2 – Never fake precision.**** Every displayed estimate is a ****range**** with honest rounding, sized to the method's real uncertainty. Point values are only shown where they are actually known (a barcode-scanned label, a user-entered weight).

****P3 – Every number is explainable and versioned.**** Each numeric surface has a "How is this calculated?" affordance (one honest paragraph, 118n message keys per Part 2 Appendix A conventions). Every **stored** estimate carries a `calc\_version` so that improving a formula later never silently rewrites a user's history – their past stays as it was computed, and you can tell which era any number came from. (Schema consequence for Part 4: estimate-bearing tables get `calc\_version SMALLINT NOT NULL`.)

****P4 – User corrections are ground truth.**** Every estimate is editable, the correction is stored **alongside** the original (never overwriting it), and corrections feed back: per-user priors (their dishware, their portion tendencies) and a global evaluation set for regression-testing any prompt or table change. The user correcting you is the most valuable data the app collects – treat it as a gift, design the UI so it takes two seconds, and thank them with visibly better defaults next time.

2. Calories burned – keep the engine, finish the table, fix the display

2.1 Audit

`calories.py` is the best-engineered numeric module in the codebase. It already does the hard things right: standard MET formula ($\text{kcal} = \text{MET} \times \text{weight_kg} \times \text{hours}$) from the 2024 Adult Compendium; ****active/rest split**** so long rests aren't credited as exercise; weight normalization with a conservative 70 kg fallback; and – rarest of all – a docstring that explicitly **refuses** to invent precision (no form-score intensity multipliers, no age/sex corrections without the data to support them). Keep every one of those decisions; they carry into the TypeScript/Fastify port unchanged.

Two real gaps:

1. ****Coverage: 9 of 58 exercises have specific METs; the other 49 silently get the 5.0 fallback.**** This is your "vague numbers" fear already live in production: a 10-minute stretching flow gets credited at 5.0 MET when hatha-style holds are ~2.5 – the app reports roughly ****double**** the real burn for every yoga/mobility session. In the other direction, skipping rope (~11 MET) is credited at less than half. Users compare these numbers against their watches and against common sense; systematic 2× errors on entire categories are exactly how trust dies. The fix is pure data: Appendix A of this document is the complete 58-exercise table, each value tied to its Compendium category. One day of work, port it as a constants file with the same shape as today's `EXERCISE\_MET`.
2. ****Display: the number reaches the user as a point value.**** The Compendium's own MET values are population averages with real spread; the honest display is a range. Spec in §2.3.

2.2 Data contract (already almost in place)

The engine's `SetSummary.durationMs` (Part 2 §2.4) is the ****active-time**** input per exercise; between-set rest comes from the app's session timer (`phase == 'rest'` today – same idea, same split, in the new session model). Nothing about the formula needs the raw pose stream, so calorie

computation stays server-side at sync time (deterministic, versionable) – the client may show a live *provisional* number during the workout, clearly styled as provisional, reconciled at sync.

2.3 Display specification

- **Range = point estimate ± 20%**, both ends rounded to the nearest 5 kcal; e.g. computed 212.4 → **"≈ 170–255 kcal."** The ±20% is not decoration – it is roughly the documented individual variation around Compendium MET values, which is the method's true uncertainty. One constant, `CALORIE_BAND = 0.20`, versioned under `calc_version`.
- **Session totals** sum the per-exercise points, then band the total (not sum of bands – bands don't add linearly and summing them overstates width).
- **Label:** "estimated" appears with the unit the first time in any view (message key `est.calories.label`).
- **Method sheet** (`est.calories.how`): one paragraph – based on exercise type, your body weight, and active time, using the standard MET method (the same reference used in clinical and research settings); a heart-rate monitor can narrow this, which is the mobile roadmap.
- **Weight nudge, not nag:** when the 70 kg fallback is in use, the calorie card shows a single quiet line – "Add your weight for a number that's actually yours" – because personal weight is the single biggest accuracy lever available for free.

2.4 Roadmap (explicitly deferred, not forgotten)

Wearable heart rate on mobile (Part 6) upgrades the method to HR-based estimation for users who have it, with MET as the universal fallback. Continue to refuse: per-rep "work" physics (bodyweight exercise mechanical work ≠ metabolic cost), form-score multipliers, and any per-user "metabolism" claims. The module's own docstring already argues this correctly – that reasoning is now policy.

3. Meal-photo nutrition – the portion problem, solved without new models

3.1 Audit – why the bowl problem is unsolvable in the current architecture

Today's pipeline (`photo\_analyzer.py`): one Groq Llama-4-Scout vision call receives the photo and a prompt that asks it to output food names ****and**** quantities ****and**** kcal ****and**** all macros directly. The router hardening around it is good (quota fail-closed, 10 MB cap, thread off-load so the event loop isn't blocked, and – importantly – ****the photo is never stored****; it is analyzed in memory and discarded. Keep that property; \$3.6 makes it an explicit promise).

But the architecture has two structural failures:

1. ****Portion blindness – your bowl problem, precisely.**** A monocular photo has no absolute scale. The same dal photographed in a 150 ml katori and a 300 ml serving bowl can produce nearly identical pixels – yet one is ~110 kcal and the other ~220. The model cannot know which it is unless something in the frame anchors the scale, and today nothing asks it to look for anchors, the user is never consulted, and the app has no memory of the user's actual dishes. Every scan silently rolls dice on a 1.5-3× quantity error – the single largest error source in the whole feature, dwarfing food-identification mistakes.
2. ****Macro hallucination.**** The model does the nutrition arithmetic itself. Same photo can yield different totals run-to-run (temperature 0.1 reduces but does not eliminate this); the numbers trace to no database; when a user asks "why 340 kcal?" there is no answer. This violates P1 at the exact place users check your work – many know the calories of their daily foods better than any model does.

3.2 The redesigned pipeline – five stages, still exactly one vision call

The vision call does what vision models are good at (seeing); deterministic code does everything else. Cost per scan is unchanged, so the 8/day quota and the unit economics from the pricing doc are untouched.

****Stage 1 – Vision: identify, never compute.**** The prompt is rewritten so the model returns identification **evidence**, no nutrition math at all. Output contract (temperature 0; JSON only):

```

```json
{
 "meal_name": "Dal with rice and roti",
 "cuisine_guess": "north_indian",
 "items": [
 { "name": "dal (lentil curry)", "canonical_hint": "dal",
 "container": "katori_or_small_bowl", "fill_level": 0.75,
 "size_class": "medium", "count": null, "confidence": "high" },
 { "name": "steamed white rice", "canonical_hint": "rice_cooked",
 "container": "plate_section", "fill_level": null,
 "size_class": "medium_mound", "count": null, "confidence": "high" },
 { "name": "roti", "canonical_hint": "roti",
 "container": null, "fill_level": null,
 "size_class": null, "count": 2, "confidence": "high" }
],
 "scale_anchors": [
 { "type": "dinner_plate_rim", "notes": "full rim visible, slight
ellipse" },
 { "type": "spoon", "notes": "steel tablespoon left of bowl" }
],
 "unknown_items": [],
 "photo_quality": "good"
}
```

```

Rules encoded in the prompt: countable items get counts (rotis, idlis, eggs – counting is the one portion task vision does reliably); containers get a type + size class + fill level; the model must list visible scale anchors (plate rims, cutlery, hands, cans) and must say "unknown" rather than guess. ****No kcal, no grams, no macros – ever.**** The model naming a **quantity in grams** is banned from the contract because that is arithmetic wearing a costume.

****Stage 2 – Deterministic portion resolver (grams + a gram-range per item).**** Pure code, fully testable, applies the first matching rung:

1. ****User's saved dishware**** (§3.4): if the user has confirmed "my dal katori $\approx$ 180 ml" before, that beats everything.
2. ****Anchor scaling**** a visible standard dinner-plate rim (26–27 cm across the world's tableware with remarkable consistency) or standard

cutlery gives an approximate mm-per-pixel; container diameter → volume class. This is coarse – its job is only to pick between S/M/L classes, which is where the 2-3× errors live.

3. ****Regional portion priors:**** cuisine guess + item + container class → the Appendix B measures table (katori ≈ 150 ml, serving bowl ≈ 300 ml, 1 roti ≈ 40 g, 1 idli ≈ 35 g...). Region defaults from the user's profile locale.

4. ****Global defaults:**** the conservative middle of the item's typical-serving range.

Each rung yields `grams\_point` and `grams\_range` (e.g. dal, medium katori, 0.75 full → 110 g, range 90-140 g). The rung used is recorded (`portion\_source: "user\_dishware" | "anchor" | "regional\_prior" | "default"`) – it drives the confidence chip and tells you, in telemetry, which rungs are earning their keep.

****Stage 3 – Nutrition arithmetic from databases, never from the model.****

`kcal = grams × per-gram values` from, in priority order: the local curated table (which already holds a good Western core plus Roti, Dal, Paneer, Idli – expand per \$7), USDA FoodData Central (already integrated), an ****IFCT 2017 pack**** for Indian foods (the Indian Food Composition Tables – the authoritative source for exactly the cuisine your first users eat), and OpenFoodFacts for packaged goods (free, global; barcode path arrives with mobile). Every item's result carries its source. Same photo + same confirmed portions now produce the same numbers every single time, and every number has a citation.

****Stage 4 – The confirmation moment (three seconds, one tap).**** Before anything is saved, one compact sheet: each item as a chip with its portion pre-selected by the resolver – *"Dal – medium katori (~110 g)"* – tappable to S/M/L cards (with gram labels and relatable visuals) or a gram stepper; wrong item → tap to search (existing food search endpoint); missing item → add. Then one confirm button. Because the resolver pre-fills well, the common case is literally one tap – but that tap converts an AI guess into a user-verified log, which changes both the data quality and how the user **feels** about the number. The displayed total before confirm is a range (from the gram ranges); after confirm it tightens around the chosen portions.

****Stage 5 – Memory and the learning loop.**** First time an unrecognized personal container appears: one question – "How big is this bowl?" with three visual cards – stored to ****My Dishware**** (per-user: label, volume, food-type association). Next scans auto-apply it (rung 1) and the question never repeats. Every correction (portion changed, item swapped) is stored as `{original\_estimate, user\_final, delta, portion\_source}`: per-user, a consistent bias (always corrects rice down ~20%) shifts **their** defaults; globally, corrections become the regression eval set – any change to the Stage-1 prompt or the measures table must not worsen agreement with the last N hundred confirmed logs (this is the golden-trace idea from Part 2 \$7, applied to nutrition).

3.3 What accuracy to promise (set the bar honestly, then clear it)

Photo-based estimation is $\pm 20\text{--}40\%$ **for trained humans**. The goal is therefore not laboratory truth; it is: **** (a) **** eliminate the 2-3 $\times$ portion-class errors (the resolver + dishware memory do this – it's the whole reason the bowl question matters), **** (b) **** make remaining error **consistent and correctable** (deterministic arithmetic + one-tap edits), **** (c) **** never display more precision than exists (kcal ranges rounded to 10; macros to 1 g; totals as ranges until confirmed). In-app framing (message key `est.meal.how`): "We identify your food from the photo, estimate portions from your dishes and regional serving sizes, and calculate nutrition from published food databases (USDA/IFCT). Confirming portions makes it accurate to **your** servings."

3.4 Data notes for Part 4

Two small tables: `user\_dishware(user\_id, label, container\_class, volume\_ml, created\_at)` and `meal\_log\_corrections(meal\_log\_id, field, original, corrected, portion\_source, created\_at)`; `meal\_logs` rows gain `portion\_source`, `nutrition\_source`, `calc\_version`. Meal photos remain ****never persisted**** – that stays an architectural guarantee, stated in the privacy copy, and it matters double for the physio-clinic segment (\$7).

3.5 Edge cases (specified now so they don't become support tickets)

Thali / multi-compartment plates → each compartment is an item with `container: "thali\_section"` (priors in Appendix B). Liquids and curries → fill-level drives grams via container volume $\times$ density class

(thin/medium/thick – three constants, not a physics model). Packaged food in frame → identify the product, prefer OpenFoodFacts label data, and on mobile suggest the barcode scan instead. Truly unknown item → shown as "couldn't identify – tap to add," which logs **honestly incomplete** rather than confidently wrong; an unknown that the user fills in is a success, not a failure. Terrible photo (`photo\_quality: "poor"`) → ask for one retake with a one-line tip before spending the user's patience on a bad estimate – but never burn a second quota unit for the retake of a failed parse.

4. Recommendations – right architecture, two missing layers

4.1 Audit

The content-based scorer (goal 40 · difficulty 20 · equipment 20 · duration 10 · variety 5 · AI-support 5, popular-items fallback for empty profiles) is the ****correct architecture at your scale****: deterministic, explainable, zero marginal cost, cold-start-friendly. Do not let anyone talk you into collaborative filtering yet – the docstring's "upgrades to Neural CF as history grows" is hereby cancelled. CF needs dense user×item interaction data across thousands of users to beat rules; below that it produces unstable, unexplainable suggestions – vague numbers again, in list form. Revisit only past ~50k MAU, and then as a **global** model trained offline, never per-request.

What the scorer is blind to today: ****how the user actually performs**** (it never reads rep counts or form scores – the richest data your product uniquely generates) and ****recovery**** (nothing stops it recommending lower-body four days straight).

4.2 The upgrade (still 100% deterministic rules)

Add two scoring terms and one output field; totals stay comparable:

– ****Progression term (±15).**** This is where Part 2 pays for itself twice: the ****family templates (\$5) are the progression graph.**** Within a family, if the user's last 3 sessions on an exercise averaged form ≥ 85 with reps at the top of their range → +15 to the next-harder family member (squat →

jump squat; wall push-up → push-up; plank → side plank) and a small -5 to the mastered one (gently graduate them). Struggling (avg form < 60 or chronic early set termination) → +15 to the easier variant. All thresholds are config, not code, per v1 §9.3 conventions.

- ****Recovery & balance term (-20 / +10).**** Primary muscle group trained within the last 48 h → -20. A muscle group untouched for 7+ days while others repeat → +10 to its exercises. Uses the `primary\_category`/muscle tags already in your seed data.

- ****Surface the "why".**** The score components are already computed and then thrown away; return the top contributors per item as message keys, rendered as chips: *"Matches your muscle-gain goal · You're ready to progress from squats · Legs are recovered."* A recommendation with a visible reason is advice; one without is an ad. This one change moves more trust than any model swap could.

4.3 Acceptance tests (deterministic → trivially testable)

Fixture profiles + histories with exact assertions: **mastered-squats user → jump squat in top 6, squat not #1**; **trained-legs-yesterday user → no lower-body in top 3**; **empty profile → popular fallback*, `method: "popular"`. These run in CI like golden traces; a scoring-weight change that flips a fixture is a reviewed decision, not an accident.

5. Predictions — replace statistical theater with transparent statistics

5.1 Audit — the hard verdict

`forecaster.py` fits ****Prophet**** on one user's daily calories (gated at only 5 sessions) and trains an ****XGBoost classifier per request**** on 3-10 weekly samples with 3 features to output "73% chance you hit your goal." At these data sizes this is statistical theater: Prophet's changepoint machinery on two noisy weeks produces forecasts that **visibly change between refreshes** (trust-killer #1, and your exact words: vague, not actual); a per-request tree ensemble on single-digit samples memorizes noise — its own average-rate fallback (already in the file!) is the better system; and the dependency chain (prophet + cmdstanpy + xgboost) is hundreds of MB that slows every cold start and per-request training burns

CPU you pay for. Notably, `detect\_plateau` and `analyze\_best\_time` in the same file are simple deterministic comparisons – those two are **already** the right kind of system. The replacement below makes everything match them, keeps the same API shapes (frontend barely changes), and deletes the heavyweight deps from the image.

5.2 Replacement spec (deterministic, same response shapes)

- ****7-day forecast → the weekday-profile method.**** For each weekday, the recency-weighted mean of that weekday's total calories over the last 4 weeks (weights 4/3/2/1, newest first; missing weekday-instances count as 0 – a skipped Tuesday is real behavior, not missing data). Band = $\pm$ the weighted mean absolute deviation, floored at $\pm 15\%$. Keeps the existing `{date, value, lower, upper}` items and the ≥ 5 -sessions gate. Explainable in one sentence – *"based on your recent Tuesdays"* – which is the entire point.
- ****Weekly-goal likelihood → an honest blend, displayed as words.****
`hit\_rate` = recency-weighted share of the last 6 weeks meeting the target; `feasibility` = can `target - current\_week\_count` still fit in the days remaining (1, else a proportional penalty; 0 if impossible).
Likelihood = hit_rate $\times$ feasibility, then ****mapped to bands, never shown as a percent****: $\geq 0.75 \rightarrow$ "On track" $\cdot 0.40-0.75 \rightarrow$ "Doable – N more sessions" $\cdot < 0.40 \rightarrow$ "Tough week – even one session keeps the streak alive." A fake-precise "73%" invites the user to catch you being wrong; a correct band with a next action is coaching.
- ****Plateau & best-time:**** keep as built (rolling averages, hour-of-day histogram); tighten only the copy – "your pace has been steady for 3 weeks" (observation) rather than "you have plateaued" (diagnosis).
- ****Framing rule for every prediction surface:**** these are **reflections of the user's own pattern**, and the copy says so. Below data gates, surfaces hide entirely (already correct today – keep) rather than showing wide nonsense.
- ****When ML returns:**** a single global model, trained offline on the whole population, shipped as versioned artifacts behind the same API – considered only when cohorts and retention data justify it (v1 \$20 scale triggers), never trained per-request again.

Migration is a drop-in: same routes, same JSON shapes, delete three heavy dependencies, and the numbers stop moving between refreshes – which users will **feel** even if they never articulate it.

6. The display standard (cross-cutting – applies to every number in the app)

One table to rule every numeric surface; new features must add a row before shipping:

| Surface | Display | Rounding | Confidence shown when | Method key |
|---------------------------|----------------------------|-----------------|---|---------------------------------|
| --- | --- | --- | --- | --- |
| Calories burned (session) | range $\pm 20\%$ | nearest 5 kcal | always "estimated" | <code>`est.calories.how`</code> |
| Meal kcal (pre-confirm) | range from gram-ranges | nearest 10 kcal | chip from <code>`portion_source`</code> | <code>`est.meal.how`</code> |
| Meal kcal (confirmed) | tightened range | nearest 10 kcal | – | <code>`est.meal.how`</code> |
| Macros | point from confirmed grams | nearest 1 g | – | <code>`est.meal.how`</code> |
| Recommendations | ranked list + "why" chips | – | – | <code>`rec.how`</code> |
| Forecast | band chart | nearest 10 kcal | hidden below data gate | <code>`pred.how`</code> |
| Goal likelihood | word band + next action | no percentages | hidden below 3 weeks | <code>`pred.goal.how`</code> |
| Form score | existing 0-100 | integer | – | <code>`form.score.how`</code> |

Plus three invariants: ****determinism**** (same inputs → same outputs on every surface; the only nondeterministic component left anywhere is Stage-1 vision identification, and even it runs at temperature 0 with its arithmetic removed); ****versioning**** (``calc_version`` stored with every persisted estimate; recomputation of history is always explicit, never silent); ****editability**** (any number about the user's own life is correctable in ≤ 2 taps, and the correction visibly wins).

7. Worldwide users and the organization segments

****Units & locale:**** kg/lb and ml/cup follow the user's profile (weight normalization already handles both units – keep); all trust-layer copy

uses the Part 2 message-key system, so translation is content work, not engineering.

****Food regionalization**** is the real localization problem, and it is data-shaped: the resolver's priors and the nutrition sources ship as ****regional packs**** – India first (IFCT 2017 values + the Appendix B measures; the local table's four Indian staples grow to the ~60 dishes that cover the overwhelming majority of Indian home meals), USDA as the global backbone, OpenFoodFacts for packaged goods everywhere. The Stage-1 prompt receives a cuisine hint from profile locale (improving identification of, say, poha vs. upma), but identification is never **restricted** by region – a user in Mumbai photographing pasta must work perfectly. New market → new pack, zero engine changes: the same exercises-as-data philosophy, applied to food.

****The four organization types**** (new gyms, premium-aspiring gyms, boutique/PT studios, physio clinics) all fit v1 \$8's tenancy design as org types with vocabulary overrides – no new architecture. Two things **do** need saying now:

– ****Physio clinics are a positioning boundary, not just a segment.**** The product is a ****wellness and exercise-adherence tool the clinician assigns and monitors**** – it does not diagnose, treat, or claim rehabilitation outcomes. That single sentence keeps you outside medical-device regimes (India CDSCO's SaMD framework, EU MDR, FDA) that would otherwise consume a solo developer whole. Enforce it in copy rules (in-app strings say "exercise plan," never "rehab protocol" or "treatment"; message-key review catches this), in the clinic dashboard's framing (adherence and activity, not clinical outcomes), and in the terms for clinic tenants (the professional retains clinical judgment). Everything the trust layer builds is **why** clinics will adopt it – honest ranges, visible methods, never-stored photos, and a clinician-visible record of what their client actually did.

– ****Privacy posture is health-adjacent worldwide:**** meal and workout data under India's DPDP Act 2023 and GDPR for EU users means the v1 \$18 items (export, deletion, consent copy) are launch requirements, not polish – and "photos are analyzed, never stored" is both a compliance simplification and a marketing line. Put it in the scan UI itself.

8. Sequencing and definition of done

Slotting into v1 \$22 without disturbing it – the form engine (Part 2) still leads; this work rides the phases where each module is being ported anyway:

| Work | Lands in | Effort | Done when | |
|---|-------------------------------|-----------|---|--|
| --- | --- | --- | --- | |
| MET table (Appendix A) → constants | Phase 2 backend port | ~1 day | | |
| Every seeded exercise resolves a category-correct MET; yoga session ≈ halves vs. today; unit test locks table completeness against the catalog. | | | | |
| | | | | |
| Forecaster replacement | Phase 2 | 1-2 days | prophet/cmdstanpy/xgboost gone from requirements; responses byte-shape-compatible; same-input-same-output test passes; image size and cold start measurably drop. | |
| Recommender upgrade | Phase 2-3 | 2-3 days | Progression/recovery terms live behind config; "why" chips render; CI fixtures pass. | |
| Meal pipeline v2 (Stages 1-3) | Phase 3 (nutrition port) | 3-4 days | | |
| Model output contains zero nutrition numbers; identical confirmed portions → identical totals; every value carries a source. | | | | |
| Confirmation UI + My Dishware (4-5) | Phase 3 | 3-4 days | Common case is one tap; second scan with a saved bowl asks nothing; corrections persist with originals. | |
| Display standard | applied per surface as ported | folded in | \$6 table implemented; `calc_version` written on every stored estimate. | |
| India food pack | content, parallel | ongoing | Top-60 Indian dishes resolve locally with IFCT values + priors. | |

Roughly two and a half weeks of focused work total, none of it blocking the engine – and at the end, every number the app shows is stable, bounded, sourced, and correctable. That is what "actual, not vague" compiles to.

Appendix A – Complete MET table for all 58 exercises

Values assigned from the 2024 Adult Compendium of Physical Activities by category; these are population bands (hence the $\pm 20\%$ display rule), conservative where a judgment call existed. * = value already in `calories.py` today, preserved exactly.

****Yoga & stretching holds (2.3–3.0)**** – hatha/static-stretch band:
Mountain Pose 2.3 · Child's Pose 2.3 · Cat-Cow Stretch 2.5 · Seated Forward Bend 2.5 · Hamstring Stretch 2.5 · Hip Flexor Stretch 2.5 · Shoulder Stretch 2.5 · Cobra Pose 2.5 · Bridge Pose 2.5 · World's Greatest Stretch 3.0 · Tree Pose 3.0 · Warrior I 3.0 · Warrior II 3.0 · Downward Dog 3.0

****Isometric holds (3.0–3.5):**** Plank 3.0* · Side Plank 3.0 · Superman Hold 3.0 · Wall Sit 3.5

****Light dynamic / warm-up (2.8–3.8):**** Arm Circles 2.8 · Heel Raises 3.0 · Calf Raises 3.5 · Wall Push-ups 3.5 · Brisk Walking 3.8 · Seated Leg Raises 3.0

****Resistance, moderate (3.5–5.0)**** – Compendium "resistance training, multiple exercises, 8–15 reps":
Bicep Curls 3.5* · Shoulder Press 3.5* · Lateral Raises 3.5 · Tricep Extensions 3.5 · Bench Press 3.5 · Resistance Band Pull 3.5 · Glute Bridge 3.5 · Crunches 3.8 · Leg Raises 3.8 · Russian Twists 3.8 · Bicycle Crunch 4.0 · Flutter Kicks 4.0 · Tricep Dips 4.0 · Hip Thrust 4.0 · Chair Squats 5.0* · Deadlifts 5.0

****Bodyweight circuit, continuous (6.0–6.5):**** Squats 6.0* · Lunges 6.0* · Bulgarian Split Squat 6.0 · Step-ups 6.5

****Calisthenics & plyometric, vigorous (7.0–8.0):**** Push-ups 8.0* · Pull-ups 8.0 · Pike Push-ups 8.0 · Burpees 8.0 · Jumping Jacks 8.0 · Mountain Climbers 8.0 · High Knees 8.0 · Jogging in Place 8.0 · Jump Squats 8.0 · Tuck Jumps 8.0 · Jump Lunges 8.0 · Skater Jumps 7.0 · Plank Jacks 7.0

****Jump rope (11.0):**** Skipping Rope 11.0 – the Compendium's moderate-pace rope value; the largest single correction vs. today's 5.0 fallback.

(58 total. `arnold\_shoulder\_press` – the non-catalog config – inherits 3.5 if ever seeded.)

Appendix B – Portion priors seed (India-first, ranges the resolver uses)

Sources: IFCT 2017 serving conventions and standard Indian dietetic references; these are ****priors**** – user dishware and corrections override them, and telemetry (§3.2 Stage 5) refines them.

****Containers:**** small katori 100-150 ml · standard katori 150-200 ml · large katori/bowl 250-300 ml · serving bowl 300-400 ml · steel tumbler 150-200 ml · chai cup 100-150 ml · thali section 100-150 g · rice mound on plate: small 100 g / medium 150 g / large 250 g.

****Countables:**** roti/chapati 35-45 g · paratha 60-90 g · puri 20-30 g · idli 30-50 g · dosa (plain) 80-120 g · medu vada 40-60 g · samosa 60-100 g · egg (large, whole) 50 g · bread slice 25-30 g · banana 100-120 g.

****Fill-density classes for curries/liquids:**** thin (rasam, thin dal) $\approx$ 0.95 g/ml · medium (dal, sambar, most curries) $\approx$ 1.0 g/ml · thick (dry sabzi, halwa, thick gravies) $\approx$ 1.1 g/ml – three constants that turn **container × fill level** into grams.

****Global starter set**** (expanded per market pack): dinner plate rim 26-27 cm (the scale anchor) · cup 240 ml · tablespoon 15 ml · teaspoon 5 ml · cereal bowl 350-400 ml · mug 300-350 ml.

– End of Part 2B. The sequence resumes per v1 §23: Part 3 Gym Dashboard (recommended next – the pilots depend on it) · Part 4 Database DDL & migration (now including §2-3's small schema notes) · Part 5 Billing & webhooks · Part 6 Mobile (wearable-HR calorie upgrade lands there) · Part 7 Retention playbook · Part 8 Ops runbook.

AI Home Gym – Part 2: The Form Engine & Exercise Definition System (EDS)

****Full engineering specification · v2.1 · follows Architecture v1 (§5, §22 Phase 1, §23 Part 2)****

****Audience:**** Kd (solo developer, owner)

****Status:**** This is the keystone spec. Every later deliverable – the mobile app, all remaining exercises, offline mode, anti-cheat, and the gym demo – depends on what is defined here. It is written to be followed without needing the v1 doc open, but it never contradicts v1.

****Changelog v2.1**** (evidence-verified reconciliation): catalog corrected to the definitive ****58 exercises**** from `scripts/seed\_exercises.py` – media-alias ghosts removed (`soldierpress`, `hamstring` were Shoulder Press / Hamstring Stretch media, confirmed by decoding GIF frames), five genuinely missing entries added (Warrior II, Bridge Pose, Arm Circles, Mountain Pose, Brisk Walking), `tracking: "pose"|"timer"` field added to the schema, Appendix B converted to a resolved evidence log, §9.1 gains the formal video-derived threshold protocol, §8.2 gains the dead-ONNX cleanup item. Companion document: ****Part 2B – The Trust Layer**** (calories, meal-photo portions, recommendations, predictions).

0. What this document locks in place

Architecture v1 made two decisions this document now specifies completely:

****D1**** – all pose analysis runs on-device (browser and phone), inside one shared, pure TypeScript package `@app/engine`. The server never sees keypoints during a workout; it receives only compact set summaries.

****D2**** – exercises are data, not code. One generic engine interprets declarative Exercise Definitions. Shipping exercise #14 or #59 is publishing a config, not writing a Python function or deploying anything.

I have verified this spec line-by-line against your actual code. Every threshold your squat, jump-squat and chair-squat analyzers use today (`form\_analyzer.py`, `angles.py`, `rep\_counter.py`, the `\_StandingCalibration` and chair-depth calibration in `pose\_ws.py`) appears in this document explicitly, mapped to the field of the new system that carries it. ****Nothing you validated with real footage is lost or**

changed – it is translated.** Where this spec proposes numbers for the 50+ exercises that don't exist yet, they are clearly marked **starting values, to be tuned via recorded traces** – never presented as validated.

One correction to v1 while we're here (small, but you should rotate these too): beyond the secrets v1 listed, the archive also contains real values for `VITE\_LOCATIONIQ\_KEY` (this one ships ****inside the public browser bundle**** – anyone can read it from your JS today), `RAPIDAPI\_KEY`, and `USDA\_API\_KEY`. Rotate all three along with the rest, and move geocoding behind the server `geo` module exactly as v1 §6.1 already plans.

1. Scope, non-goals, and the six invariants

1.1 In scope

The engine package (`packages/engine`), its input/output contracts, the eight-stage per-frame pipeline, the signal library, the calibration modules, the rep and hold state machines, the fault DSL, the scoring model, the Exercise Definition JSON schema, the 12 family templates, the complete mapping of your real catalog onto those templates, the golden-trace test harness, the Python→TypeScript parity protocol, and the authoring/publishing workflow that gets you from 3 exercises to 59.

1.2 Non-goals

No UI specification (the existing `ActiveWorkout` surface survives; it consumes engine events). No ML classifiers – the CTR-GCN/ST-GCN/YOLO work stays parked in `ml-training/` as a research track exactly as v1 §1.3 ruled; the rule engine is the product. No server code – the sync endpoint's contract is restated here only so the engine's output is unambiguous.

1.3 The six invariants (violating any of these is a build failure, not a style issue)

****I1 – Purity.**** `@app/engine` imports nothing platform-specific: no DOM, no React, no React Native, no Node APIs, no timers, no `Date.now()`. Time

enters only through frame timestamps. This is what lets the identical code run in the browser, in Hermes on a phone, and in the Node test runner.

****I2 – Determinism.**** Same frame sequence in → same events out, bit-for-bit, on every platform, every run. No randomness, no wall-clock reads, no locale-dependent formatting. Determinism is what makes golden traces (§7) and server-side replay audits (v1 §14) possible at all.

****I3 – One canonical skeleton.**** The engine consumes exactly the BlazePose 33-landmark layout, on every platform. If a future mobile pose provider emits a different layout, the **adapter** converts it before the engine ever sees it; the engine never branches on platform.

****I4 – Versioned everything.**** Every session records ``engineVersion`` (semver of the package) and ``defsVersion`` (bundle number) – both already appear in the v1 §5.3 sync payload. A definition declares ``minEngineVersion``; a client whose engine is older simply doesn't offer that exercise. No silent behavioral drift, ever.

****I5 – Performance budget.**** Per-frame evaluation (stages 2-8, i.e. everything after pose inference itself) must run in ****≤ 3 ms p95 on a mid-range Android (₹12k class)**** and allocate no growing memory across a 30-minute session. Pose inference (MediaPipe/BlazePose) is outside this budget – it dominates and is the pose provider's cost. The budget is enforced by a benchmark test in CI (§7.6), not by hope.

****I6 – Fail soft, never silently wrong.**** Missing landmarks, NaN, dropped frames, fps collapse – every degraded input maps to a defined behavior in this spec (usually: hold state, don't count, surface a visibility hint). An exception escaping the engine is a P0 bug; an incorrectly counted rep because inputs were garbage is also a bug. The correct response to garbage is **no decision**, exactly the principle your current ``RepCounter.update()`` already applies when the angle is ``None``.

2. Canonical data contracts

2.1 ``PoseFrame`` – the engine's only input during a set

One frame from the pose provider:

```
```jsonc
{
 "t": 123456.7, // monotonic ms since session start (float). NOT
wall-clock.
 "kp": [[x, y, z, vis], /* x 33 */]
}
```
```

Landmark semantics (identical to what MediaPipe Tasks gives you today, and what your `KP` map in `angles.py` indexes):

| Field | Range / convention |
|--------------|--|
| `x`, `y` | Normalized image coordinates in [0, 1]. Origin top-left, increases downward (your elevation code already depends on this: "risen above standing" = y decreased). |
| `z` | BlazePose relative depth, roughly hip-origin, same scale as x. Used sparingly (§3.4) – it is the noisiest channel. |
| `vis` | Visibility/presence score [0, 1]. The engine's visibility gate (§3.2) uses your existing threshold of 0.3 . |
| Index layout | BlazePose 33: 0 nose ... 11/12 shoulders, 13/14 elbows, 15/16 wrists, 23/24 hips, 25/26 knees, 27/28 ankles, 29/30 heels, 31/32 foot index. The engine ships this index map as a frozen constant; it is the same table your `angles.py` `KP` dict encodes. |

Frame-rate contract: the engine accepts **8–40 fps** and adapts (all temporal logic below is specified in milliseconds, then converted to frames using measured inter-frame deltas – never hardcoded frame counts at an assumed fps). The target remains ~15 fps analysis / ~30 fps overlay, exactly your current web throttling; those numbers now live in the *pose provider adapters*, not in the engine.

2.2 Mirroring and left/right policy (this bites every camera app – pin it now)

– "left_" always means the **person's anatomical left**, which is MediaPipe's convention. It usually appears on the *right side of the image* for a front-facing camera.

- Front cameras typically render a mirrored **preview**. The rule: ****mirror only the drawn overlay, never the coordinates.**** The engine always receives un-mirrored provider coordinates. (Your current pipeline already behaves this way implicitly; making it a stated rule prevents the classic mobile bug where someone mirrors the landmark array to "fix" the overlay and silently flips every left/right and valgus sign.)

- Signals that live in image-space (valgus x-positions) keep your documented convention: signs are in image space, and the **baseline-relative delta** is what carries meaning – which is precisely why your ``standing_baseline`` calibration design is correct and survives unchanged.

2.3 Session inputs

A session is created with: the resolved ****ExerciseDefinition**** (already parsed and lint-validated, §4), optional ****carry-over calibration**** from a previous set of the same exercise in the same workout (so set 2 doesn't recalibrate – your ``active_sessions`` reconnect-persistence behavior, reproduced locally), and an optional ****user profile hint**** block (reserved: height, mobility limitations – v2 territory, ignored by engine v1).

2.4 Outputs – three event levels plus one document

****FrameResult**** (every analyzed frame; feeds the overlay and live UI): ``phase``, ``repCount``, ``isActive``, ``view``, ``visibilityOk``, ``liveCue`` (nullable message key), ``signals`` (the small set the definition declares, for debug/telemetry overlays), ``calibrationState`` (``pending`` | ``ready``).

****RepEvent**** (at the moment a rep is credited): ``repIndex``, ``score`` (0-100), ``faults`` (array of fault ids observed in that rep cycle), ``durationMs``, ``phaseTimings`` (ms in descent/bottom/ascent), ``romExtreme`` (e.g. min knee angle reached), ``view``.

****HoldTick / HoldEvent**** (isometric family only): accumulated qualifying ms, in-band or out-of-band, ended-hold summary.

****SetSummary**** (at set end – this is **the only thing that leaves the device**, byte-compatible with v1 §5.3):

```

```jsonc
{
 "exercise": "squat", "setIndex": 1, "reps": 12, "durationMs": 48000,
 "avgFormScore": 84, "repScores": [90, 88, 76, ...],
 "faultCounts": { "shallow_depth": 2, "knee_valgus": 1 },
 "tempoMsAvg": 3900,
 "romStats": { "metricMinAvg": 96 },
 "view": "side",
 "holdMs": null, // isometrics: qualifying hold time
 "calibration": { "usedStandingBaseline": true, "chairDepthTarget": null
},
 "engineVersion": "1.0.0", "definitionVersion": 5
}
```

```

Design note: per-rep scores stay as a small array inside the set (matches v1 \$7.1's "no billion-row rep_events table"); everything a screen or a gym report needs is derivable from SetSummary.

3. The pipeline — eight stages, fully specified

Order per frame: ****ingest** → **condition** → **view** → **signals** → **calibration** → **FSM** → **faults** → **score/emit.**** Each stage below states its algorithm, its parameters with defaults, and its failure behavior. Parameters marked ⚙ are overridable per-definition; everything else is engine-global.

3.1 Ingest & validation

Port of your `\_valid\_keypoints` verbatim: exactly 33 entries, each with ≥ 4 finite numbers (NaN check `v !== v`). Additionally: `t` must be finite and strictly greater than the previous frame's `t` (out-of-order frames are dropped, counted in a session diagnostic). An invalid frame is ****dropped silently**** — state holds, nothing counts, no event. Three consecutive invalid frames flip `visibilityOk=false` on FrameResult so the UI can show "step back into frame" (your existing "cannot see your legs clearly" behavior, generalized).

3.2 Conditioning: smoothing + visibility gating

- **Angle smoothing:** rolling mean over a **7-frame window** – your `RepCounter.angle_buffer`` value, kept. Implemented as a per-signal ring buffer; window is time-aware (falls back to fewer samples at low fps rather than stretching latency).
- **Two smoothing channels, deliberately:** FSM/fault evaluation reads *smoothed* signals; the **elevation signals used for airborne detection read raw** values with only a 3-frame spike filter. Reason: a 7-frame mean at 15 fps is ~470 ms of lag, and a jump-squat flight phase is often 300–500 ms – heavy smoothing would erase the very event you're detecting. Your current server code effectively gets away with this because elevation is computed per-frame from geometry; the engine makes the raw-path explicit so nobody "optimizes" it away later.
- **Visibility gating with hysteresis:** a landmark is *usable* when `vis ≥ 0.30`` (your threshold) and becomes *unusable* only when `vis < 0.15`` – the 0.15 gap stops flicker at the boundary from toggling signals on/off every frame. A signal whose required landmarks are unusable returns `null`` for the frame; consumers of `null`` follow §3.6's no-decision rule.

3.3 View classifier

Direct port of your `detect_view`` – it is measured against labeled clips and the clusters don't overlap, so it stays:

- Reference height = `|ankle_y - shoulder_y|` (distance-invariant normalizer).
- `shoulder_width_norm`` and `hip_width_norm`` = x-spans divided by that height.
- **side** if both `< 0.15`` · **front** if `shoulder > 0.20` or `hip > 0.15`` · else **unknown**. Requires shoulders, hips, one ankle visible; otherwise **unknown**.

Two engine-level additions:

1. **Hysteresis:** the reported view changes only after the new classification persists **10 consecutive frames** (~0.7 s at 15 fps). Prevents mid-rep view flapping when someone rotates slightly, which would otherwise toggle view-scoped faults (valgus) on and off within one rep.
2. **View policy per definition** (☞): `views: ["front", "side"]`` plus `preferredView`` and `requireView`` flags. If a definition *requires* a view

(e.g. valgus coaching genuinely needs front), the UI gets a ``liveCue: "cue.turn_to_front"`` instead of silently skipping checks. ``quarter`` (~45°) is not a v1 output class – your measured clusters are binary + unknown; a quarter stance lands in ``unknown`` and simply gets the view-agnostic checks, which is safe. (Recorded as a v1.1 candidate, not scope creep now.)

3.4 Signal library v1

A *signal* is a named, typed, per-frame scalar with a declared formula, view validity, smoothing channel, and calibration dependency. Definitions declare which signals they need; the engine computes **only those** (perf budget I5). The v1 library – 22 signals – covers every exercise in §6. Formulas are exact where ported from your code.

| # | Signal | Formula / source | Valid views | Needs calibration | Notes |
|-------|---|---|-----------------|-------------------|---|
| 1-6 | <code>`knee_L/R`</code> , <code>`hip_L/R`</code> , <code>`elbow_L/R`</code> | Interior angle at joint via your 3-point <code>`calculate_angle`</code> (arccos of normalized dot product, degrees) | any | – | The workhorses. <code>`hip`</code> = shoulder-hip-knee. |
| 7-8 | <code>`shoulder_L/R`</code> | Interior angle elbow-shoulder-hip | any | – | Drives raises, presses, jacks. |
| 9 | <code>`knee_avg`</code> | Mean of visible knee angles (1 or 2) | any | – | Your <code>`analyze_squat`</code> uses exactly this. |
| 10 | <code>`trunk_incline`</code> | Angle of shoulder-hip vector from vertical, degrees; prefers left pair, falls back right | any (best side) | – | 0 = upright. |
| 11 | <code>`shin_incline`</code> | Knee-ankle vector from vertical, same helper | side | – | Used in <i>relative</i> lean checks (trunk vs shin). |
| 12-13 | <code>`valgus_L/R`</code> | <code>`(knee_x - ankle_x) / hip_width`</code> – your validated-notebook formula, image-space sign | front | – | Raw stance value. |
| 14-15 | <code>`valgus_delta_L/R`</code> | Raw minus standing-baseline value | front | standing_baseline | Negative = inward drift. The only valgus numbers rules may reference. |
| 16 | <code>`hip_elevation`</code> | <code>`(baseline_hip_y - hip_y) / torso_height`</code> , raw channel | any | standing_baseline | Positive = airborne rising. Your jump-squat signal, normalized exactly as captured by <code>`_StandingCalibration`</code> . |

```

| 17 | `ankle_elevation` | Same construction on ankles, raw channel | any
| standing_baseline | Confirms true flight (both feet leave). |
| 18 | `body_line` | Angle at hip between shoulder-hip and hip-ankle
vectors; 180 = straight body | side | - | Plank/push-up sag-or-pike. Your
current plank rule (`left_hip` 160-180) is this signal by another name. |
| 19 | `elbow_under_shoulders` |  $\frac{||\text{elbow}_x - \text{shoulder}_x||}{\text{torso\_height}}$  |
side | - | Plank stacking cue. |
| 20 | `cadence` | Cycle events per minute from the FSM (§3.6 cadence
mode) | any | - | Derived, not geometric. |
| 21 | `stillness` | Windowed (700 ms) mean displacement of hip+shoulder
midpoints, torso-normalized | any | - | Hold quality for
isometrics/stretch holds; also the "person is set up and ready" detector.
|
| 22 | `symmetry_knee` / `symmetry_elbow` |  $||L - R||$  of the smoothed
pair, degrees | front | - | Uneven press/curl/squat cue; only evaluated
when both sides usable. |

```

Reliability annotations ship with the library (and drive the risk notes in §6): `z` is used by ****no**** v1 signal (too noisy – rotation-type exercises are handled via x-span oscillation instead, see russian twist in §6); prone/supine poses (superman, cobra, bridges) reduce landmark confidence, so their definitions require the side view and lean on hip/knee/shoulder angles that survive it.

Adding a signal later = adding one pure function + its row here + trace coverage. Signals are versioned with the engine (I4), not with definitions.

3.5 Calibration modules

Calibration exists so checks are **relative to this person's own body and setup** rather than absolute cutoffs – the single smartest property of your current squat system, generalized. Three modules in v1:

****C1 `standing\_baseline` – port, unchanged semantics.**** Arms automatically when smoothed knee $\geq$ ****160°**** (`STANDING\_KNEE\_MIN`); captures after ****8 consecutive**** qualifying frames; buffers reset on any non-qualifying frame – your exact `\_StandingCalibration` state machine. Captures: `valgus\_L/R` baselines (front view), `hip\_y`, `ankle\_y`, `shoulder\_y`, and `torso\_height = |shoulder\_y - hip\_y|`. Lifetime: the exercise session;

****carried across sets**** of the same exercise in the same workout (§2.3), matching today's reconnect-surviving server sessions. Invalidations that force re-capture: view flips front↔side, or subject-lost for > 3 s (the person who walks back into frame may be standing elsewhere).

****C2 `adaptive\_target` – port of chair-depth calibration.**** Observes the metric extreme (min knee angle) across the ****first 2 completed reps****, sets `session\_target` = mean of those extremes, clamped to a definition-declared range (⚙ chair squat: clamp [80°, 120°], fallback ****100°**** = your `CHAIR\_FALLBACK\_TARGET`). Until it fires, scoring uses the fallback – precisely today's early-rep behavior. Declared generically so any "target-depth, not max-depth" exercise (step-up height, wall-sit angle) can reuse it.

****C3 `floor\_reference` – new, tiny.**** For supine/prone starts (glute bridge, superman, leg raises): captures resting `hip\_y` / `shoulder\_y` during the first ****1 s of stillness**** (signal 21 below threshold). Gives bridges and leg raises the same "relative to your own setup" quality – camera height and mat position stop mattering.

UI contract: `calibrationState: "pending"` on FrameResult until required modules are ready; the UI shows the "stand still facing the camera" beat your users already experience implicitly. Faults that depend on an unready calibration are ****not evaluated**** (no decision on missing data, I6) – depth and rep counting still work, exactly as today.

3.6 Rep & hold state machines

One generic FSM, four modes (⚙ `rep.mode`). All modes share the anti-garbage guards, which are straight ports of your `RepCounter` hardening:

- Null metric this frame → ****hold state, count nothing, mark `isActive:false`**** (your None-angle rule).
- Debounce: state must persist ****≥ 3 frames down / ≥ 2 frames up**** (time-normalized at low fps).
- ****Minimum rep interval 450 ms**** engine-wide, and per-definition `minRepMs` / `maxRepMs` (⚙) on top – the same numbers the server later uses for plausibility validation (v1 §14), so honest clients can't trip anti-cheat.

- ****Bilateral gate**** (⚙️, on by default for two-leg knee metrics): the **other** knee must be $\leq$ ****150°**** when both are visible – your one-legged-phantom-squat rejector, kept, including its fall-back-to-single-knee-when-occluded behavior.

****Mode A – `alternating\_threshold`**** (squats, push-ups, curls, presses...). Metric + `upAt`/`downAt` hysteresis + `countOn: "up" | "down"`. Phase model derived for free: `top` ($\geq$ upAt), `descent` (falling between), `bottom` (rising-edge of local minimum, captured as the ROM-extreme event), `ascent`. Faults may scope to phases; "at bottom" faults evaluate once per rep at the captured extreme (this is how depth scoring becomes per-rep instead of per-frame noise).

****Mode B – `hold`**** (plank, wall sit, side plank, stretch holds). Enter qualifying hold when metric inside `holdBand` ****and**** `stillness` below threshold for `enterMs` (default 700 ms); accumulate qualifying ms; exit when out-of-band for `exitMs` (default 500 ms – brief wobbles don't reset a plank, which is today's biggest hold-UX complaint pattern in every competitor). Emits HoldTicks (UI timer) and per-hold summaries. "Reps" for isometrics = number of qualifying holds; the headline number is `holdMs`.

****Mode C – `alternating\_sides`**** (lunges, Bulgarian split squat, step-ups, cycling crunch). Two Mode-A sub-machines, one per side; frame routes to the side whose metric is **more flexed**; total = L + R with per-side counts kept (the UI can show 6+6). A side-switch mid-descent voids that partial rep (no double-count exploit, no confusion when someone alternates legs freely).

****Mode D – `cadence`**** (jumping jacks, high knees, mountain climbers, skipping...). Counts **cycles** of a periodic signal: rising-edge crossings of the signal's own rolling midline, accepted only when peak-to-trough amplitude $\geq$ `minAmplitude` (⚙️) and instantaneous cadence within `[minCadence, maxCadence]` (⚙️). Robust to the ROM sloppiness that's **normal** in conditioning moves, and cheap. Reports cadence (signal 20) for pace cues.

3.7 Fault rules – the DSL

A fault rule is data. Full shape:

```

```jsonc
{
 "id": "knee_valgus",
 "view": "front", // optional; omit = any view
 "phase": ["descent", "ascent"], // optional; omit = any phase
 "when": "valgus_delta_min < -0.15", // condition (grammar below)
 "sustainMs": 0, // condition must hold this long (0
= instant)
 "perRep": true, // fire at most once per rep cycle
 "severity": 20, // score deduction weight (§3.8)
 "severe": "valgus_delta_min < -0.30", // optional: forces rep
'incorrect' regardless of score
 "msg": "fault.squat.valgus", // message key — localization &
voice for free
 "cueCooldownMs": 4000 // don't re-speak/re-show within
this window
}
```

```

****Condition grammar**** (kept deliberately small — every operator below is required by some exercise in §6, and nothing else is included):
 comparisons ``<``, ``<=``, ``>``, ``>=``, ``==`` between a signal reference and a number;
 boolean ``&&``, ``||``, parentheses; signal refs may take per-rep aggregates ``_min``, ``_max``, ``_avg`` (evaluated at rep completion, e.g. ``knee_avg_min > 130`` = your shallow-depth check) or calibration refs (`target`, from C2, e.g. ``knee_avg_min > target + 25`` = your chair shallow-margin check).
 That's the whole grammar. It is parsed ****once at definition load**** into an expression tree — zero per-frame parsing (I5). The linter (§9.2) rejects unknown signals, view-invalid signals (valgus in a side-only definition), and unsatisfiable ranges at authoring time, not at runtime in a gym.

****Runtime semantics.**** Frame-scoped rules evaluate on smoothed signals each frame within their phase/view scope, honoring ``sustainMs`` (a lean must persist ~300 ms to fault — single-frame noise never speaks). Rep-scoped rules (`_min/_max/_avg`, ``perRep``) evaluate exactly once at rep completion. Fired faults join the current rep's fault set; ``severe`` conditions additionally mark the rep incorrect — the generalization of your ``severe_faults`` list (trunk > 60° and valgus < -0.30 both port as ``severe`` expressions).

****Coaching output policy – port of your best UX decision.**** At most ****2 corrections**** surfaced at a time, deduplicated, ordered by severity – exactly your ``deduped[:2]``. Voice gets only the single worst, throttled by ``cueCooldownMs``. Message **keys**, not strings: the engine emits ``fault.squat.depth``; the client's locale table renders English/Hindi/Assamese text and TTS. Appendix A seeds the key catalog from your current correction strings so nothing regresses.

3.8 Scoring

Two-layer model that reproduces your squat scorer exactly and generalizes it:

****Layer 1 – component curves.**** A definition declares scoring components, each mapping a signal (or rep-aggregate) through a ****piecewise-linear band curve**** – the declarative form of your ``_score_depth`` / ``_score_trunk_lean`` / ``_score_valgus`` functions. Example, squat depth, which is **numerically identical** to ``_score_depth``:

```
```jsonc
{ "component": "depth", "input": "knee_avg_min",
 "curve": [[100, 100], [130, 30], [155, 0]], // ≤100°→100 pts ...
 "inactiveAbove": 155 } // standing = no depth
opinion (your None)
```
```

Trunk lean ports as ``[[45,100],[75,0]]`` (100 down to 0 across your 30° falloff); valgus as a curve on ``|valgus_delta_min|``: ``[[0.15,100],[0.30,0]]``, inactive for outward drift. Chair depth uses C2's ``target``: full credit $\leq$ ``target+10``, linear to 40 at ``target+25``, then your low-credit tail – the three chair constants (``TOLERANCE 10``, ``SHALLOW_MARGIN 25``, standing cutoff 155) become curve parameters.

****Layer 2 – aggregation.**** Per-rep score = ****mean of active components**** (your rule), ****80 neutral**** when none are active (your standing-between-reps rule), floored at a definition ``floor`` (default 0; squat keeps behavior via curves). ``formCorrect`` per rep = score $\geq$ ****70**** and no ``severe`` fault – verbatim. Set ``avgFormScore`` = mean of rep scores. Session ****Form Score™**** (the v1 \$21 retention anchor) = exposure-weighted

mean across the workout – computed by the engine so web, mobile, and the server's audit replays can never disagree about the headline number.

Isometrics score differently (they have no reps): hold score = `% of hold time in-band` (0-100) with a stillness bonus band (up to +10, capped 100); definition declares the band. Wall-sit example in §6.

3.9 Emission & the set lifecycle

`startSet(definition, carryOverCalibration?) → feedFrame(PoseFrame) → endSet() : SetSummary`. `endSet` is idempotent; an abandoned set (app killed) is reconstructed from the last emitted RepEvents by the client shell – the engine additionally exposes `snapshot()` (plain-JSON state) every 5 s to the shell for exactly this, replacing the server-side session store's crash-resilience role.

4. The Exercise Definition schema

Field-by-field. One definition = one exercise = one JSON document, validated by the linter (§9.2) before it can be published. (Schema formalized as JSON-Schema/Zod in `packages/shared` – the table below is the human contract.)

| Field | Type / constraints | Meaning |
|--|-------------------------|---|
| --- | --- | --- |
| `key` | slug, unique, immutable | e.g. `chair_squat`. Aliases list absorbs your existing plural keys (`chair_squats`) so old clients/URLs keep working. |
| `version` | int, monotonic | Bumped on any behavioral change. Appears in every SetSummary. |
| `minEngineVersion` | semver | Client gate (I4). |
| `family` | enum of §5 | Template this definition instantiates. |
| `name`, `muscles`, `equipment`, `difficulty`, `mediaRef` | catalog metadata | Carried for the library UI; not read by the engine. |
| `views` | array: front/side | Allowed views. |
| `preferredView`, `requireView` | slug / bool | §3.3 policy; `requireView` triggers the turn-cue. |

- | ``calibration`` | array of C1/C2/C3 refs + params | e.g. ``[{ "module": "adaptive_target", "clamp": [80,120], "fallback": 100, "observeReps": 2 } ]``. |
- | ``signals`` | array of §3.4 names | Everything the engine must compute. Linter cross-checks every rule/curve reference against this list. |
- | ``rep`` | object | ``mode`` (A-D §3.6) + mode params: ``metric``, ``upAt``, ``downAt``, ``countOn``, ``minRepMs``, ``maxRepMs``, ``bilateralGate``, ``holdBand``, ``enterMs``, ``exitMs``, ``cadenceSignal``, ``minAmplitude``, ``minCadence``, ``maxCadence``, ``perSide``. |
- | ``phases`` | array | Names for Mode A phases (UI + fault scoping). |
- | ``faults`` | array of §3.7 rules | The coaching brain. |
- | ``scoring`` | object | Components with curves (§3.8), ``floor``, ``neutral`` (default 80), ``correctAt`` (default 70), isometric band if Mode B. |
- | ``setup`` | object | Pre-set guidance: ``cameraHint`` key ("place phone sideways, 3 m away, full body visible"), ``positionCheck`` (a boolean signal condition that must hold before the set arms – e.g. body horizontal for planks). Kills the #1 real-world failure mode: wrong camera placement. |
- | ``safety`` | object | ``contraindicationNote`` key + ``maxRecommendedReps`` (UI nudge only, never a hard block). |
- | ``status`` | draft / beta / live | Publishing state (§9.3). |

****Worked example – ``squat`` v5, expressing today's Python behavior exactly**** (this is the parity target for §7.5; abbreviated to the load-bearing parts):

```
```jsonc
{
 "key": "squat", "version": 5, "minEngineVersion": "1.0.0", "family":
"squat_pattern",
 "tracking": "pose",
 "views": ["front", "side"], "preferredView": "side", "requireView":
false,
 "calibration": [{ "module": "standing_baseline" }],
 "signals": ["knee_avg", "knee_L", "knee_R", "trunk_incline",
 "valgus_delta_L", "valgus_delta_R"],
 "rep": { "mode": "alternating_threshold", "metric": "knee_avg",
 "upAt": 160, "downAt": 100, "countOn": "up",
 "minRepMs": 900, "maxRepMs": 12000, "bilateralGate": 150 },
 "phases": ["standing", "descent", "bottom", "ascent"],
 "faults": [
```

```

 { "id": "shallow_depth", "when": "knee_avg_min > 130", "perRep": true,
 "severity": 25, "msg": "fault.squat.depth" },
 { "id": "trunk_lean", "when": "trunk_incline > 45", "sustainMs": 300,
 "severe": "trunk_incline > 60", "severity": 20, "msg":
"fault.squat.lean" },
 { "id": "knee_valgus", "view": "front", "phase": ["descent","ascent"],
 "when": "valgus_delta_min < -0.15", "severe": "valgus_delta_min <
-0.30",
 "perRep": true, "severity": 20, "msg": "fault.squat.valgus" }
],
 "scoring": {
 "components": [
 { "component": "depth", "input": "knee_avg_min",
 "curve": [[100,100],[130,30],[155,0]], "inactiveAbove": 155 },
 { "component": "trunk", "input": "trunk_incline",
 "curve": [[45,100],[75,0]] },
 { "component": "valgus", "input": "abs(valgus_delta_min)", "view":
"front",
 "curve": [[0.15,100],[0.30,0]], "inactiveWhenPositiveDrift": true
 }
],
 "neutral": 80, "correctAt": 70
 },
 "setup": { "cameraHint": "setup.squat.camera" },
 "status": "live"
}
...

```

**\*\*`tracking` (added v1.1):\*\*** ` "pose" | "timer"`, default ` "pose"`. Your seed already contains one non-camera entry (Brisk Walking, `ai\_supported: false`), so the schema must represent it honestly rather than pretending every catalog row is pose-tracked. ` "timer"` entries carry no `signals`/`fsm`/`faults`/`scoring` blocks – the linter *\*rejects\** them if present – and the app renders duration logging instead of the camera flow. This is also the future home of GPS-tracked entries on mobile (` "gps"` reserved, not implemented). One enum field now prevents a class of "why is the camera opening for walking?" bugs later.

---

## ## 5. The 12 family templates

A template = a pre-filled definition (FSM mode, phase model, default fault set, required signals, calibration, view policy) exposing a small parameter surface. Authoring exercise #23 means picking its template and filling parameters – hours, not weeks. v1 sketched ~11 families; building the real mapping against your actual catalog (§6) refined that to **\*\*12\*\*** – the deltas: bridges split from hinge (supine FSM + C3, not standing), holds split from mobility (a plank and a hamstring stretch share an FSM but not a fault philosophy), single-joint isolations unified (curl/extension/raise/calf-raise are one template with the metric swapped), and a dedicated cadence family for conditioning moves.

#	Template	FSM mode	Default metric	Core default faults	Calib.	Views
F1	`squat_pattern`	A	`knee_avg` 160/100	shallow depth, trunk lean, valgus (front)	C1	front+side
F2	`hinge`	A	`trunk_incline` (hip-driven: down = incline rises while knees stay > 140)	knees-bend-too-much (turns it into a squat), ROM short, jerk (tempo min)	C1	<b>**side req.**</b>
F3	`bridge_supine`	A (inverted: countOn "up" = hips peak)	`hip_elevation` vs C3 floor	incomplete lockout (hip angle < 160 at top), feet drift, hold-at-top missing	C3	side req.
F4	`lunge_split`	C (per-side)	front `knee` 160/100	front knee past toe (shin incline > 25 side-view), torso lean, shallow back knee	C1	side pref.
F5	`push_horizontal`	A	`elbow_avg` 160/90	half reps (elbow_min > 110), hip sag (body_line < 160), hip pike (> 195 equiv.), flared symmetry	–	side req.
F6	`push_vertical`	A	`elbow_avg` 160/90	incomplete lockout, asymmetric press (symmetry_elbow > 15), trunk lean-back > 20	C1	front pref.
F7	`pull`	A	`elbow_avg` 160/70	partial ROM, kip (hip_elevation oscillation), chin-short	C1	front
F8	`isolation_single_joint`	A	configured joint (elbow / shoulder / heel-elev.)	partial ROM both ends, swing (trunk_incline oscillation > 10), speed	C1	for elevation variants   front/side per exercise

```
| F9 | `core_dynamic` | A or C | `hip_avg` or knee-to-chest proxy | ROM
short, momentum (maxCadence), lower-back arch (body_line on leg raises) |
C3 | side req. |
| F10 | `hold_isometric` | B | exercise band (body_line 160-180 plank;
knee 80-100 wall-sit) | out-of-band (implicit), sag/pike cues, elbow
stacking | C1/C3 | side (plank), front (wall-sit ok) |
| F11 | `cadence_conditioning` | D | exercise periodic signal | amplitude
too small ("knees higher"), cadence out of band | C1 | front pref. |
| F12 | `mobility_hold_flow` | B (or D for cat-cow) | pose-specific band,
generous | none scored harshly – coaching tone is encouragement + hold
timer; score = hold time attained | – | per pose |
```

Template defaults are starting values; every concrete exercise tunes via traces (§7). F12 deliberately has a different *product* posture: stretches are guided-and-timed, not graded – grading a hamstring stretch's ROM without knowing the user's history reads as punishment, and these are your retention-friendly rest-day content (v1 §21 #14 territory).

---

## ## 6. The complete catalog mapping – every exercise, its template, its starting parameters, its risks

Ground truth (verified, v1.1): the canonical catalog is `scripts/seed\_exercises.py`, which seeds **exactly 58 exercises** into Mongo – matching your dashboard count. The earlier "57" was an artifact of counting media *filenames*: several misspelled GIF basenames (`hillraise` → Heel Raises, `soldierpress2.gif` → Shoulder Press's second demo, `hamstring2.gif` → Hamstring Stretch's second demo, `truckjump` → Tuck Jumps) are media aliases inside `frontend/src/utils/exerciseMedia.js`, not separate exercises – confirmed by decoding the GIF frames themselves. Two seeded exercises (**Mountain Pose**, **Brisk Walking**) are hidden by the frontend's `REMOVED\_EXERCISES` set, so the visible library is 56; both remain in this mapping because they exist in the system of record and need an explicit migration decision (§6, Tier 3). `arnold\_shoulder\_press` has a rep-counter config but is **not** in the 58 – it's noted as a ready-made expansion candidate, not counted.

**Tiers:** T1 = the 12-exercise gym-demo set (v1 Phase 4 bar) – chosen for landmark reliability, camera simplicity, and coverage of the movements a

skeptical trainer will actually test. T2 = next 17. T3 = remainder. ✓ = validated logic exists today and ports with its numbers.

### **\*\*Tier 1 – the demo dozen\*\***

Exercise	Family	Rep metric & thresholds	Primary faults (starting values)	View / Calib	Risk notes
squat ✓	F1	knee_avg 160/100, count up	depth >130 · lean >45 (severe 60) · valgus -0.15/-0.30	F+S / C1	None – parity target.
chair squat ✓	F1	knee 155/110	shallow > target+25 · lean: >50		
<b>**and**</b> trunk-shin >20   F+S / C1+C2   None – parity target.					
jump squat ✓	F1+airborne	knee 160/130	no-flight (hip_elev <0.15 in cycle) · stiff landing (knee >130 at touchdown) · shallow load (<40° flex) · lean 50(+shin 15)/55 front	F+S / C1	Raw-channel elevation (\$3.2) is mandatory.
push-up	F5	elbow 160/90 (your config)	half-rep elbow_min>110 · hip sag body_line<160 (your 160-180 rule) · pike	S req / –	Wrist landmarks jitter on the floor – metric uses elbows only.
wall push-up	F5	elbow 155/95	half-rep · hips-to-wall (trunk_incline < 10 means standing too upright)	S / –	
Beginner-critical for gym demos; easiest camera setup in the catalog.					
lunge	F4	front knee 160/100 (your config)	front-knee-past-toe shin>25 · torso lean >20 · back-knee shallow	S pref / C1	
Side-detection of <i>*which*</i> leg is forward comes free from Mode C routing.					
glute bridge	F3	hip_elevation vs floor, top = lockout	no lockout (hip angle <160 at peak) · partial height (<60% of calibrated best)	S req / C3	Supine landmark confidence drops – require side, mark first release beta.
plank	F10	hold: body_line 160-180 (your band)	sag <160 · pike >posture-max · elbow not under shoulder	S req / –	Your existing rule set maps 1:1 to the hold band.
wall sit	F10	hold: knee 80-100	too-high knee>110 sustained · heels lifting (ankle_elev)	F or S / C1	Very reliable; great demo.
bicep curl	F8 (elbow)	elbow 150/60 (your config)	partial ROM (min>70 or max<140, your 50-160 band) · swing trunk osc >10	F / C1	
Dumbbell occlusion of wrist is fine – elbow drives everything.					
shoulder press	F6	elbow 160/90 (your config)	lockout short (max<160, your 80-170 band) · asym >15° · lean-back >20	F / C1	

| high knees | F11 | cadence on knee\_L/R alternation | amplitude: knee must pass hip height ("higher!") · cadence <100/min | F / C1 | Amplitude via hip-relative knee\_y - visibility-robust. |

**\*\*Tier 2 – next 17\*\*** (compressed: metric → faults → view · risk)

Exercise	Family	Spec seed

| deadlift (RDL pattern) | F2 | hip-hinge: trunk\_incline rises to 60-90 while knee stays >140 → faults: knees-collapse-to-squat (knee<120 mid-rep), ROM short, tempo-jerk · **\*\*S req\*\*** · Honest limitation, stated in-app: spine *\*rounding\** is not directly measurable from 33 landmarks – we coach hinge *\*pattern\**, and say so. Never oversell; trainers respect the honesty in a demo. |

| hip thrust | F3 | as glute bridge with bench setup hint; lockout hip ≥170 · S · bench occludes hips at bottom – positionCheck gates start. |  
| bulgarian split squat | F4 | per-side, front knee 160/95; balance fault via stillness spikes · S · rear-foot elevation makes ankle vis low – metric ignores rear ankle. |

| step up | F4 | per-side knee 160/100 + hip\_elevation gain; fault: push-off-with-floor-leg (bilateral gate inverted) · S · needs C2-style height target – reuse `adaptive\_target` on hip\_elevation. |

| calf raises | F8 (heel-elev) | heel/ankle\_elevation vs C1, up/down on elevation band | partial raise, bounce (maxCadence) · S · smallest ROM in catalog – raw channel + generous bands; beta until traces confirm. |

| crunch | F9 | shoulder-lift proxy (shoulder\_y drop vs C3) or hip angle 140/110 · faults: neck-pull unobservable (excluded – don't fake it), momentum cadence cap · S. |

| leg raises | F9 | hip angle 170/95 lying · lower-back arch via body\_line during descent · S / C3. |

| mountain climber | F11 | cadence on alternating knee-to-chest; plank-line fault from F10 rules concurrently · S · the one exercise combining two templates' fault sets – supported: faults are just rules. |

| jumping jack | F11 | cadence on wrist-above-head + stance width oscillation; amplitude faults ("arms all the way up") · F. |

| superman | F9 (prone) | chest+thigh elevation vs C3 floor, hold-ish reps · S req · prone = worst-case landmark confidence; **\*\*beta\*\***, generous thresholds. |

| side plank | F10 | body\_line lateral (shoulder-hip-ankle) 160-180, per-side holds · S · elbow-under-shoulder check reused. |

| tricep dips | F6 | elbow 160/80; shoulder-shrug fault (shoulder\_y rise)  
· S · bench occlusion – camera hint critical. |  
| lateral raises | F8 (shoulder) | shoulder abduction 20/80-90; fault:  
shrug, swing, over-raise >100 · F. |  
| russian twist | F9 (rotation) | oscillation of shoulder-line x-span  
(side-to-side), Mode D counting; fault: feet-down (if variant), tempo · F  
· rotation via x-span oscillation, **\*\*not z\*\*** (I-noisy) – twist *\*depth\** is  
approximate; framed as tempo/count coach. |  
| tricep extension | F8 (elbow, overhead) | elbow 160/70 overhead;  
elbow-flare via symmetry+elbow\_x drift · S. |  
| lunge jump | F4+D hybrid | per-side landing detection + flight (hip\_elev  
raw) · S · uses jump-squat's airborne module verbatim. |  
| skipping rope | F11 | cadence on ankle/hip elevation micro-oscillation;  
rope invisible – count jumps, coach rhythm only, never "form" · F · set  
expectations in-app copy. |

**\*\*Tier 3 – remaining 29\*\*** (one line each; 28 pose-tracked through the same  
pipeline, 1 timer-only)

pull-up (F7 – hardest camera story in the catalog: bar height forces  
low-angle camera; kip detection via hip\_elevation oscillation; ship late,  
beta, with explicit camera hint) · bench press (F5 side-view low camera;  
bar path not visible – elbow ROM + tempo only, stated honestly) · pike  
push-up (F6) · resistance band pull (F7, wrist-x span 0.4→1.1  
torso-widths) · burpees (F11 composite cycle: floor→jump detection chain)  
· plank jack (F11 + F10 concurrent) · skater jump (F11 lateral hip\_x  
oscillation) · tuck jump *\*(media alias `truckjump` – confirmed from GIF  
frames)\** (F11 + airborne + knee-to-chest amplitude) · jogging in place  
(F11) · flutter kick (F9 cadence on alternating ankle\_y, small amplitude –  
beta) · bicycle crunch (F9 per-side) · seated leg raise (F9) · heel raise  
*\*(media alias `hillraise` – GIF frames show unsupported standing heel  
raise; distinct catalog entry from Calf Raises, which is the supported  
variant)\** (F8, elevation vs C1, generous bands, beta until traces) · **\*\*arm  
circles\*\*** (F11/F8 hybrid: wrist circular oscillation via wrist\_y + wrist\_x  
phase, Mode D cadence; amplitude fault "full circles" via wrist-path span;  
F view – trivially reliable, good warm-up demo) · **\*\*mountain pose\*\*** (F12  
posture hold: stillness score + trunk\_incline < 5° + arms-at-side;  
currently hidden by frontend `REMOVED\_EXERCISES` – *\*migration decision:\**  
reinstate as a 30s "posture reset" between mobility flows, or drop from  
the Postgres seed; both are one-line changes) · **\*\*brisk walking\*\***

(`tracking: "timer"` – the catalog's one non-camera entry, already  
`ai\_supported: false` in your seed; no pose definition; at migration  
either reclassify under the running/steps module (mobile GPS) or  
timer-only logging on web – do **\*\*not\*\*** delete: gym walking-warmup programs  
reference it) · **\*\*mobility thirteen\*\*** (F12): cat-cow (Mode D slow cycles  
on trunk flexion oscillation), child's pose, cobra, downward dog (body  
inverted-V: hip angle 60-100 band), **\*\*bridge pose\*\*** (supine backbend  
\*hold\* – same landmark story as glute bridge but F12 hold semantics, not  
F3 reps: hip\_elevation ≥ calibrated band sustained; S req / C3; beta with  
superman), hamstring stretch *\*(its second demo GIF is the misnamed  
`hamstring2.gif` – one entry, not two)\**, hip-flexor stretch, seated  
forward bend, shoulder stretch, world's greatest stretch (3-position flow  
= sequenced holds – the only *\*sequenced\** definition; template supports  
ordered hold stages), tree pose (stillness-scored balance hold – great  
share-card moment), warrior I (front knee 100-130 + arms-overhead band),  
**\*\*warrior II\*\*** (front knee 100-130 + arms-horizontal band via wrist\_y ≈  
shoulder\_y ±0.05 torso-heights + stance width > 1.4 shoulder-widths; F  
view – the width signal makes it *\*front\**-view unlike Warrior I) – all:  
generous bands, timer-first UX, no harsh scoring.

*\*Expansion candidate (not in the 58):\** `arnold\_shoulder\_press` – a  
validated rep-counter config (160/90) already exists with no catalog entry  
or media; adding it later is one seed row + one definition inheriting  
shoulder press with rotation unscored.

Coverage check: **\*\*12 + 17 + 29 = 58 – every seeded exercise mapped, zero  
open slots.\*\*** (Shoulder Press's strict variant and the "hamstring curl"  
that earlier drafts listed were media-alias ghosts, now removed; Warrior  
II, Bridge Pose, Arm Circles, Mountain Pose, and Brisk Walking were the  
genuinely missing five.) Live visible today = 56 (58 – the two  
`REMOVED\_EXERCISES`).

---

## ## 7. The golden-trace harness – how "no bugs" is actually engineered

This is the quality system. Budget ~2 days to build; it protects every  
exercise forever after.

### ### 7.1 Trace file format

One recorded session = one file: a JSON header + JSONL frames (streams cheaply, diffs cleanly in git):

```
```jsonc
// header line
{ "traceVersion": 1, "exercise": "squat", "recordedWith": { "engine":
"1.0.0", "defs": 41 },
  "device": "poco-x5", "platform": "web", "fps": 15.2, "view": "side",
  "label": "clean_10_reps",
  "expected": { "reps": 10, "faultsExact": {}, "scoreRange": [82, 95],
                "formCorrectAll": true } }
// then one PoseFrame per line
{ "t": 0.0, "kp": [ ... ] }
{ "t": 66.9, "kp": [ ... ] }
```
```

``expected`` is authored by *\*you watching the video once\** and locking the truth – after that, CI is the reviewer. Raw video is never stored (privacy invariant); the trace is stick-figure data only, ~1-2 MB per 10-minute session uncompressed, ~150 KB gzipped in the repo.

### ### 7.2 Recording mode

A dev-flag in the web app (and later mobile) adds a "record trace" toggle to ActiveWorkout: it tees the exact PoseFrames the engine consumes into a downloadable file. Because it captures *\*engine input\**, a trace recorded on web replays identically on mobile CI – that's I2 and I3 doing their job.

### ### 7.3 Required fixture matrix – per exercise, before it may leave ``beta``

At least **\*\*6 traces\*\***: (1) clean set, 8-12 reps · (2-3) one trace per *\*major\** fault, deliberately performed · (4) occlusion/interruption – walk partially out of frame mid-set, return · (5) wrong-view attempt (front-only exercise done side-on) – expected: cue fires, nothing miscounts · (6) speed extremes – rushed reps and grinding-slow reps. Isometrics swap (6) for a wobble-and-recover hold. The matrix is a checklist in the authoring workflow (§9.1), not folklore.

### ### 7.4 Assertions & tolerances

Rep count: **\*\*exact\*\***. Fault multiset: **\*\*exact\*\*** for scripted-fault traces; clean traces assert *\*empty\**. Scores: within the trace's declared ``scoreRange`` ( $\pm 5$ -point authoring slack absorbs smoothing-window edge effects). Hold time:  $\pm 700$  ms. Phase sequences: optionally asserted for FSM-sensitive traces. A failing assertion names the first divergent frame — debugging is "open frame 412", not archaeology.

### ### 7.5 The parity protocol (Python → TypeScript port gate)

Before ``pose_ws.py`` is decommissioned: (1) run the *\*current\** system in a capture mode that logs, per frame, both the incoming keypoints and the server's full response; (2) record **\*\* $\geq 6$  squat, 4 jump-squat, 4 chair-squat\*\*** real sessions this way, across front and side views, including your known real footage cases (the chair-squat clips bottoming at  $\sim 62^\circ$  and  $\sim 71\text{--}90^\circ$  that your comments cite); (3) these become traces whose ``expected`` values are *\*the Python outputs\**; (4) the TS engine must match rep counts exactly and scores within  $\pm 3$  before the port is called done. This converts "I think the port is faithful" into a green checkmark.

### ### 7.6 CI wiring & the performance gate

Every push: lint definitions → unit tests (signal formulas against hand-computed geometry fixtures) → **\*\*replay all traces\*\*** → perf bench (replay a 10-minute trace in a tight loop; assert p95 frame cost  $\leq 3$  ms scaled to the CI baseline machine, and stable heap) → fuzz pass (random NaN injection, landmark dropout bursts, fps jitter 8-40 on a clean trace; assert: no throw, no count *\*increase\**, recovery after signal returns). A threshold tweak that would break squat counting fails the build before any gym member ever sees it — this, concretely, is the "no bug" machinery.

---

## ## 8. Porting plan — nothing validated gets lost

### ### 8.1 Constant-preservation table (the audit trail for the port PR)

| Today (file · constant) | Value         | Lands in  |                         |
|-------------------------|---------------|-----------|-------------------------|
| --- --- ---             |               |           |                         |
| rep_counter             | squat up/down | 160 / 100 | squat `rep.upAt/downAt` |

```

| rep_counter jump squat down | 130 | jump_squat `rep.downAt` (+ rationale
comment carried into def description) |
rep_counter chair up/down	155 / 110	chair_squat `rep`
smoothing window	7 frames	engine conditioning (§3.2)
min down / up frames	3 / 2	FSM debounce defaults
bilateral engage angle	150°	`bilateralGate` default
min rep interval	0.45 s	engine floor; defs add `minRepMs: 900`
SQUAT_STANDING_KNEE_MIN / SHALLOW / PARALLEL	155 / 130 / 100	depth
curve `[[100,100],[130,30],[155,0]]`		
SQUAT_TRUNK_LEAN_MAX (+severe)	45 (+15→60)	trunk fault
`when`/`severe` + curve `[[45,100],[75,0]]`		
VALGUS flag / severe	0.15 / 0.30	valgus fault + curve
neutral score / correct bar / corrections cap	80 / ≥70 / 2	scoring
`neutral`, `correctAt`; coaching policy		
_StandingCalibration frames / knee min	8 / 160°	C1 params
CHAIR fallback / tolerance / shallow margin / standing	100 / 10 / 25 /	
155	C2 fallback + chair curves	
CHAIR trunk absolute / shin delta	50 / 20	chair lean fault (relative
check)		
JUMP airborne elevation / stiff landing / loading depth / lean S+F		
0.15 / 130 / 140 / 50+15, 55	jump_squat airborne module + faults	
visibility threshold	0.3	conditioning gate (+0.15 hysteresis floor,
new)		
view thresholds	side <0.15/<0.15 · front >0.20 or >0.15	view
classifier (§3.3)		
FORM_RULES bands (push-up, lunge, plank, curls, presses)	as coded	
T1/T2 definitions' starting fault bands (§6) |

```

### ### 8.2 Sequence (fits v1 Phase 1, weeks 2-4)

Week 2: engine skeleton – contracts, conditioning, view, signals 1-17, C1, Mode A, DSL parser, scorer; harness built; parity traces recorded *\*first\** (while the Python path still runs). Week 3: squat/jump/chair definitions to parity green; C2, airborne module; web `usePoseDetection` swapped to local engine behind a flag (the hook's public API barely changes – `ActiveWorkout` mostly doesn't notice); `/v1/workouts/sync` live. Week 4: Modes B-D, C3, signals 18-22; T1's nine new exercises drafted to `beta`; flag flipped, WS path deleted. Exit = v1 Phase 1's acceptance line: full workout with the API server switched off, then a clean sync.

**\*\*Cleanup item (verified dead code, v1.1):\*\***

`frontend/src/ai/FormClassifier.js` and the 17 MB  
`public/models/ctr\_gcn\_clean\_ensemble\_quant.onnx` it loads are imported by  
**\*\*nothing\*\*** in the app – a remnant of the ML-classifier path you correctly  
abandoned for rule-based (the right call: no server inference cost, no  
dataset-limitation ceiling, deterministic and auditable). Delete both plus  
the `onnxruntime-web` dependency in Week 2. It costs nothing at runtime  
today but is 17 MB of deploy weight, a misleading signal to future-you,  
and a dependency with its own security-update surface.

---

**## 9. Definition lifecycle: authoring → publishing → rollback**

**### 9.1 Authoring workflow (the production line)**

Pick exercise → copy family template → fill parameters (\$6 row is the  
seed) → derive/verify thresholds via the protocol below → self-record the  
\$7.3 fixture matrix → adjust until traces green locally → PR (definitions  
are code-reviewed data; the diff *is* the behavioral change) → CI green →  
status `beta`.

**\*\*The threshold-derivation protocol (v1.1 – your existing method,  
formalized and upgraded).\*\*** The method that produced the validated  
squat/chair/jump thresholds – extract joint angles from good-form videos  
in Colab, cross-check against published exercise research – is the right  
method; MediaPipe-measured numbers beat textbook numbers because they  
include MediaPipe's own systematic biases. Keep it, with four upgrades  
that fix its blind spots:

1. **\*\*3-5 good-form videos, not 2\*\*** – different body proportions (limb  
ratios, torso length) shift measured angles by 5-15° for the *same*  
quality of movement. Two videos give you the center of the distribution;  
they cannot give you its width. From 3-5 sources, take the **\*\*p10-p90  
band\*\*** of the target metric at each phase (bottom, lockout, etc.), not a  
single value. The pass threshold sits at the *permissive* edge of the  
band; the coaching-cue threshold sits at the strict edge. Where a  
subject's own standing baseline matters more than the population (valgus,  
chair depth), the C1/C2 calibration modules absorb the variance – that's  
what they're for.

2. **\*\*2-3 deliberate-fault videos per fault rule\*\*** – record yourself intentionally shallow, intentionally leaning, intentionally caving the knees. A fault threshold set only from good reps is a guess about where "bad" begins; a threshold set with true fault positives is a measurement. Every fault rule in a definition must have at least one trace where it fires and one where it correctly stays silent (this is already the §7.3 fixture requirement – the fault videos are those fixtures).

3. **\*\*Research pass is corroboration, not source\*\*** – the desk-research step (published ROM norms, coaching standards) stays, but its role is to *\*sanity-check\** the video-derived band, flag safety-relevant limits (e.g., knee-past-toe conventions), and supply coaching language. When research and your measured band disagree, trust the measurement and write one line in the definition PR explaining the delta – that line is gold six months later.

4. **\*\*Numbers land in the definition JSON, never in code\*\*** – the Colab notebook's output is a filled §4 definition (plus its band evidence pasted into the PR description). The same session's videos become the golden traces. One recording session per exercise therefore yields the thresholds *\*and\** the regression fixtures – that's the whole production line for the 2-3/week cadence in §10.

### ### 9.2 The linter (runs in CI and on publish)

Rejects: unknown signal references; rules/components referencing signals absent from `signals`; view-scoped rules in definitions that exclude that view; curves not monotonic in x; `upAt ≤ downAt`; hold bands inverted; missing `msg` keys or keys absent from the locale table; `severe` without a base fault; missing camera hint; fixture matrix incomplete for a `live` promotion. Every rejection message says which field and why – authoring stays a one-person-friendly loop.

### ### 9.3 Publishing & staged rollout

Definitions live in Postgres (`exercise\_definitions`, v1 §7.1) and ship as the **\*\*signed, ETag-cached bundle\*\*** (v1 §5.2): `beta` definitions are included only for flagged users (you + consenting pilot-gym members – one feature flag); `live` for everyone. Promotion `beta → live` requires: fixture matrix complete, ≥ 1 week of beta telemetry with fault-rate and reps-per-set distributions inside expected bands (a fault firing on 95% of reps means the threshold is wrong, not that everyone squats badly – the

telemetry check catches exactly this). Rollback = repoint the bundle to the previous version row; clients pick it up on next fetch; no deploy, no app-store anything, in either direction.

### ### 9.4 Post-publish telemetry to watch (per definition, on your admin panel)

Fault-rate per fault id (distribution, not average) · reps-per-set distribution vs. family norm · % sessions ending < 60 s (rage-quit proxy — usually a setup/camera problem) · view distribution vs. `preferredView` · score histogram shape. These five catch every bad-threshold incident class I can anticipate, cheaply.

---

## ## 10. Production cadence, and definitions of done

**\*\*Cadence:\*\*** with templates and harness in place, budget **\*\*2–3 exercises/week\*\*** sustainably (each ≈ half a day of parameterization + one recording session + tuning). T1's nine new exercises land inside v1 Phase 4's window; T2 completes ~5 weeks later; the full 59 well inside a quarter — while the gym pilots run on T1, which is the point: gyms buy on 12 solid, not 59 (unchanged conclusion from the business discussion).

**\*\*Definition of done — engine v1:\*\*** all §1.3 invariants enforced in CI · parity traces green (§7.5) · perf gate green on the mid-Android profile · fuzz pass green · web app runs a full offline workout through the local engine · sync payload byte-matches §2.4.

**\*\*Definition of done — any exercise:\*\*** linter green · fixture matrix complete and green · beta telemetry week clean · camera-hint copy + all message keys in EN (HI/AS keys may lag but must exist) · §6 risk note either resolved or surfaced as honest in-app copy.

---

## ## Appendix A — Message-key seed (from your current strings, so nothing regresses)

```
`fault.squat.depth` "Squat lower – aim to get your hips level with your knees" · `fault.squat.lean` "Keep your chest up – you're leaning too far forward" · `fault.squat.valgus` "Push your knees out – don't let them cave inward" · `fault.body.visibility` "Cannot see your legs clearly – step back so your full body is in frame" · `fault.pushup.depth` "Lower your chest closer to the floor" · `fault.pushup.line` "Keep your body in a straight line" · `fault.pushup.sag` "Don't let your hips sag" · `fault.lunge.frontknee` "Bend your front knee to 90 degrees" · `fault.lunge.backknee` "Bend your back knee toward the floor" · `fault.lunge.torso` "Keep your torso upright" · `fault.plank.hips` "Raise your hips – keep body straight" / "Don't let your hips drop" · `fault.plank.elbows` "Keep elbows directly under shoulders" · `fault.curl.rom` "Full range of motion – curl all the way up" · `fault.press.lockout` "Press all the way up" · plus new `cue.turn_to_front`, `cue.turn_to_side`, `setup.<exercise>.camera` family. Hindi/Assamese columns are a translation task, not an engineering one – schedule with a native speaker before T1 goes to pilot gyms (it's a demo-day differentiator, v1 $21 #10).
```

## ## Appendix B – Reconciliation log (RESOLVED, v1.1 – evidence-verified against your repo)

All six questions closed. Method: decoded frames from every ambiguous GIF, cross-referenced `frontend/src/utils/exerciseMedia.js` (the name-media map), and treated `scripts/seed\_exercises.py` (58 entries, unique-name index) as the canonical catalog.

1. `hillraise` → **\*\*Heel Raises\*\*** – real catalog entry (seed line ~1424). GIF frames show an unsupported standing heel raise; `exerciseMedia.js` maps the files to the `heel raises` key. Distinct from **\*\*Calf Raises\*\*** (the supported/holding-on variant per its own GIF). *\*Not\** hill sprints – no running content in the media; hill/incline running belongs to the GPS module if you ever want it.
2. `hamstring2.gif` → the **\*\*second demo GIF of Hamstring Stretch\*\***, not a separate hamstring-curl exercise (`hamstring stretch': [..., 'hamstringstretch1.gif', 'hamstring2.gif']`). The "hamstring curl" row in earlier drafts was a ghost – removed.
3. `soldierpress` → **\*\*media alias of Shoulder Press\*\*** (`shoulder press': ['soldierpress.jpg', 'shoulderpress1.gif', 'soldierpress2.gif']`). GIF confirms a standing overhead press. The separate "military press" row was

a ghost – removed; the strict-form lean-back fault ( $\leq 10^\circ$ ) is now simply part of the single Shoulder Press definition.

4. `truckjump` → **\*\*Tuck Jumps\*\***, real entry, confirmed from GIF frames (knees-to-chest airborne). Misspelling only.

5. Count: **\*\*58, not 59 or 57.\*\*** The seed script is unambiguous (58 inserts, unique index on `name`). Your dashboard figure was correct. The five entries earlier drafts missed: Warrior II, Bridge Pose, Arm Circles, Mountain Pose, Brisk Walking – all now mapped in §6. The frontend hides Mountain Pose + Brisk Walking via `REMOVED\_EXERCISES`, hence 56 visible. Nothing is missing; no exercises need inventing.

6. T1 demo-dozen: stands as specified in §6 (no objection raised). Swappable at any time – T1 membership is a tag on the definition, not engine structure.

**\*\*Two product decisions surfaced by the reconciliation (make them during Part 4's migration, both one-liners):\*\*** (a) Mountain Pose – reinstate as a posture-reset hold or drop from the Postgres seed; (b) Brisk Walking – reclassify `tracking: "timer"` on web and fold into the running module on mobile, or timer-only forever.

*\*– End of Part 2. Next deliverables, one at a time as agreed (v1 §23):  
Part 3 Gym Dashboard screen-by-screen · Part 4 Database DDL & migration ·  
Part 5 Billing & webhooks · Part 6 Mobile deep spec · Part 7 Retention  
playbook · Part 8 Ops runbook.\**

## **# Part 3 – The Organization Console (Gyms · Studios/PT · Physio Clinics): Full Specification**

**\*\*Prerequisites:\*\*** Architecture v1 (§8 tenancy, §9 plans, §11 sketch) · Part 2 (SetSummary contract §2.4, T1/T2/T3 tiers, message-key i18n) · Part 2B (display standard §6, org segments & clinic positioning §7).

**\*\*Scope:\*\*** the owner/staff-facing web console for every organization type, screen by screen – every widget's data contract, every query it reads, every empty/edge state – plus the org-type deltas and the build slices. The **\*\*member-facing\*\*** app is out of scope except where the console creates obligations in it (visibility disclosure, leaderboard opt-out, join flow).

**## 0. Amendments to Architecture v1 (recorded here so the document set stays consistent)**

1. **\*\*Secrets addendum (Part 2 §0):\*\*** beyond v1 §18's list, rotate `VITE\_LOCATIONIQ\_KEY` (shipped inside the public JS bundle – readable by anyone today), `RAPIDAPI\_KEY`, `USDA\_API\_KEY`. Geocoding moves behind the server `geo` module as already planned.
2. **\*\*Meal photos are never persisted\*\*** (Part 2B §3 supersedes v1 §7.3's `meal-photos/` R2 bucket): analysis is in-memory, one vision call, photo discarded. Delete that bucket from the infra plan; the privacy line "photos are analyzed, never stored" appears in the scan UI and in every org-facing privacy sheet – it matters double for clinics.
3. **\*\*"Gym Dashboard" → "Organization Console."\*\*** Four segments (new gyms, premium-aspiring gyms, boutique/PT studios, physio clinics) resolve to **\*\*three product org types\*\*** – `gym` (both gym segments; "premium-aspiring" is a *\*pitch\**, not a product fork), `studio` (boutique/PT), `clinic` – implemented per Part 2B §7 as vocabulary + feature-matrix overrides on v1 §8's tenancy. No new architecture.

---

## ## 1. Personas & jobs-to-be-done (what each screen must answer in one glance)

| Persona                                 | Role                                   | The question they open the console with                                                        | The number that decides renewal       |
|-----------------------------------------|----------------------------------------|------------------------------------------------------------------------------------------------|---------------------------------------|
| ---                                     | ---                                    | ---                                                                                            | ---                                   |
| Gym owner (Jorhat-tier, 60-300 members) | `owner`                                | "Are my members actually using this, and is it keeping them from quitting?"                    | <b>**Adoption %**</b> + at-risk saves |
| Gym manager / front desk                | `manager`                              | "Get this walk-in signed up in 30 seconds"                                                     | time-to-join                          |
| Trainer / Coach                         | `trainer`                              | "Who needs my attention today, and is their form improving?"                                   | member form-score trend               |
| Studio / PT owner (10-40 clients)       | `owner`                                | "Which of <i>*my*</i> clients trained this week, on the plan I gave them?"                     | per-client adherence                  |
| Clinic clinician (Phase 2)              | `trainer` (vocab: <i>*Clinician*</i> ) | "Did my client do their assigned home exercises, safely?"                                      | <b>**adherence %**</b> per client     |
| Kd (internal)                           | `admin`                                | – lives on the separate admin panel (v1 §6.1, Part 2 §9.4), <b>**not**</b> in this console   – |                                       |

Design law derived from these: the console is a **\*\*retention instrument the org uses on its members\*\***, not an analytics toy. Every screen ends in an action (nudge, print, invite, upgrade) – data that doesn't lead to an action is decoration and gets cut.

---

## ## 2. Org model, roles, vocabulary & feature matrix

### ### 2.1 Additions to the v1 \$8 schema (columns only; DDL lands in Part 4)

`gyms` (table keeps its name; it's the *\*org\** table) gains: `org\_type` ('gym' | 'studio' | 'clinic'), `timezone` (IANA, set at onboarding, default from browser), `locale`, `currency\_display`, `owner\_included\_as\_member` bool default true, `activation\_jsonb` (checklist state, \$5.1).  
`gym\_members` gains `removed\_at` timestampz null – membership is a **\*\*validity interval\*\*** `[joined\_at, removed\_at)`; every org rollup counts a workout iff the user was a member **\*\*on the workout's date\*\*** (this one rule kills a whole class of "my numbers changed" bugs: history never re-attributes when people join or leave).  
`gym\_codes` gains `label` (v1: one code; now multiple named codes per org – "Front Desk", "Morning Batch", "Dr. Sen's clients" – each is a **\*\*group tag\*\*** on the memberships it creates; this is the entire mechanism behind studio cohorts and clinic caseloads, at the cost of one column).

### ### 2.2 Roles & permissions matrix (enforced server-side per route; UI merely hides)

| Capability                                      | owner | manager | trainer                        |               |
|-------------------------------------------------|-------|---------|--------------------------------|---------------|
| Overview, Leaderboard, Reports (view)           | ✓     | ✓       | ✓                              |               |
| Members list + detail                           | ✓     | ✓       | <b>**assigned/group only**</b> | (gym: all)    |
| Invite (share code / print poster)              | ✓     | ✓       | ✓                              |               |
| Remove / restore member                         | ✓     | ✓       | –                              |               |
| Create / rotate / expire codes                  | ✓     | ✓       | –                              |               |
| Send "we miss you" nudge                        | ✓     | ✓       | ✓                              | (own members) |
| Billing (view, upgrade, payment method, cancel) | ✓     | –       | –                              |               |
| Staff management                                | ✓     | –       | –                              |               |
| Settings (profile, logo, vocabulary, privacy)   | ✓     | ✓       | (no privacy)                   | –             |
|                                                 |       |         |                                |               |

|                                |   |   |   |
|--------------------------------|---|---|---|
| TV-mode token create/revoke    | ✓ | ✓ | — |
| CSV export (members, activity) | ✓ | ✓ | — |

Vocabulary overrides (Part 2B §7), applied by org\_type through the same message-key system as the member app — zero forked screens: `gym`: Members / Workouts / Trainer · `studio`: Clients / Sessions / Coach · `clinic`: Clients / Sessions / Clinician. Copy rules for clinic strings are **\*\*linted\*\*** exactly like Part 2's definition linter: the words "rehab", "treatment", "therapy", "diagnosis", "recovery %" are build failures in clinic-visible keys. Adherence and activity — nothing clinical, ever.

### ### 2.3 Feature matrix by org type

| Feature                           | gym          | studio               | clinic                                                                                                                  |
|-----------------------------------|--------------|----------------------|-------------------------------------------------------------------------------------------------------------------------|
| Leaderboard + TV mode + poster    | ✓ default on | ✓ (per-group boards) | <b>**off by default**</b> (patients ranking each other is harm, not motivation; owner may enable for wellness programs) |
| Challenges (v1.1)                 | ✓            | ✓                    | off                                                                                                                     |
| At-risk list                      | ✓            | ✓                    | ✓ (renamed "Inactive clients")                                                                                          |
| Groups (named codes)              | optional     | <b>**core**</b>      | <b>**core**</b> (caseloads)                                                                                             |
| Trainer scoping to group          | optional     | ✓                    | ✓                                                                                                                       |
| Program/plan assignment surfacing | v1.5         | v1.5 core            | Phase 2 core                                                                                                            |
| Adherence report per client       | —            | v1.5                 | Phase 2 core                                                                                                            |
| Monthly org report PDF            | ✓            | ✓                    | ✓ (adherence framing)                                                                                                   |

### ### 2.4 The org-visibility boundary (a promise, not a setting)

Organizations see, for their members and only during the membership interval: **\*\*workout activity\*\*** (dates, exercises, sets/ reps/hold time, duration), **\*\*form scores & fault categories\*\*** (from `SetSummary` — aggregates and per-set values), **\*\*streaks and challenge participation\*\***. Organizations **\*\*never\*\*** see: meal logs or nutrition anything, body weight/measurements, AI-coach conversations, run GPS routes, or activity from before joining / after leaving. This boundary is (a) enforced in the `reports`/`gyms` repos (queries physically cannot join those tables for org callers), (b) disclosed to the member at join time — the join screen shows a "What {org} can see" sheet with exactly this list — and (c) restated in Settings → Privacy on both sides. For clinics this disclosure doubles as the consent record (DPDP/GDPR): joining a clinic code =

explicit consent checkbox, timestamped in `gym\_members.consent\_at`. This is the Trust Layer's doctrine applied to B2B: the org console is powerful *\*because\** its limits are legible.

---

## ## 3. Information architecture & data contracts

### ### 3.1 Navigation

Responsive web app (owners live on phones; **\*\*no native console app\*\***). Left rail (desktop) / bottom tabs (mobile): **\*\*Overview · Members · Leaderboard · Reports · Billing · Settings\*\*** (+ **\*\*Staff\*\*** under Settings; + **\*\*Groups\*\*** surfaced as a filter everywhere rather than a screen). A persistent trial/renewal banner slot sits above all screens (\$4.2). URL scheme `/console/:orgSlug/...` – the console is a separate route group in `apps/web` (v1 \$4's `apps/dashboard` folds into `apps/web` as a route group; one deploy, shared auth, less infra – recorded as the implementation choice).

### ### 3.2 The rollup layer (every widget reads these; nothing queries raw tables ad hoc)

| Source                         | What                   | Refresh                                                                                                            | Notes                                                                                                                                                 |
|--------------------------------|------------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| ---                            | ---                    | ---                                                                                                                | ---                                                                                                                                                   |
| `org_daily_stats`              | (table, worker-built)  | per org × day: `active_members, workouts, sets, total_reps, minutes, avg_form_score, new_members, removed_members` | nightly at 02:00 <b>**org TZ**</b> + rebuild-on-demand (admin)   day boundaries in `gyms.timezone`; membership-interval rule \$2.1 applied here, once |
| `org_live_counters`            | (Redis, 5-min TTL)     | today's workouts, actives-today                                                                                    | on read, cached 5 min   powers "today" tiles without touching PG per view                                                                             |
| `lb:org:{id}:{yyyymm}:{board}` | (Redis ZSET, v1 \$7.2) | leaderboard standings                                                                                              | updated at each workout sync   boards: `formscore` (avg, min 8 scored sets in month), `workouts`, `streak`                                            |
| `org_member_stats`             | (view)                 | per member: last_active_at, workouts_7d/30d, avg_form_30d, joined_at, group                                        | live view over indexed columns   Members table + detail drawer                                                                                        |

| `at\_risk` (query over the above) | §4.3 definition | computed on page load, cached 1 h | capped list, ordered by value |

**\*\*Definitions (canonical – reports, tiles, and sales decks must all match these):\*\***

- **\*\*Active member (Nd):\*\***  $\geq 1$  synced workout in the last N days (org TZ), while a member.
- **\*\*Adoption %:\*\***  $\text{active\_30d} \div \text{current members}$ . **\*\*Utilization %:\*\***  $\text{current members} \div \text{seat cap}$ . (Two different numbers; the Overview shows adoption, Billing shows utilization – never blur them.)
- **\*\*At-risk:\*\*** current member · joined > 14 days ago · had  $\geq 1$  workout in their first 21 days or in the prior 30-day window · **\*\*0 workouts in the last 14 days\*\***. Sorted by lifetime workouts desc (save the most invested first), capped at 20.
- **\*\*Avg form score:\*\*** mean of `SetSummary.avgFormScore` over scored sets in window; displayed as an integer per Part 2B §6; suppressed (–) below 5 scored sets.

**### 3.3 Console API surface (consumed only by these screens; feeds Part 4/5)**

`GET /v1/orgs/:id/overview` (tiles + 8-wk series + at-risk) · `GET /members?query&group&sort&cursor` · `GET/POST /members/:uid/nudge` · `DELETE /members/:uid` (soft: sets `removed\_at`) · `GET /leaderboard?board&month` · `POST /tv-token` / `DELETE /tv-token/:id` · `GET /reports` / `POST /reports/generate` · `GET /billing` (+ Part 5 endpoints) · `GET/PATCH /settings` · `GET/POST/PATCH /codes` · `GET/POST/DELETE /staff` · `GET /export/members.csv`. All org-scoped routes pass the RBAC middleware (§2.2) and are rate-limited per user; every mutating call writes `audit\_log`.

---

## **## 4. Screen-by-screen specification**

Format per screen: **\*\*Purpose** → **Zones/Widgets (with data contract & refresh)** → **Actions** → **States (loading/empty/error/edge)** → **Events emitted.\*\***  
Loading state is skeletons everywhere (never spinners-over-blank); every error state has a retry and never dead-ends.

**### 4.0 Onboarding wizard (first-run; also the self-serve trial start)**

**\*\*Purpose:\*\*** stranger → org on trial with a printed poster in **\*\*≤ 5 minutes\*\***. The metric that predicts everything downstream is **\*\*TTFMJ – time to first member joined\*\*** (instrumented start-to-join).

Steps (each skippable except 1-2; progress persisted per step so abandonment resumes):

1. **\*\*Org basics\*\*** – name, city, `org\_type` picker (three cards with one-line descriptions; picking `clinic` shows the positioning line from Part 2B §7 verbatim and the consent implications), timezone (prefilled), locale.
2. **\*\*Size & plan\*\*** – member-count slider → recommended seat tier (v1 \$9.2 tiers; `studio`/`clinic` see the **\*\*Micro tier, ≤ 25 seats\*\*** – price proposed at ₹999/mo gym-studio / ₹1,499 clinic, **\*\*final numbers ratified in Part 5\*\***) → **\*\*Start 7-day free trial\*\*** (no card required; `plans.trial\_days`; pilot codes redeemable here extend `trial\_ends\_at` per v1 \$9.2).
3. **\*\*Branding\*\*** – logo upload (PNG/JPG ≤ 2 MB → R2 `gym-assets/`, server-resized to 512px; shown live on a member-app preview so the owner *\*sees\** the white-labeling immediately).
4. **\*\*First join code\*\*** – auto-created (`label:"Front Desk"`, human-safe 6-char alphabet, no 0/O/1/I); **\*\*QR poster PDF\*\*** generated inline (worker job, ≤ 10 s, \$4.5) with a big **\*\*Print\*\*** button and a **\*\*WhatsApp share\*\*** button (poster + code as an image – this is how Assam actually distributes it).
5. **\*\*Add your team\*\*** – invite manager/trainer by email/phone (skippable).
6. **\*\*Done\*\*** → Overview, with the **\*\*Activation checklist\*\*** pinned (\$5.1) and, if `owner\_included\_as\_member`, the owner silently joined as member #1 (complimentary, not seat-counted) – so the demo works on their own phone in the parking lot.

States: resume-on-return · logo upload failure → keep going, checklist carries it · code collision → regenerate silently. Events: `org\_created{type}`, `org\_trial\_started`, `org\_logo\_added`, `org\_poster\_downloaded`, `TTFMJ` timer start.

### ### 4.1 Overview

**\*\*Purpose:\*\*** the renewal argument, self-served daily. One screen: are members using it, is it trending right, who needs saving.

| Zone | Widget | Data (source · refresh) | Notes / states |  
|---|---|---|---|  
| Banner | Trial / renewal / payment state | `subscriptions` · live | \$4.2  
state table |  
| KPI row | **\*\*Active members (30d)\*\*** with 7d sub-stat · **\*\*Workouts this week\*\*** · **\*\*Adoption %\*\*** · **\*\*Avg form score\*\*** | tiles: `org\_daily\_stats` + `org\_live\_counters` · nightly + 5-min | each tile shows Δ vs previous period (▲▼, org TZ weeks); form tile suppressed < 5 scored sets; adoption tile tap → Members filtered to inactive |  
| Chart | 8-week trend — workouts (bars) + active members (line) | `org\_daily\_stats` rolled to weeks · nightly | empty (< 1 wk data): friendly "first week collecting" state with the checklist beside it |  
| List | **\*\*At-risk members\*\*** (top 5 of the \$3.2 query, "Inactive clients" for clinic) | cached 1 h | row = name · last active · lifetime workouts · **\*\*[Send nudge]\*\*** one-tap (push via member app: "Your gym misses you 💪 — {org}"; rate-limit 1/member/7d; trainer sees own group only). Empty: "Nobody's slipping — nice." Error: retry chip |  
| Side | Activation checklist (until complete, \$5.1) · This month's report shortcut | `gyms.activation` | disappears forever once done |

Events: `console\_overview\_viewed`, `at\_risk\_nudged`, tile taps. **\*\*Edge:\*\*** org with 0 members ever → Overview *\*is\** the checklist + poster CTA (no sad empty charts).

### ### 4.2 Banner state machine (persistent slot, all screens)

| State | Condition | Copy & CTA |  
|---|---|---|  
| Trial info | trial, > 3 days left | "Trial — X days left · **\*\*Add payment\*\***" (dismissible/day) |  
| Trial urgent | ≤ 3 days | amber, not dismissible: "Trial ends {date}. Members keep Pro features only if a plan is active. **\*\*Choose plan\*\***" |  
| Trial expired → grace | 0-14 days past, unpaid | red: "Trial ended — members have moved to the free tier. Reactivate to restore. **\*\*Reactivate\*\***" — console goes **\*\*read-only\*\*** except Billing |  
| Past-due | payment failed | v1 \$10 dunning (5-day grace): "Payment failed — retrying. **\*\*Update payment method\*\***" |  
| Seat pressure | members ≥ 90 % of cap | "87/100 seats — **\*\*Upgrade tier\*\***" (the cost-ceiling-as-upsell mechanism, v1 \$8) |

| Healthy | else | no banner |

Entitlement truth (restating v1 §8 so support tickets answer themselves):  
on trial/subscription expiry, member entitlements degrade to `free`  
**\*\*immediately\*\***; data is never deleted; the console stays read-only 14  
days, then archived (restorable by reactivating). Member-side, each ex-gym  
member gets the win-back Pro prompt – the B2B–B2C funnel firing exactly  
where v1 designed it.

### ### 4.3 Members

**\*\*Purpose:\*\*** roster, seats, and the 30-second walk-in join.

Header: **\*\*Seat meter\*\*** (`members / cap`, color at 90 %) · search · filters  
(group/code · status active|at-risk|inactive|removed · joined range) ·  
**\*\*[Invite]\*\*** (sheet: show code big, QR, WhatsApp share, poster PDF per  
code) · **\*\*[Export CSV]\*\*** (owner/manager; columns = table columns; respects  
§2.4 boundary).

Table (`org\_member\_stats`, cursor-paginated 50/page, server sort): Display  
name · Group (code label) · Joined · Last active · Workouts 30d · Avg form  
30d (– under 5 sets) · Streak · ⋮ (Nudge / View / Remove). Sort default:  
last active desc. Trainer role: pre-filtered to their group(s), Remove  
hidden.

**\*\*Member detail drawer\*\*** (tap row): identity chip (display name, member  
since, group) · 12-week activity sparkline · exercise mix (top 5 by sets)  
· form-score trend line · fault mix (top 3 fault categories by count –  
*\*this is what makes a trainer nod\**: "his squats cave inward, the app sees  
it too") · last 10 sessions list (date, exercises, sets, avg score) ·  
actions: Nudge · Assign program (v1.5, disabled with "coming" tag) ·  
Remove. **\*\*Everything here derives from SetSummary aggregates – nothing  
outside the §2.4 boundary is even queryable by this screen's repo.\*\***

Remove flow: confirm sheet → sets `removed\_at` (seat freed instantly;  
history retained; member app notifies "You've left {org} – your workouts  
are yours forever" + Pro win-back). **\*\*Restore\*\*** available 30 days (clears  
`removed\_at` if seats allow). Edge cases: duplicate display names →  
disambiguate with join-month suffix in org views only · member of 2 orgs →  
appears in both rosters, each sees only membership-interval activity

(§2.1) · code used by a stranger → they're a member like any other; owner removes; **\*\*rotate code\*\*** suggested inline after any removal.

### ### 4.4 Leaderboard

**\*\*Purpose:\*\*** retention theater inside the facility; the poster/TV are the app's own marketing to non-users on the floor.

Boards (month picker, org TZ): **\*\*Form Score\*\*** (avg over month, min 8 scored sets – quality gate stops one lucky set topping the board) · **\*\*Workouts\*\*** · **\*\*Streak\*\***. Row: rank · display name (member-app setting: first name + last initial default; **\*\*opt-out toggle lives in the member app\*\***, opted-out members simply don't appear) · value · movement vs last week. **\*\*Board renders only at  $\geq 3$  eligible members\*\*** (below that: invite CTA instead – a leaderboard of one is an insult). Groups filter (studio: per-cohort boards). Clinic: whole screen hidden unless owner enables (§2.3).

**\*\*TV mode:\*\*** owner/manager creates a **\*\*display token\*\*** → URL `/tv/:token` – fullscreen, logo + rotating boards (20 s), auto-refresh 60 s (Redis read; ~free), joins-this-month ticker, QR + code footer ("Scan to join {org}"). Token is view-only, revocable, expires with subscription; safe on any lobby screen. States: pre-3-members → shows QR + "Be first on the board".

**\*\*Poster generator:\*\*** per-code A4 PDF (worker  $\leq 10$  s): org logo, "Free with your membership" line, QR + code, 3-step how-to, bilingual (org locale + English; AS/HI keys from Part 2 Appendix A conventions). Monthly variant: top-3 podium + QR – the notice-board refresh that keeps the app visible offline. Delivery: download + WhatsApp share; stored in R2 `reports/`.

Events: `tv\_mode\_started/heartbeat`, `poster\_downloaded{variant}`, `leaderboard\_viewed`.

### ### 4.5 Reports

**\*\*Purpose:\*\*** the monthly renewal argument, generated for the owner and *\*sent\** to them – never assume they'll come looking.

**\*\*Monthly org report (PDF, worker job on the 1st, 06:00 org TZ; emailed + WhatsApp-ready; archived in R2 `reports/`, listed here for 18 months):\*\***

Page 1 – logo, month, four KPIs with  $\Delta$  vs prior month, one-line headline auto-picked from the best true stat ("Your members completed **\*\*1,240\*\*** workouts – up 18%"). Page 2 – weekly trend chart · top-10 leaderboard · new members / removed. Page 3 – exercise mix, avg form score trend, **\*\*at-risk snapshot\*\*** ("6 members going quiet – open the console to nudge them") – the report always ends by sending the owner *\*back into the console to act\**. Clinic variant reframes page 3 as adherence summary (Phase 2, §6.2). Numbers follow Part 2B §6 (integers, honest deltas, no fake precision; calories, if ever shown, as ranges).

On-demand **\*\*[Generate now]\*\*** (rate-limit 3/day). States: first month partial → watermark "partial month"; generation failure → auto-retry ×3 then surfaced with support link. Events: `report\_generated/emailed/opened` (open tracking = renewal-risk signal for *\*you\**: an owner who never opens two reports in a row is churning – this feeds your admin panel, not theirs).

### ### 4.6 Billing

Plan card (tier, seats used/cap – **\*\*utilization\*\***, renewal date, price in org currency) · **\*\*Change tier\*\*** (up: immediate, prorated · down: blocked while members > target cap with guided remove/upgrade choice; mechanics in Part 5) · Payment method (Razorpay B2B rails per v1 §10 – B2B billing never touches app stores) · Invoice list (PDF download) · **\*\*Cancel\*\*** (owner only): one honest retention screen (pause 1 month option · Micro-tier downshift if eligible) → reason survey (4 options + text; this survey is your churn dataset) → confirm; post-cancel = the §4.2 expiry path. Trial state shows "Choose plan" instead of plan card. All money mutations are idempotent + audit-logged (v1 §10 webhooks doctrine).

### ### 4.7 Settings (+ Staff)

**\*\*Profile:\*\*** name, city, logo, timezone (changing TZ takes effect next rollup; historical rollups never rewritten – `calc\_version` philosophy applied to time itself), locale, vocabulary preview. **\*\*Codes:\*\*** list (label, uses, created, status) · create (label + optional expiry/max-uses) · rotate (old joins keep their group tag; old code dies) · pause. **\*\*Privacy:\*\*** the §2.4 visibility sheet verbatim (what the org can/can't see) + data-request routing (member deletion/export requests are

member-initiated in *\*their\** app; the console only ever sees the \$2.4 surface – stated here so owners can answer members' questions correctly). **\*\*Notifications:\*\*** report email recipients, at-risk weekly digest on/off, quiet hours. **\*\*Staff:\*\*** list, invite by email/phone with role, change role, remove; every staff mutation audit-logged; last-owner removal blocked.

---

## ## 5. Activation, console analytics & staff notifications

### ### 5.1 Activation checklist (pinned on Overview until complete)

☐ Logo added · ☐ Poster printed/shared · ☐ First **\*\*5\*\*** members joined · ☐ First **\*\*20\*\*** workouts synced · ☐ Monthly report viewed. Stored in `gyms.activation`; completing all five = **\*\*activated\*\***. Rationale: activation-by-day-7 will be your single best predictor of trial→paid conversion – instrument it (`org\_activated{days\_since\_signup}`) and steer onboarding copy/pilot playbooks by it.

### ### 5.2 Console analytics (server-side PostHog, per v1 \$16)

`console\_login` (per staff) · `overview\_viewed` · `at\_risk\_nudged` · `member\_removed` · `code\_rotated` · `poster\_downloaded` · `tv\_mode\_started` · `report\_opened` · `upgrade\_clicked/completed` · `cancel\_started/reason/completed`. Owner-engagement composite (logins + report-opens + nudges per month) is the leading renewal indicator – surfaced on *\*your\** admin panel next to per-org cost (v1 \$9.3) so every renewal call starts with the two numbers that matter.

### ### 5.3 Staff notifications (through the v1 \$15 module; same caps & quiet hours)

Weekly digest (Mon 09:00 org TZ: KPIs + at-risk count + deep link) · report-ready (monthly) · seat-pressure (once per crossing) · trial D-5 / D-1 · payment events. WhatsApp channel adapter (Phase 2 per v1 \$15) upgrades digest + report delivery for India orgs – architected now, shipped later.

---

## ## 6. Org-type deltas (what actually differs, and when it ships)

### ### 6.1 Studio / PT (v1.5 – after gym console is stable; mostly configuration, one real feature)

Groups become first-class in UI (they already exist as code labels): group filter pinned on Members/Leaderboard; per-group boards; trainer scoped to group(s) by default. **\*\*The one real feature: program assignment surfacing\*\*** – depends on Programs (v1 §21 #6): trainer assigns a program to a member/group; member app shows "Assigned by {Coach}"; Members table gains an **\*\*On-plan %\*\*** column (sessions matching assigned program ÷ assigned sessions, week window). Micro seat tier live. Pitch delta: gyms buy *\*differentiation\**, studios buy *\*client accountability\** – same console, different first screen emphasis (Members-first default tab for studios).

### ### 6.2 Physio clinic (Phase 2 – **\*\*gate: one committed pilot clinic or 5 inbound clinic requests; do not build speculatively\*\***)

Everything in 6.1, plus: vocabulary set (Clients/Sessions/Clinician) · leaderboards/challenges off by default · join = consent flow (§2.4, `consent\_at`) · **\*\*Client adherence view\*\***: assigned plan vs completed (calendar heat-strip), avg form score, fault mix, hold-time & ROM trend for mobility/stretch items (F10/F12 data – the honest, measurable signal a clinician actually wants) · **\*\*printable per-client adherence PDF\*\*** (for their paper file; adherence & activity only) · copy linter active (§2.2). Positioning boundary (Part 2B §7) is enforced product-wide: the console *\*reports what the client did\**; the clinician judges what it means. This sentence appears in clinic terms, onboarding, and the report footer.

### ### 6.3 Worldwide

Console currency is **\*\*display-localized\*\*** (₹/\$/€ from `currency\_display`), pricing tables per region resolved in Part 5; org TZ drives all time boundaries (§3.2) – an Auckland gym's "this week" is Auckland's week; member-side quotas remain UTC (v1) and the two never mix. All console strings are message keys from day one; RTL deferred until a market demands it.

---

## ## 7. Edge cases & policies (the support-ticket pre-emption list)

**\*\*Seat downgrade below headcount\*\*** → blocked with guided flow (§4.6).  
**\*\*Code leaked publicly\*\*** → rotate + optional expiry/max-uses on new codes; joins are members, not incidents. **\*\*Member in multiple orgs\*\*** → allowed; entitlement = best active (v1 §8); stats per membership interval. **\*\*Owner leaves own gym as member\*\*** → toggle off `owner\_included\_as\_member`; staff role unaffected. **\*\*Org deletes account\*\*** → subscription cancels, memberships end (`removed\_at = now`), members keep their own data + win-back prompt, org data archived 90 days then purged (DPDP), audit\_log retained. **\*\*Timezone change\*\*** → future rollups only (§4.7). **\*\*Display-name abuse on TV mode\*\*** → owner can hide a member from boards (member keeps playing privately); profanity list on display names at the member-app layer. **\*\*Minors\*\*** → member app policy question, not console's; boards show display names only, never photos, partly for this reason. **\*\*Two staff edit simultaneously\*\*** → last-write-wins + audit trail; no locking ceremony at this scale.

---

## ## 8. Build slices & acceptance criteria

**\*\*Slice A – lands in v1 Phase 3 (weeks 6–8), the "gym console v1":\*\***

Onboarding wizard · Overview (tiles, trend, at-risk, checklist) · Members (table, drawer, invite, remove, CSV) · Codes · Billing (trial→pay, tier change up, invoices) · Settings/Staff · rollup worker + `org\_daily\_stats` · monthly report job + email · banner machine.

✓ *\*Done when:\** a stranger self-serves org→trial→poster in  $\leq 5$  min (TTFMJ instrumented) · a member joins by code and their next workout moves Overview within 5 min (live counter) and next-day rollups · at-risk nudge delivers a push · trial expiry degrades entitlements exactly per §4.2 with zero manual steps · report PDF emails itself on the 1st · every §2.2 permission enforced by an API test, not just hidden UI.

**\*\*Slice B – v1.1 (rides Phase 4):\*\*** Leaderboards + TV mode + poster generator + weekly digest. ✓ *\*TV URL runs 24 h on a lobby screen without interaction; poster prints correctly A4; boards respect opt-out and the 3-member gate.\**

**\*\*Slice C – v1.5 (with Programs):\*\*** §6.1 studio set. **\*\*Slice D – Phase 2 (gated):\*\*** §6.2 clinic set.

**\*\*Sequencing note:\*\*** Slice A intentionally ships *\*before\** leaderboards – a gym owner renews on adoption numbers and saved members; the poster/TV layer then multiplies adoption. Don't invert it.

---

*\*– End of Part 3. Remaining queue (v1 §23, one at a time):* **\*\*Part 4 – Database DDL & Mongo→PG migration\*\*** (now carrying: §2.1 columns, Part 2B's `calc\_version`/dishware/corrections tables, Part 2's definition tables, and the two catalog decisions – Mountain Pose & Brisk Walking) · Part 5 – Billing & webhooks (+ Micro-tier pricing ratification) · Part 6 – Mobile · Part 7 – Retention playbook · Part 8 – Ops runbook.\*

## # Part 4 – Database: Full Schema (DDL), Indexes, Lifecycle & the Mongo→Postgres Migration

**\*\*Prerequisites:\*\*** Architecture v1 (§7 data sketch, §8 tenancy) · Part 2 (SetSummary §2.4, definition versioning §9) · Part 2B (`calc\_version`, dishware/corrections §3.4) · Part 3 (org columns §2.1, rollups §3.2, visibility boundary §2.4).

**\*\*What this is:\*\*** the system of record, table by table, with the indexes justified by the exact queries that read them, the canonical SQL for the four make-or-break operations (entitlement, seat, rollup, at-risk), retention & DPDP mechanics, and the one-time Mongo→Postgres migration mapped against your **\*\*actual collections\*\*** (`users, workout\_sessions, workout\_templates, meal\_logs, coach\_conversations, running\_sessions, running\_routes, running\_schedules, body\_measurements, exercises`).

### ## 0. Amendments to earlier parts (recorded once, here)

1. **\*\*Partition-ready, not partitioned\*\*** (supersedes v1 §7.1 "partitioned by month from day one"). Declarative partitioning forces the partition key into every PK/FK and pushes Drizzle into hand-written SQL for the two hottest tables – real, permanent complexity – while the thing it buys (cheap archival of old months) matters only at Stage 2-3 scale (v1 §20). Instead: `workouts`/`workout\_sets` are designed so partitioning bolts on

later without schema change (time column `NOT NULL`, BRIN-indexed, no cross-month FK assumptions), and §3.5 includes the exact pg\_partman recipe with its trigger: **\*\*table > 50 GB or autovacuum falling behind two weeks running.\*\*** Nothing else in the doc set changes.

2. **\*\*Free-tier history is an access gate, not a deletion\*\*** (supersedes v1 §7.1's "nightly job prunes free-tier history > 90 days"). Aggregate-only rows cost ~2 KB/workout; deleting them destroys an upsell ("upgrade and your full history is *still there*") to save pennies, and contradicts Part 3 §4.2's "data is never deleted" promise to orgs. The 90-day window is enforced in the workouts repo's read path from `entitlements.history\_days`; hard deletion happens only via account deletion (§5.2). One less destructive cron to fear.

3. **\*\*`workouts` carries no `gym\_id`\*\*** (supersedes v1 §7.1's `gym\_id` null column). Part 3 §2.1's membership-interval rule makes stored attribution both redundant and wrong (multi-org members, join/leave/rejoin). Org attribution is always computed through `gym\_members` validity intervals and materialized once, in `org\_daily\_stats`.

4. **\*\*Money-unit law.\*\*** User-facing money = `\*\_minor` integer + `currency` (paise/cents). Internal cost telemetry = `cost\_micro` bigint + `currency` (default `USD`, since LLM/vision bill in USD; the v1 §9.3 circuit breaker converts with one config FX rate). The words *minor* and *micro* in a column name are load-bearing; nothing stores floats for money.

## **## 1. Conventions (every table obeys these; deviations are called out inline)**

- **\*\*IDs:\*\*** `uuid` PRIMARY KEY DEFAULT `gen_random_uuid()` everywhere (client-generated for `workouts` — that *is* the sync idempotency). Human handles (`slug`, `code`) are separate unique columns.

- **\*\*Time:\*\*** `timestampz` only, UTC stored; org-local day boundaries computed in the rollup worker per `gyms.timezone` (Part 3 §3.2); `created\_at` timestampz NOT NULL DEFAULT `now()` on every table (not repeated below).

- **\*\*Enums:\*\*** `text` + `CHECK (col IN (...))`, never PG enums — adding a state is a one-line migration, not an `ALTER TYPE` ceremony.

- **\*\*JSONB policy:\*\*** JSONB is for *documents the app treats as a value* (a definition, a report payload, entitlements) — never for anything filtered/joined/aggregated in SQL. If a query needs `WHERE payload->>'x'`, that field earns a column.

- **\*\*Deletion policy:\*\*** soft state columns (`removed\_at`, `status`) for anything with history semantics; hard `DELETE` only in the DPDP cascade (§5.2) and TTL sweeps (§5.1). `ON DELETE` defaults to `RESTRICT`; `CASCADE` appears only where a child is meaningless without its parent (sets→workout, messages→thread) — each is flagged.
- **\*\*Multi-tenancy:\*\*** no Postgres RLS — one app role, tenancy enforced in module repos per v1 §6.2 (RLS would double every query's planning cost to defend against a caller that doesn't exist; revisit if the API is ever multi-role).
- **\*\*Extensions:\*\*** `pgcrypto` (gen\_random\_uuid), `citext` (emails), `vector` (§3.7).
- **\*\*Drizzle discipline:\*\*** this document is the source of truth; Drizzle schema files mirror it 1:1; migrations are generated, human-reviewed as SQL, **\*\*expand-then-contract, forward-only\*\*** (no down-migrations in prod), one migration per PR, CI runs them against a Neon branch.

## ## 2. Domain map

```

...

identity — users — auth_identities — refresh_tokens
tenancy — gyms — gym_codes — gym_members — gym_staff
money — plans — subscriptions — invoices — webhook_events
catalog — exercises — exercise_definitions — definition_bundles
training — workouts — workout_sets · workout_templates
trust — meal_logs — user_dishware — meal_log_corrections ·
body_measurements
coach — coach_threads — coach_messages · kb_chunks(pgvector)
game — streaks · achievements — user_achievements · challenges —
participants · leaderboard_snapshots
geo — runs · saved_routes · run_schedules · geo_cache
metering — usage_daily · api_cost_events
org-analytics — org_daily_stats · org_member_stats(view) ·
gym_usage_reports
ops — push_tokens · feature_flags · audit_log · trace_samples
...

```

---

## ## 3. DDL by domain

### ### 3.1 Identity & auth

```
```sql
CREATE TABLE users (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  email citext UNIQUE,                                -- null allowed:
OAuth-only accounts
  password_hash text, hash_algo text CHECK (hash_algo IN
('bcrypt','argon2id')),
  display_name text NOT NULL,                        -- leaderboard identity;
profanity-filtered at write
  leaderboard_opt_out boolean NOT NULL DEFAULT false, -- Part 3 §4.4
  locale text NOT NULL DEFAULT 'en', units text NOT NULL DEFAULT 'metric',
  timezone text,                                     -- recaps only; quotas
stay UTC (v1)
  weight_kg numeric(5,2),                            -- calorie lever (2B
§2.3); history in body_measurements
  status text NOT NULL DEFAULT 'active' CHECK (status IN
('active','deleted')),
  deleted_at timestamptz,
  legacy_mongo_id text UNIQUE,                       -- migration traceability
($7); drop after 6 months
  last_active_at timestamptz
);
CREATE TABLE auth_identities (                      -- Google today; providers
later
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  provider text NOT NULL, subject text NOT NULL,
  UNIQUE (provider, subject)
);
CREATE TABLE refresh_tokens (                      -- rotation + reuse
detection (v1 §6.1)
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  family_id uuid NOT NULL,                          -- a login session; reuse
of a revoked member kills the family
  token_hash text NOT NULL UNIQUE,
  expires_at timestamptz NOT NULL, revoked_at timestamptz, replaced_by
uuid,
```

```

    ip inet, user_agent text
);
CREATE INDEX ON refresh_tokens (user_id, family_id);
```


*Reads that shape this: login by email; OAuth by (provider,subject); refresh by token_hash; "log out everywhere" by user_id. bcrypt→argon2id upgrade happens on successful login (verify with `hash_algo`, rehash, flip — §7).


```

### ### 3.2 Organizations & membership

```

```sql
CREATE TABLE gyms (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    slug text UNIQUE NOT NULL, name text NOT NULL, city text,
    org_type text NOT NULL DEFAULT 'gym' CHECK (org_type IN
('gym','studio','clinic')),
    timezone text NOT NULL DEFAULT 'Asia/Kolkata', locale text NOT NULL
DEFAULT 'en',
    currency_display text NOT NULL DEFAULT 'INR',
    logo_key text,
    owner_user_id uuid NOT NULL REFERENCES users(id),
    owner_included_as_member boolean NOT NULL DEFAULT true,
    activation jsonb NOT NULL DEFAULT '{}',
    state
    status text NOT NULL DEFAULT 'active' CHECK (status IN
('active','archived')),
    archived_at timestamptz
);
CREATE TABLE gym_codes (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    gym_id uuid NOT NULL REFERENCES gyms(id) ON DELETE CASCADE,
    code text UNIQUE NOT NULL,
    label text NOT NULL DEFAULT 'Front Desk',
    max_uses int, uses int NOT NULL DEFAULT 0,
    expires_at timestamptz, paused boolean NOT NULL DEFAULT false
);

```

-- the org table; name kept for continuity (v1 §8)

-- R2 gym-assets/

-- Part 3 §5.1 checklist

-- the group mechanism (Part 3 §2.1)

-- 6-char, ambiguity-free alphabet

```

CREATE TABLE gym_members (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  gym_id uuid NOT NULL REFERENCES gyms(id),
  user_id uuid NOT NULL REFERENCES users(id),
  code_id uuid REFERENCES gym_codes(id),          -- group attribution at
join
  joined_at timestamptz NOT NULL DEFAULT now(),
  removed_at timestamptz,                          -- membership =
[joined_at, removed_at)
  consent_at timestamptz,                          -- clinic join consent
(Part 3 §2.4); NOT NULL enforced in service for clinics
  hidden_from_boards boolean NOT NULL DEFAULT false,
  complimentary boolean NOT NULL DEFAULT false    -- owner seat (Part 3
§4.0); excluded from seat counts
);
CREATE UNIQUE INDEX gym_members_live_uq ON gym_members (gym_id, user_id)
WHERE removed_at IS NULL;
CREATE INDEX ON gym_members (gym_id, removed_at);    -- roster & seat
count
CREATE INDEX ON gym_members (user_id, removed_at);    -- entitlement
resolver
CREATE INDEX ON gym_members (gym_id, joined_at);      -- interval joins
(rollups)
CREATE TABLE gym_staff (
  gym_id uuid NOT NULL REFERENCES gyms(id) ON DELETE CASCADE,
  user_id uuid NOT NULL REFERENCES users(id),
  role text NOT NULL CHECK (role IN ('owner', 'manager', 'trainer')),
  PRIMARY KEY (gym_id, user_id)
);

```

The ****partial unique index**** is the leave/rejoin design: historical membership rows stack (each a closed interval), exactly what Part 3's attribution rule needs, while only one **live** membership per (gym,user) can exist. Rejoin = new row, not resurrection.

3.3 Plans, subscriptions, billing

```
```sql
```

```

CREATE TABLE plans (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),

```

```

 code text UNIQUE NOT NULL, --
 'free','pro_in_m','pro_in_y','pro_us_m','org_micro','org_starter','org_standard',
 'org_growth','org_scale'
 audience text NOT NULL CHECK (audience IN ('consumer','org')),
 org_types text[], -- null = all org types
 (org plans only)
 name_key text NOT NULL,
 price_minor int NOT NULL, currency text NOT NULL, "interval" text NOT
 NULL CHECK ("interval" IN ('month','year')),
 seat_cap int, -- org plans;
 complimentary members excluded from the count
 trial_days smallint NOT NULL DEFAULT 0, -- 7 for org self-serve
 (Part 3 $4.0); pilot codes extend subscriptions.trial_ends_at
 rank smallint NOT NULL, -- resolver precedence:
 free=0, pro=10, org member_entitlements=10
 entitlements jsonb NOT NULL, -- what the SUBSCRIBER
 gets
 member_entitlements jsonb, -- org plans: what each
 MEMBER gets (Pro-level per v1 $9.2)
 active boolean NOT NULL DEFAULT true
);
CREATE TABLE subscriptions (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 owner_type text NOT NULL CHECK (owner_type IN ('user','gym')),
 owner_id uuid NOT NULL,
 plan_id uuid NOT NULL REFERENCES plans(id),
 status text NOT NULL CHECK (status IN
 ('trialing','active','past_due','canceled','expired')),
 trial_ends_at timestamptz, current_period_end timestamptz,
 cancel_at_period_end boolean NOT NULL DEFAULT false,
 provider text NOT NULL CHECK (provider IN
 ('razorpay','stripe','revenuecat','pilot')),
 provider_ref text, cancel_reason text
);
CREATE UNIQUE INDEX subs_one_live_uq ON subscriptions (owner_type,
owner_id)
 WHERE status IN ('trialing','active','past_due'); -- exactly one
live sub per owner — double-charging is now unrepresentable
CREATE INDEX ON subscriptions (status, current_period_end); -- dunning &
expiry sweeps

```

```

CREATE TABLE invoices (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 subscription_id uuid NOT NULL REFERENCES subscriptions(id),
 amount_minor int NOT NULL, currency text NOT NULL,
 status text NOT NULL CHECK (status IN
('paid','failed','refunded','void')),
 provider_ref text UNIQUE, issued_at timestamptz NOT NULL, pdf_key text
);
CREATE TABLE webhook_events (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 provider text NOT NULL, event_id text NOT NULL,
 payload jsonb NOT NULL, received_at timestamptz NOT NULL DEFAULT now(),
 processed_at timestamptz, status text NOT NULL DEFAULT 'pending' CHECK
(status IN ('pending','done','failed')),
 UNIQUE (provider, event_id) -- the dedupe that makes
v1 $10's enqueue-then-ack safe
);
```

```

****Canonical entitlements JSON**** (the only shape the quota middleware and repos read; every key has a default so old plan rows survive new features):

```

```jsonc
{ "exercises": { "mode": "tier", "tier": "T1" }, // or {"mode":"all"} -
free set = the T1 tag (Part 2 App B #6), swappable as data
 "coach": { "window": "day", "limit": 30 },
 "meal_scan": { "window": "day", "limit": 8 },
 "route_gen": { "window": "day", "limit": 5 },
 "history_days": -1, // -1 = unlimited; free
= 90 (read-gate, $0.2)
 "programs": "all", // "starter" | "all"
 "global_leaderboards": true, "share_watermark": false }
```

```

****Seed rows (v1 \$9 numbers; Micro from Part 3 – prices ratified in Part 5):**** `free` (rank 0: T1 exercises, coach 5/****month****, meal 3/month, routes 2/month, history 90, starter program, watermark on) · `pro\_in\_m` ₹149 / `pro\_in\_y` ₹999 / `pro\_us\_m` \$3.99 / `pro\_us\_y` \$29.99 (rank 10: all-58, 30/8/5 per day, history -1) · org: `org\_micro` ≤25 ₹999 (org_types

`{studio,gym}`) & clinic Micro ₹1,499 · `org\_starter` ≤100 ₹1,499 ·
`org\_standard` ≤150 ₹1,999 · `org\_growth` ≤400 ₹3,499 · `org\_scale` 400+
₹4,999 – each org plan's `member\_entitlements` = the Pro block,
`entitlements` = console features.

3.4 Exercise catalog & definitions

```
```sql
CREATE TABLE exercises (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 slug text UNIQUE NOT NULL, name_key text NOT NULL,
 family text NOT NULL CHECK (family ~ '^F([1-9]|1[0-2])$'), -- F1..F12
(Part 2 §5)
 tier text NOT NULL CHECK (tier IN ('T1','T2','T3')),
 tracking text NOT NULL DEFAULT 'pose' CHECK (tracking IN
('pose','timer')),
 status text NOT NULL DEFAULT 'live' CHECK (status IN
('live','hidden','retired')),
 met numeric(3,1) NOT NULL, -- Part 2B Appendix A
lands here, one value per row
 difficulty smallint, equipment text[], muscles text[],
 legacy_mongo_id text UNIQUE
);

CREATE TABLE exercise_definitions (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 exercise_id uuid NOT NULL REFERENCES exercises(id),
 version int NOT NULL,
 status text NOT NULL CHECK (status IN
('draft','beta','live','retired')),
 definition jsonb NOT NULL, -- the Part 2 §4 document,
immutable once beta+
 min_engine_version text NOT NULL,
 published_at timestamptz, published_by uuid REFERENCES users(id),
 UNIQUE (exercise_id, version)
);

CREATE INDEX ON exercise_definitions (exercise_id, status);
CREATE TABLE definition_bundles (-- what clients actually
download (Part 2 §9.3)
 bundle_version int GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 channel text NOT NULL CHECK (channel IN ('live','beta')),
```

```

 sha256 text NOT NULL, -- the ETag
 manifest jsonb NOT NULL -- {exercise_slug:
definition_version}
);
```

Publishing = insert a new immutable definition version → build a new
bundle row from current pointers. **Rollback = build a bundle from the
previous versions** — no mutation anywhere. The two catalog decisions,
executed in seed (per Part 2 Appendix B): **Mountain Pose → `status
'live'`, tier T3, F12 hold, MET 2.3** (zero cost, serves the
older/rest-day audience the F12 tone targets; the `REMOVED_EXERCISES`
frontend hack dies with the migration) · **Brisk Walking → `tracking
'timer'`** (web logs a timed session; mobile deep-links it into the
Running module; excluded from every form-score surface).
`arnold_shoulder_press` stays unseeded — first expansion candidate, one
authoring session away.

### 3.5 Workouts & sets (the big tables — partition-ready per $0.1)

```sql
CREATE TABLE workouts (
 id uuid PRIMARY KEY, -- CLIENT-generated (v1
$5.3): the PK is the idempotency key
 user_id uuid NOT NULL REFERENCES users(id),
 started_at timestamptz NOT NULL, ended_at timestamptz,
 platform text NOT NULL CHECK (platform IN ('web','android','ios')),
 engine_version text NOT NULL, bundle_version int,
 sets_count smallint NOT NULL DEFAULT 0, total_reps int NOT NULL DEFAULT
0,
 avg_form_score smallint, -- null until a scored set
exists
 duration_ms int, kcal_point int, kcal_calc_version smallint, --
computed server-side at sync (2B $2.2)
 quality_flags text[] NOT NULL DEFAULT '{}')
);
CREATE INDEX ON workouts (user_id, started_at DESC); -- history screens,
member drawer
CREATE INDEX workouts_started_brin ON workouts USING brin (started_at);
-- rollups, future archival
CREATE TABLE workout_sets (

```

```

 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 workout_id uuid NOT NULL REFERENCES workouts(id) ON DELETE CASCADE,
 user_id uuid NOT NULL, -- denormalized on
purpose: per-user set queries skip the join
 exercise_id uuid NOT NULL REFERENCES exercises(id),
 started_at timestamptz NOT NULL, -- copied from parent:
partition-ready + BRIN-scannable alone
 set_index smallint NOT NULL, view text, mode text,
 reps smallint NOT NULL DEFAULT 0, hold_ms int,
 duration_ms int NOT NULL,
 avg_form_score smallint, rep_scores smallint[], -- Part 2 §2.4
verbatim; no rep_events table (v1 §7.1)
 fault_counts jsonb NOT NULL DEFAULT '{}',
 tempo_ms_avg int, rom_stats jsonb, calibration jsonb,
 engine_version text NOT NULL, definition_version int NOT NULL
);
CREATE INDEX ON workout_sets (workout_id);
CREATE INDEX ON workout_sets (user_id, started_at DESC); --
exercise-mix, form trend per member
CREATE INDEX ON workout_sets (exercise_id, started_at DESC); -- Part
2 §9.4 per-definition telemetry
CREATE INDEX sets_started_brin ON workout_sets USING brin (started_at);
CREATE TABLE workout_templates (-- existing WorkoutBuilder
feature, kept (audit find)
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 name text NOT NULL, items jsonb NOT NULL, -- [{exercise_slug,
target_sets, target_reps|hold_s}]
 updated_at timestamptz NOT NULL DEFAULT now(),
 legacy_mongo_id text UNIQUE
);
...

Sync contract: `POST /v1/workouts/sync` inserts the workout `ON
CONFLICT (id) DO NOTHING`, then sets keyed `(workout_id, set_index)`
upserted – a retried sync is a no-op by construction; no Idempotency-Key
bookkeeping table needed for this path. **The pg_partman recipe (applied
at the §0.1 trigger, ~1 evening):** install pg_partman → `CREATE TABLE
workout_sets_p (LIKE workout_sets INCLUDING ALL) PARTITION BY RANGE
(started_at)` → partman monthly config, premake 3 → backfill via
`partman.partition_data_proc` in batches → swap names in one transaction →

```

detach-and-archive months older than the retention line to R2 via `pg\_dump --table`. FKs \*into\* sets don't exist (nothing references a set), which is precisely why this stays an evening and not a project.

### ### 3.6 Nutrition & body (Trust Layer tables, Part 2B §3.4)

```
```sql
CREATE TABLE meal_logs (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id),
  taken_at timestamptz NOT NULL, meal_name text,
  items jsonb NOT NULL,          -- [{name, canonical, grams_point,
grams_range:[lo,hi], portion_source, nutrition_source, kcal, protein_g,
carbs_g, fat_g}]
  kcal_point int NOT NULL, kcal_low int NOT NULL, kcal_high int NOT NULL,
-- range survives confirmation, tightened (2B §3.2 S4)
  protein_g numeric(6,1), carbs_g numeric(6,1), fat_g numeric(6,1),
  confirmed boolean NOT NULL DEFAULT false,
  portion_source text NOT NULL,          -- worst rung used:
user_dishware|anchor|regional_prior|default|legacy
  nutrition_sources text[] NOT NULL,          -- e.g. {ifct,usda}
  calc_version smallint NOT NULL,
  legacy_mongo_id text UNIQUE
);
CREATE INDEX ON meal_logs (user_id, taken_at DESC);
CREATE TABLE user_dishware (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  label text NOT NULL, container_class text NOT NULL, volume_ml int NOT
NULL,
  food_hint text
);
CREATE TABLE meal_log_corrections (          -- P4 doctrine: corrections
are ground truth, originals preserved
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  meal_log_id uuid NOT NULL REFERENCES meal_logs(id) ON DELETE CASCADE,
  field text NOT NULL, original jsonb NOT NULL, corrected jsonb NOT NULL,
  portion_source text
);
CREATE TABLE body_measurements (          -- existing feature (audit
find), kept; org-invisible by §2.4 boundary
```

```

    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    measured_at timestamptz NOT NULL, weight_kg numeric(5,2),
    metrics jsonb NOT NULL DEFAULT '{}',           -- waist_cm etc. -
display-only, never computed upon except weight
    source text NOT NULL DEFAULT 'manual',
    legacy_mongo_id text UNIQUE
);
CREATE INDEX ON body_measurements (user_id, measured_at DESC);
```

Calorie computation uses the latest `weight_kg` measurement $\leq$ workout
time, else `users.weight_kg`, else 70 (2B §2.1) – resolved at sync,
stamped into `workouts.kcal_*`, never recomputed silently (P3 doctrine).

3.7 Coach (Chroma → pgvector, v1 §6.1)

```sql
CREATE TABLE coach_threads (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id),
    title text, last_message_at timestamptz, legacy_mongo_id text UNIQUE
);
CREATE TABLE coach_messages (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    thread_id uuid NOT NULL REFERENCES coach_threads(id) ON DELETE CASCADE,
    role text NOT NULL CHECK (role IN ('user', 'assistant', 'system')),
    content text NOT NULL,
    model text, tokens_in int, tokens_out int, cost_micro bigint, currency
text DEFAULT 'USD'
);
CREATE INDEX ON coach_messages (thread_id, created_at);
CREATE TABLE kb_chunks (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    doc text NOT NULL, chunk_index int NOT NULL,
    content text NOT NULL, meta jsonb NOT NULL DEFAULT '{}',
    embedding vector(384),           -- dimension PINNED to the
embedder chosen at port; 384 = MiniLM-class assumption
    UNIQUE (doc, chunk_index)
);
```

```

Five knowledge docs ≈ a few hundred chunks: **\*\*no ANN index needed\*\*** – a sequential scan with cosine distance is single-digit ms at this size. Add HNSW only if the KB grows 100×; noting this prevents cargo-cult index tuning.

### ### 3.8 Gamification

```
```sql
CREATE TABLE streaks (
  user_id uuid PRIMARY KEY REFERENCES users(id) ON DELETE CASCADE,
  current int NOT NULL DEFAULT 0, longest int NOT NULL DEFAULT 0,
  last_activity_date date, freezes_available smallint NOT NULL DEFAULT 0,
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE achievements (                                -- seeded from badges.py
port
  code text PRIMARY KEY, name_key text NOT NULL, criteria jsonb NOT NULL,
  icon text
);

CREATE TABLE user_achievements (
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  code text NOT NULL REFERENCES achievements(code),
  earned_at timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (user_id, code)
);

CREATE TABLE challenges (                                -- v1.1 (Part 3 §2.3);
tables now, screens later
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  scope text NOT NULL CHECK (scope IN ('user', 'org', 'global')),
  gym_id uuid REFERENCES gyms(id), template_code text NOT NULL,
  starts_on date NOT NULL, ends_on date NOT NULL, config jsonb NOT NULL
  DEFAULT '{}',
  status text NOT NULL DEFAULT 'active' CHECK (status IN
('active', 'ended', 'canceled'))
);

CREATE TABLE challenge_participants (
  challenge_id uuid NOT NULL REFERENCES challenges(id) ON DELETE CASCADE,
  user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  progress jsonb NOT NULL DEFAULT '{}', joined_at timestamptz NOT NULL
  DEFAULT now(),
```

```

    PRIMARY KEY (challenge_id, user_id)
);
CREATE TABLE leaderboard_snapshots (          -- weekly persistence of
Redis ZSETs (v1 $7.2); Redis stays rebuildable
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    gym_id uuid REFERENCES gyms(id),            -- null = global board
    board text NOT NULL, period text NOT NULL,  -- '2026-07'
    entries jsonb NOT NULL,                    -- [{user_id,
display_name, value, rank}] - display_name scrubbed on account deletion
($5.2)
    UNIQUE (gym_id, board, period)
);
```

3.9 Geo & runs (mobile-first; web = browse/plan per v1 $12)

```sql
CREATE TABLE runs (
    id uuid PRIMARY KEY,                        -- client-generated, same
idempotency pattern as workouts
    user_id uuid NOT NULL REFERENCES users(id),
    started_at timestamptz NOT NULL, duration_s int NOT NULL, distance_m int
NOT NULL,
    polyline text, route_name text, splits jsonb,
    kcal_point int, kcal_calc_version smallint,
    source text NOT NULL DEFAULT 'mobile', legacy_mongo_id text UNIQUE
);
CREATE INDEX ON runs (user_id, started_at DESC);
CREATE TABLE saved_routes (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    name text NOT NULL, polyline text NOT NULL, distance_m int NOT NULL,
    legacy_mongo_id text UNIQUE
);
CREATE TABLE run_schedules (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    rule jsonb NOT NULL, active boolean NOT NULL DEFAULT true,
    legacy_mongo_id text UNIQUE
);

```

```
CREATE TABLE geo_cache (                                -- persistent layer under
the Redis geo cache (v1 $6.1)
    lat3 numeric(7,3) NOT NULL, lng3 numeric(7,3) NOT NULL,
    name text NOT NULL, provider text NOT NULL, cached_at timestampz NOT
NULL DEFAULT now(),
    PRIMARY KEY (lat3, lng3)
);
```
```

GPS polylines are the most sensitive data the app holds (home addresses fall out of them). They live only here, are **\*\*org-invisible\*\*** (Part 3 §2.4), excluded from all exports except the user's own DPDP export, and deleted in the §5.2 cascade.

### ### 3.10 Metering & cost telemetry

```
```sql
CREATE TABLE usage_daily (                                -- nightly Redis rollup
(v1 $7.2); feeds reports & Part 3 analytics
    user_id uuid NOT NULL, feature text NOT NULL, day date NOT NULL,
    count int NOT NULL,
    PRIMARY KEY (user_id, feature, day)
);
CREATE TABLE api_cost_events (                            -- the v1 $9.3 breaker's
ledger
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    at timestampz NOT NULL DEFAULT now(),
    user_id uuid, gym_id uuid,                            -- gym resolved via live
membership at spend time; null for direct consumers
    feature text NOT NULL, provider text NOT NULL,
    units numeric(12,4) NOT NULL, unit_type text NOT NULL,
    cost_micro bigint NOT NULL, currency text NOT NULL DEFAULT 'USD'
);
CREATE INDEX ON api_cost_events (gym_id, at);
CREATE INDEX cost_at_brin ON api_cost_events USING brin (at);
```
```

### ### 3.11 Org analytics & reports (Part 3 §3.2 made physical)

```
```sql
CREATE TABLE org_daily_stats (
```

```

    gym_id uuid NOT NULL REFERENCES gyms(id) ON DELETE CASCADE,
    day date NOT NULL,                                -- in the ORG's timezone
(worker computes the UTC window)
    active_members int NOT NULL, workouts int NOT NULL, sets int NOT NULL,
    total_reps int NOT NULL, minutes int NOT NULL,
    avg_form_score numeric(4,1), scored_sets int NOT NULL,
    new_members int NOT NULL, removed_members int NOT NULL,
    PRIMARY KEY (gym_id, day)
);
CREATE VIEW org_member_stats AS                      -- Members table & drawer
read ONLY this + workout aggregates
SELECT m.id AS membership_id, m.gym_id, m.user_id, u.display_name,
       c.label AS group_label, m.joined_at, m.hidden_from_boards,
       u.last_active_at,
       (SELECT count(*) FROM workouts w WHERE w.user_id = m.user_id
        AND w.started_at >= now() - interval '30 days'
        AND w.started_at >= m.joined_at) AS workouts_30d,
       (SELECT round(avg(s.avg_form_score),0) FROM workout_sets s
        WHERE s.user_id = m.user_id AND s.avg_form_score IS NOT NULL
        AND s.started_at >= now() - interval '30 days'
        AND s.started_at >= m.joined_at) AS avg_form_30d
FROM gym_members m JOIN users u ON u.id = m.user_id
LEFT JOIN gym_codes c ON c.id = m.code_id
WHERE m.removed_at IS NULL AND u.status = 'active';
CREATE TABLE gym_usage_reports (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    gym_id uuid NOT NULL REFERENCES gyms(id) ON DELETE CASCADE,
    period text NOT NULL, stats jsonb NOT NULL, pdf_key text,
    emailed_at timestampz, opened_at timestampz,
    UNIQUE (gym_id, period)
);
...

```

3.12 Platform / ops

```

```sql
CREATE TABLE push_tokens (
 user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
 token text PRIMARY KEY, platform text NOT NULL, last_seen_at timestampz
);

```

```

CREATE TABLE feature_flags (key text PRIMARY KEY, rules jsonb NOT NULL,
updated_at timestamptz NOT NULL DEFAULT now());
CREATE TABLE audit_log (
 id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
 at timestamptz NOT NULL DEFAULT now(),
 actor_user_id uuid, gym_id uuid, action text NOT NULL,
 target_type text, target_id text, meta jsonb NOT NULL DEFAULT '{}'
);
CREATE INDEX audit_at_brin ON audit_log USING brin (at);
CREATE INDEX ON audit_log (gym_id, at DESC);
CREATE TABLE trace_samples (-- v1 $14 anti-cheat
audits; R2 90-day lifecycle
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 user_id uuid NOT NULL, exercise_id uuid NOT NULL,
 r2_key text NOT NULL, engine_version text NOT NULL,
 verdict text NOT NULL DEFAULT 'pending' CHECK (verdict IN
('pending','match','mismatch','error'))
);
```

```

4. The four canonical operations (exact SQL; these are the correctness core)

****4.1 Entitlement resolver**** (v1 \$8 – fetch candidates in one query, merge in code, cache Redis 60 s, bust on billing/membership events):

```

```sql
SELECT p.rank, p.entitlements, NULL::jsonb AS member_entitlements
FROM subscriptions s JOIN plans p ON p.id = s.plan_id
WHERE s.owner_type='user' AND s.owner_id=$1 AND s.status IN
('trialing','active','past_due')
UNION ALL
SELECT p.rank, NULL, p.member_entitlements
FROM gym_members m
JOIN subscriptions s ON s.owner_type='gym' AND s.owner_id=m.gym_id
AND s.status IN ('trialing','active','past_due')
JOIN plans p ON p.id = s.plan_id
WHERE m.user_id=$1 AND m.removed_at IS NULL;
```

```

Merge rule in the `entitlements` service: start from the `free` plan document, overlay the highest-`rank` candidate, key-by-key max for numeric limits, OR for booleans, `all > tier` for exercises. `past\_due` still grants (v1 \$10's 5-day grace); `expired/canceled` rows simply don't match.

****4.2 Seat-safe join**** (v1 \$8; lock the **org row**, not a count):

```
```sql
BEGIN;
SELECT 1 FROM gyms WHERE id=$gym FOR UPDATE; --
serialize joins per org
-- validate code: exists, not paused/expired, uses < max_uses (increment)
-- seat check: (count live, non-complimentary members) < plan.seat_cap →
else raise SEAT_CAP
INSERT INTO gym_members (gym_id,user_id,code_id,consent_at,complimentary)
VALUES (...);
UPDATE gym_codes SET uses = uses + 1 WHERE id=$code;
COMMIT; -- on unique_violation of gym_members_live_uq → idempotent
success (already a member)
```
```

****4.3 Nightly rollup upsert**** (worker groups orgs by timezone; `\$from/\$to` = that org-day's UTC window; the membership-interval rule lives here and only here):

```
```sql
INSERT INTO org_daily_stats AS ods (gym_id, day, active_members, workouts,
sets, total_reps,
minutes, avg_form_score, scored_sets,
new_members, removed_members)
SELECT g.id, $day,
count(DISTINCT w.user_id), count(DISTINCT w.id),
coalesce(sum(w.sets_count),0),
coalesce(sum(w.total_reps),0),
coalesce(sum(w.duration_ms),0)/60000,
round(avg(w.avg_form_score) FILTER (WHERE w.avg_form_score IS NOT
NULL),1),
count(*) FILTER (WHERE w.avg_form_score IS NOT NULL),
(SELECT count(*) FROM gym_members jm WHERE jm.gym_id=g.id AND
jm.joined_at >= $from AND jm.joined_at < $to),
```

```

 (SELECT count(*) FROM gym_members rm WHERE rm.gym_id=g.id AND
rm.removed_at >= $from AND rm.removed_at < $to)
FROM gyms g
LEFT JOIN gym_members m ON m.gym_id = g.id
LEFT JOIN workouts w ON w.user_id = m.user_id
 AND w.started_at >= $from AND w.started_at < $to
 AND w.started_at >= m.joined_at
 AND (m.removed_at IS NULL OR w.started_at < m.removed_at)
WHERE g.id = ANY($org_ids)
GROUP BY g.id
ON CONFLICT (gym_id, day) DO UPDATE SET
 active_members=excluded.active_members, workouts=excluded.workouts,
sets=excluded.sets,
 total_reps=excluded.total_reps, minutes=excluded.minutes,
 avg_form_score=excluded.avg_form_score,
scored_sets=excluded.scored_sets,
 new_members=excluded.new_members,
removed_members=excluded.removed_members;

```

**\*\*4.4 At-risk\*\*** (Part 3 §3.2 definition, verbatim in SQL):

```

```sql
SELECT m.user_id, u.display_name, u.last_active_at,
        (SELECT count(*) FROM workouts w WHERE w.user_id=m.user_id AND
w.started_at >= m.joined_at) AS lifetime
FROM gym_members m JOIN users u ON u.id=m.user_id
WHERE m.gym_id=$1 AND m.removed_at IS NULL AND u.status='active'
        AND m.joined_at < now() - interval '14 days'
        AND NOT EXISTS (SELECT 1 FROM workouts w WHERE w.user_id=m.user_id
                        AND w.started_at >= now() - interval '14 days')
        AND EXISTS (SELECT 1 FROM workouts w WHERE w.user_id=m.user_id
                    AND (w.started_at BETWEEN m.joined_at AND m.joined_at +
interval '21 days'
                        OR w.started_at >= now() - interval '44 days'))
ORDER BY lifetime DESC LIMIT 20;

```

5. Data lifecycle, retention & DPDP

5.1 Retention table (jobs live in the BullMQ `rollups` queue; each is idempotent)

Data	Retention	Mechanism
workouts / sets / runs / meals / measurements	forever (free tier read-gated at 90 d, \$0.2)	—
refresh_tokens	expiry + 30 d	nightly DELETE
webhook_events	90 d after processed	nightly DELETE
trace_samples (+ R2 `traces/`)	90 d	R2 lifecycle + row sweep
usage_daily	24 months	monthly partition-free DELETE by day
api_cost_events	24 months	same
audit_log	24 months (money rows 7 y)	sweep with action-class filter
gym_usage_reports (+ PDFs)	18 months (Part 3 \$4.5)	sweep + R2 lifecycle
archived orgs	purge 90 d after archive (Part 3 \$7)	weekly job
meal photos	**never stored** (2B)	—

5.2 DPDP/GDPR account deletion (member-initiated in-app; 14-day soft window, then hard cascade)

Day 0: `users.status='deleted'`, `deleted\_at=now()`, all refresh tokens revoked, push tokens deleted, memberships closed (`removed\_at`), login blocked, 14-day undo email. Day 14 job, in one transaction per user:

****delete**** meal_logs(+corrections), dishware, coach threads/messages, runs/saved_routes/schedules, body_measurements, workout_templates, workouts(+sets CASCADE), user_achievements, streaks, challenge_participants, auth_identities; ****anonymize**** `users` row to a tombstone (email→null, display_name→'Deleted user', weight→null; row kept so FKs from audit/invoices resolve); ****scrub**** display_name from `leaderboard\_snapshots.entries`; ****keep**** org_daily_stats (already aggregate, no PII), audit_log (actor now tombstoned), and paid invoices for statutory tax retention (India: ~7-8 y — **confirm exact period with your CA**; invoices carry the minimum identity the law requires and nothing more). ****Export**** (the other DPDP right): a worker builds a JSON zip of every user-owned table above + profile, delivered via signed URL, 7-day expiry. Both flows exist at launch, not later (2B \$7).

6. Sizing (why Neon's small tiers hold for a long time)

Per ****active**** user·month at heavy use (12 workouts × 3 sets, 60 meals, 40 coach msgs): workouts ~3 KB + sets ~15 KB + meals ~60 KB + coach ~30 KB + misc ~10 KB ≈ ****~120 KB/user·month****. 1,000 MAU ≈ 120 MB/month ≈ 1.4 GB/year **before** TOAST compression (jsonb compresses well; realistic ~60%). With \$5.1 sweeps, 1k MAU cruises for years under a few GB; 10k MAU ≈ 12 GB/yr — still one Neon Launch-tier database. Indexes ≈ 30-40% on top. The DB will not be your scaling problem; the v1 \$20 triggers stand unchanged.

7. The Mongo → Postgres migration (one-time, idempotent, verifiable)

****Strategy:**** a single Node script in `apps/api/tools/migrate-mongo/`, reading Mongo ****read-only****, writing with `ON CONFLICT DO NOTHING`. Every new row's UUID = ****UUIDv5(NAMESPACE_AIHG, mongo ObjectId hex)**** — deterministic, so re-runs are no-ops and cross-collection references survive without a lookup table; the source id is also kept in `legacy\_mongo\_id`. Old stack keeps running until cutover (v1 Phase 2); "rollback" = flip the flag back, Mongo untouched.

****Mapping (your exact collections):****

```
| Mongo | → Postgres | Transform notes |
|---|---|---|
| `users` (Mongoose **User** + ml `users`, matched on `_id`/email) |
| `users` (+`streaks`, `user_achievements`) | keep bcrypt hash
| (`hash_algo='bcrypt'`, upgrade-on-login $3.1); merge duplicate email pairs
(auth doc wins credentials, ml doc wins profile); embedded gamification
fields → if present map, **else recompute deterministically from workout
history** (badges/streak logic is pure — recomputing is safer than
trusting drift) |
| `workout_sessions` | `workouts` + `workout_sets` | embedded sets array
unrolled; per-set fields → $3.5 columns; missing new fields → null;
`engine_version='legacy-py'`, `definition_version=0`; `kcal_` recomputed
```

```

with `calc_version=1` + the new MET table (2B) – legacy displayed totals
may shift; ship with the one-line release note "calorie estimates improved
for 49 exercises" (honesty per 2B, not a silent rewrite – this is the
*one* sanctioned history recomputation, done at migration, versioned) |
| `workout_templates` | `workout_templates` | exercise name → slug map
from the seed |
| `meal_logs` | `meal_logs` | `confirmed=true` (user saved them),
`portion_source='legacy'`, `nutrition_sources={legacy_model}`,
`calc_version=0`, range = point ±30% honest retro-band |
| `coach_conversations` | `coach_threads` + `coach_messages` | costs null
for legacy rows |
| `running_sessions` / `running_routes` / `running_schedules` | `runs` /
`saved_routes` / `run_schedules` | polyline passthrough |
| `body_measurements` | `body_measurements` | weight entries also refresh
`users.weight_kg` (latest) |
| `exercises` | **not migrated** – re-seeded canonically (§8);
`legacy_mongo_id` set on seed rows by name-match so old workout references
resolve |
| Chroma vectors, prediction caches, sessions/JWTs, Redis quota counters |
**not migrated** – re-ingest KB into `kb_chunks`; predictions are
deterministic recomputes (2B §5); everyone re-logs-in once |

**Runbook:** (1) staging dry-run against a Mongo dump; row-count +
checksum report per table. (2) Verification gates: counts match ±0; 100
sampled users' workout/sets/meals counts identical; 20 sampled bcrypt
logins verify; UUIDv5 spot-check on cross-references (workout→user). (3)
Prod: announce, freeze writes (maintenance flag), final run (idempotent),
gates again, flip `DATA_BACKEND=pg`, unfreeze. (4) Mongo → read-only for
30 days, then export-to-R2 and decommission. Total downtime target: **< 30
minutes**; the script must process your current volume in single-digit
minutes (assert in the dry-run).

```

8. Seeds & fixtures (checked into the repo, applied by migration `0001\_seed`)

```

`plans` (§3.3 rows) · `exercises` – all 58 from Part 2 §6 with
family/tier/tracking/**MET from 2B Appendix A**, Mountain Pose live, Brisk
Walking `timer` · `exercise_definitions` – squat/jump/chair v1

```

```
(`status='live'`, ported constants) + T1 drafts as they land ·  
`definition_bundles` - bundle 1 · `achievements` - badges.py port ·  
`feature_flags` - `{data_backend, engine_rollout, beta_definitions}` ·  
dev-only: one demo org + 20 synthetic members with 8 weeks of plausible  
workouts (the console is undevelopable without data - this fixture is how  
every Part 3 screen gets built and screenshotted).
```

9. Acceptance criteria - "Part 4 done" means

Schema applies from zero via migrations on a fresh Neon branch · every \$4
query EXPLAINS to index scans on the seeded demo data · sync retry
produces zero duplicates (test: fire the same payload 5x) · seat race test
(20 parallel joins, cap 10) admits exactly 10 · entitlement resolver
returns Pro for a gym member, free the second `removed\_at` is set
(cache-bust test) · rollup for a UTC+13 org bins a 23:30 local workout
into the correct local day · deletion cascade leaves zero PII rows
(automated scan for the tombstone's uuid across all tables) · migration
dry-run gates green on a real Mongo dump · `pg\_dump | restore` drill
documented and executed once (v1 \$17's "an untested backup is a rumor").

**- End of Part 4. Queue (one at a time): **Part 5 - Billing &
entitlements** (provider flows, webhook state machines, proration,
dunning, Micro-tier price ratification) · Part 6 - Mobile · Part 7 -
Retention playbook · Part 8 - Ops runbook.**

Part 5 - Billing & Entitlements: Full Specification

****Prerequisites:**** Architecture v1 (\$9 plans, \$10 payments doctrine) ·
Part 3 (\$4.2 banner machine, \$4.6 Billing screen, \$6.3 worldwide) · Part 4
(\$3.3 DDL, one-live-sub index, entitlements canon, \$4.1 resolver).

****Scope:**** final price books · provider adapters and their webhook state
machines · checkout, trial, pilot-code, plan-change, dunning,
cancel/refund flows · entitlement propagation · invoicing & tax hooks ·
abuse safeguards · the test matrix that makes money code boring.

0. Doctrine & schema addenda

****The one rule everything obeys (v1 \$10, made operational):**** a webhook is a **doorbell**, never a **messenger**. On receipt: verify signature → dedupe on `webhook\_events(provider,event\_id)` → ack 200 immediately → worker ****fetches the authoritative object from the provider API**** → reconciles local state through the \$3 machine. Out-of-order delivery, duplicates, and replay attacks all become non-events, because we never apply a webhook's **payload** – only its **pointer**. The only exception is RevenueCat, whose webhook body is itself the normalized truth feed (that is RC's product); its adapter still dedupes and still treats the RC REST API as tiebreaker.

****Part 4 addendum A (small DDL deltas this part requires; fold into migration `0002\_billing`):****

```
```sql
ALTER TABLE subscriptions
 ADD COLUMN pending_plan_id uuid REFERENCES plans(id), -- scheduled
downgrades / interval switches ($7)
 ADD COLUMN plan_change_at timestamptz,
 DROP CONSTRAINT subscriptions_provider_check,
 ADD CONSTRAINT subscriptions_provider_check CHECK (provider IN
('none','pilot','razorpay','stripe','revenuecat'));
 -- 'none' = card-less self-serve trial ($6.1); 'pilot' = hand-sold
extended trial ($6.2)
ALTER TABLE invoices ADD COLUMN tax_minor int NOT NULL DEFAULT 0, ADD
COLUMN invoice_no text UNIQUE;
CREATE TABLE billing_profiles (
 id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
 owner_type text NOT NULL CHECK (owner_type IN ('user','gym')), owner_id
uuid NOT NULL,
 legal_name text, gstin text, address jsonb, country text NOT NULL,
 billing_email citext,
 UNIQUE (owner_type, owner_id)
);
CREATE TABLE invoice_counters (fy text PRIMARY KEY, next int NOT NULL);
-- sequential numbering per Indian FY ($11)
CREATE TABLE pilot_codes (
 code text PRIMARY KEY, plan_id uuid NOT NULL REFERENCES plans(id),
 trial_days smallint NOT NULL, max_redemptions int NOT NULL DEFAULT 1,
 redeemed int NOT NULL DEFAULT 0,
 org_types text[], expires_at timestamptz, created_by uuid, note text
);
```

---  
## 1. Pricing ratification (the final price books; all values are `plans` seed rows – changing any is a data edit)

### ### 1.1 Consumer

Plan	India (Razorpay)	International (Stripe)	Mobile (IAP via RevenueCat)
Free	₹0 – permanent (v1 \$9.1 unchanged)	\$0	\$0
Pro monthly	<b>₹149/mo</b>	<b>\$3.99/mo</b>	nearest store tier to web price
Pro annual	<b>₹999/yr</b> (44% off – the anchor)	<b>\$29.99/yr</b>	nearest store tier

IAP notes: enroll in **Apple's Small Business Program and Google's 15% tier** before launch (both cut commission 30→15% at your scale – this is free margin, application takes days); store price tiers may drift a few percent from web – accept it, never advertise cross-channel price matching. One RevenueCat entitlement id: `pro`.

### ### 1.2 Organizations – ratified, including Micro

Plan code	Seats	India /mo	Intl /mo	Annual (both books)
`org_micro`	≤ 25	<b>₹999</b>	<b>\$19</b>	×10 (2 months free)
`org_micro_clinic`	≤ 25	<b>₹1,499</b>	<b>\$29</b>	×10   `{clinic}`
`org_starter`	≤ 100	₹1,499	\$39	×10   all
`org_standard`	≤ 150	₹1,999	\$59	×10   all
`org_growth`	≤ 400	₹3,499	\$99	×10   all
`org_scale`	400+	₹4,999 → custom	\$149 → custom	×10   all

**Micro rationale (closing Part 3's open item):** a ≤25-seat org's worst-month metered cost is ~₹150-250 under the v1 \$9.3 quotas → ₹999 nets ~₹750+, and it converts the 10-40-client PT studio that ₹1,499 would lose.

**\*\*Clinic Micro carries a ₹500 premium\*\*** because the segment carries real extra weight – consent flow, elevated privacy posture, the Phase-2 adherence reporting it's buying into – and because clinicians anchor on per-client value (₹60/client/mo is trivial against their session fees). Larger clinics are rare enough that they take the standard ladder until one asks for custom. **\*\*Annual org = 10× monthly\*\***, offered from Starter up and on Micro too (cash-flow + churn armor; the console upsells it at first renewal). Currency book selection: `gyms.currency\_display`/billing country picks INR vs USD book; consumer book picked by payment country (provider truth) with an IP/locale-based *\*display\** default and a manual switcher pre-checkout.

---

## ## 2. Provider topology (who owns what – no overlaps, no gaps)

Surface	Provider	Instrument	Why
Consumer web, India	Razorpay Subscriptions	UPI Autopay preferred, cards	you already run Razorpay (travels app); UPI Autopay is how India actually subscribes
Consumer web, intl	Stripe Billing (Checkout + Customer Portal)	cards	least-code path; portal offloads card-update/cancel UI
Consumer mobile	RevenueCat over Apple/Google IAP	store billing	store rules require IAP for consumer digital subs (v1 \$10)
Orgs, India	Razorpay Subscriptions (console web only)	UPI Autopay / e-mandate / cards	B2B stays outside app stores entirely – zero commission
Orgs, intl	Stripe Billing	cards	
Hand-sold pilots	internal (`provider='pilot'`)	none	\$6.2

Merchant of record: you. Cross-provider invariants: **\*\*entitlements are read only from our `subscriptions` table\*\*** (v1 \$10, restated because it is the whole design); Part 4's `subs\_one\_live\_uq` makes a second live subscription per owner *\*unrepresentable\** – the \$5 flows below are written so every path respects it (mobile paywall hides IAP when a web sub is live, and vice versa, \$5.4).

---

### ## 3. The canonical subscription state machine (provider-agnostic; every adapter compiles into these events)

**\*\*States\*\*** (Part 4 CHECK, unchanged): `trialing` → `active` ↯ `past\_due` → `expired`, plus `canceled` (terminal-at-period-end) and direct `trialing` → `expired` (card-less trial lapse).

**\*\*Normalized events → transitions → side-effects\*\*** (side-effect columns: E = entitlement cache bust, N = notification per Part 3 §5.3 / v1 §15, A = analytics per v1 §16, B = Part 3 §4.2 banner recomputes automatically from state):

Event (emitted by adapters or sweeps)	From → To	Side-effects
--- --- ---		
`TRIAL_STARTED`	∅ → trialing	E · A `org_trial_started` · schedule D-5/D-1 N
`PAYMENT_METHOD_ATTACHED`	trialing (provider none→rzp/stripe)	N "you're set — first charge {date}"
`ACTIVATED` (first successful charge)	trialing/expired → active	E · invoice row · N receipt · A `subscribe_activated`
`RENEWED` (periodic charge ok)	active/past_due → active	invoice · `current_period_end` advance · E if was past_due
`PAYMENT_FAILED`	active → past_due	E (grace still grants, Part 4 §4.1) · N day-0 of dunning (§8)
`GRACE_EXPIRED` (sweep, day 5)	past_due → expired	E (degrade) · N · A `churned{reason:payment}`
`TRIAL_LAPSED` (sweep)	trialing → expired	E · Part 3 §4.2 read-only path · A
`CANCEL_REQUESTED`	active → canceled (`cancel_at_period_end=true`)	N confirm + paid-through date · A + reason survey
`PERIOD_ENDED` (sweep or provider)	canceled → expired	E · win-back N (member-side prompts per v1 §8)
`UNCANCELED`	canceled → active	E · N
`PLAN_CHANGE_APPLIED`	any live (plan_id swap)	E · invoice (proration, §7) · A
`REFUND_FULL_FIRST`	active → expired	invoice refunded · E · A

Two sweeps own time-based edges (BullMQ repeatable, idempotent, fake-clock-testable): the **\*\*grace sweep\*\*** (`past\_due` older than 5 days → `GRACE\_EXPIRED`) and the **\*\*expiry sweep\*\*** (`trialing` past `trial\_ends\_at`

with `provider='none'|'pilot'` → `TRIAL\_LAPSED`; `canceled` past  
`current\_period\_end` → `PERIOD\_ENDED`; also fires `pending\_plan\_id`  
applications, \$7). Every transition writes `audit\_log`; illegal  
transitions (e.g., `RENEWED` on `expired`) log + alert instead of applying  
— that alert firing means an adapter bug, and it will be the first you  
hear of it rather than a customer.

---

## ## 4. Provider adapters (each ≤ ~200 lines of mapping code; the state machine does the thinking)

Adapter contract: `verifySignature(req) → dedupe → ack → enqueue{provider, event\_id, pointer}` then worker: `fetchAuthoritative(pointer) → normalize → apply(\$3 event)`. Event names below are the current documented sets —  
**\*\*the adapter is the only file that knows them\*\***, so provider renames are a one-file fix; verify against live docs at implementation.

### ### 4.1 Razorpay (consumer IN + orgs IN)

```
| Razorpay event | Normalized |
|---|---|
| `subscription.authenticated` | `PAYMENT_METHOD_ATTACHED` (mandate approved) |
| `subscription.activated` / first `subscription.charged` | `ACTIVATED` |
| `subscription.charged` (subsequent) | `RENEWED` |
| `payment.failed` (sub context) / `subscription.pending` | `PAYMENT_FAILED` |
| `subscription.halted` | ensure past_due (dunning continues; halted ≠ dead) |
| `subscription.cancelled` | `CANCEL_REQUESTED` (if user-initiated at provider) or reconcile |
| `subscription.completed` | `PERIOD_ENDED` (fixed-count subs — we don't use, reconcile only) |
| `refund.processed` | `REFUND_*` per $9 |
```

Mechanics: subscriptions created server-side against a Razorpay Plan mirroring our `plans` row (mapping table in config, asserted at boot — a mismatch fails deploy, not a customer); **\*\*UPI Autopay mandate\*\*** collected via hosted checkout; card-less trials attach later with `start\_at = trial\_ends\_at` so the mandate charges only when the trial converts (\$6.1).

Signature: `X-Razorpay-Signature` HMAC-SHA256 over raw body with the webhook secret – verified before parsing, constant-time compare.

### ### 4.2 Stripe (consumer intl + orgs intl)

`checkout.session.completed` → attach/ACTIVATE per mode · `invoice.paid` → `RENEWED`/`ACTIVATED` · `invoice.payment\_failed` → `PAYMENT\_FAILED` · `customer.subscription.updated` → reconcile (plan/interval/cancel flags) · `customer.subscription.deleted` → `PERIOD\_ENDED`. Use **\*\*Stripe Checkout\*\*** for purchase and **\*\*Customer Portal\*\*** for card-update/cancel/interval-switch (portal configured to allow only our plan set) – two redirects replace four screens of PCI-adjacent UI. Stripe Smart Retries configured to 3 attempts over 5 days so provider dunning and our \$8 timeline coincide. Signature: `Stripe-Signature` with tolerance window; raw-body route (no JSON middleware before verify – classic footgun, flagged here so it never happens).

### ### 4.3 RevenueCat (consumer mobile)

`INITIAL\_PURCHASE` → `ACTIVATED` · `RENEWAL` → `RENEWED` · `BILLING\_ISSUE` → `PAYMENT\_FAILED` (store grace ≈ our past\_due; expiry date from RC is authoritative) · `CANCELLATION` → `CANCEL\_REQUESTED` (auto-renew off; access continues) · `UNCANCELLATION` → `UNCANCELED` · `EXPIRATION` → `PERIOD\_ENDED`/`GRACE\_EXPIRED` · `PRODUCT\_CHANGE` → `PLAN\_CHANGE\_APPLIED` · `TRANSFER` → move `provider\_ref` between our users (RC app-user-id = our user uuid – set at login, never anonymous, which is what makes restore-purchases and transfers deterministic). Store receipts, prices, and proration are the stores' problem; we mirror state and never compute money for IAP.

## ## 5. Checkout & payment-method flows

**\*\*5.1 Consumer web IN:\*\*** paywall (plan cards, annual pre-selected) → `POST /v1/billing/checkout {plan\_code}` (server prices everything; client-sent amounts don't exist) → Razorpay hosted checkout → handler verifies payment signature → optimistic "Pro active" UI while the webhook path confirms (state machine remains the truth; UI reconciles within seconds).

**\*\*5.2 Consumer web intl:\*\*** same endpoint → Stripe Checkout redirect → success URL → portal link stored on Billing page.

**\*\*5.3 Org (console Billing, Part 3 §4.6):\*\*** tier picker (org\_type-filtered per `plans.org\_types`, annual toggle) → Razorpay/Stripe by currency book → mandate/checkout → `ACTIVATED` flips the Part 3 banner machine out of trial.

**\*\*5.4 Mobile:\*\*** RC Paywall → purchase → RC webhook `ACTIVATED`. Cross-channel guards: paywall first calls `GET /v1/me/entitlements`; if `source=web`, show "You're already Pro (manage on the web)" – no purchase button; web Billing for `source=revenuecat` shows "Managed through your app store" with deep links. Restore Purchases wired on login and on paywall. This pair of guards is the entire defense against double-paying, and it's tested in §14.

## ## 6. Trials & pilot codes

**\*\*6.1 Org self-serve (card-less 7-day, Part 3 §4.0):\*\*** signup → local `trialing`, `provider='none'`, `trial\_ends\_at+=7d`. Add payment any time during trial → provider sub created with first charge at `trial\_ends\_at` → `PAYMENT\_METHOD\_ATTACHED`. Lapse without payment → `TRIAL\_LAPSED` → Part 3's read-only grace. Reactivation within 90 days of archive → new checkout on the *same* subscription row (`expired` → `active` via `ACTIVATED`), history intact.

**\*\*6.2 Pilot codes (the hand-sold Jorhat play, v1 §9.2):\*\*** admin issues `pilot\_codes` rows (30-90 d, org-type-scoped, single-redemption default); redeemed at onboarding step 2 → `trialing` with `provider='pilot'` and the long `trial\_ends\_at`. Everything downstream – banners, D-5/D-1 nudges, conversion checkout – is byte-identical to 6.1; a pilot is just a trial with a longer fuse and a `note` telling you which gym owner shook your hand. Consumer trials: **\*\*none\*\*** – permanent free tier is the funnel (v1 §9.1; unchanged, restated so nobody "helpfully" adds one later).

## ## 7. Plan changes & proration (the money-math rules, exhaustively)

Change	Effect	Billing mechanics
--- ---	---	---
<b>**Org upgrade**</b> (seat pressure / feature)	entitlements + seat cap	
<b>**immediately**</b>	one-off proration charge now = `(new-old) × days_left/period_days` (minor-units, floor); provider plan swap scheduled at cycle end (`pending_plan_id` + provider schedule API). Stripe: native `proration_behavior=create_prorations` does both in one call.	

**\*\*Org downgrade\*\***	at period end	blocked while live non-complimentary members > target cap (Part 3 §4.6 guided flow); else `pending\_plan\_id`, applied by the expiry sweep → `PLAN\_CHANGE\_APPLIED`
Consumer **\*\*monthly** → **annual\*\***	immediately	charge full year now; `current\_period\_end = now + 1y + days\_left\_of\_month` (remaining paid days convert to bonus days – no refund math, user visibly wins)
Consumer **\*\*annual** → **monthly\*\***	at period end	`pending\_plan\_id`; Stripe portal handles its own book natively
Mobile any	store-managed	`PRODUCT\_CHANGE` mirrors it; we never compute
Currency-book switch	not supported in-place	cancel at period end → resubscribe in new book (rare; support-documented)

Property test (§14): for every (old, new, day-in-cycle) triple – proration  $\geq 0$ , never exceeds new plan price, and upgrade-then-immediate-downgrade never mints negative money.

## ## 8. Dunning & grace (one timeline, all providers)

Day 0 `PAYMENT\_FAILED` → `past\_due` (**\*\*entitlements keep granting\*\*** – Part 4 resolver; punishing a member mid-workout for their gym owner's expired card is how you lose both) → notify owner/user (email + push; org: banner turns red per Part 3 §4.2). Providers retry on their own schedules (Razorpay auto-retry / Stripe Smart Retries  $\approx 3$  attempts); our nudges at day 0, 2, 4 with an "update payment" deep link (Razorpay: fresh mandate flow; Stripe: portal). Day 5 grace sweep → `GRACE\_EXPIRED` → degrade + honest copy ("members are on the free tier until payment resumes"). Any successful charge at any point → `RENEWED` → instant restore. Org-side dunning mail goes to `billing\_profiles.billing\_email` U owner – the front desk paying the bills is often not the owner reading the app.

## ## 9. Cancellation, refunds, disputes

**\*\*Cancel\*\*** is always *at period end* (access through what was paid; `CANCEL\_REQUESTED`); immediate termination exists only inside the refund path. Consumer cancel: two taps, no retention screen (dark-pattern-free is a brand asset; the win-back happens later via v1 §8's prompts, not at the exit door). Org cancel: Part 3 §4.6's one honest retention screen (pause-1-month · Micro downshift) → reason survey → confirm.

**\*\*Refund policy (ratified):\*\*** consumer – **\*\*7-day no-questions refund on the first charge\*\*** of any plan (self-serve button on Billing;

`REFUND\_FULL\_FIRST` → expire; kills most disputes before they exist);  
renewals – goodwill case-by-case via support, default no (they had 26+ notified days). Org – no refunds by default (they had a free trial);  
pro-rata service-credit for verified outages, at your discretion, issued as bonus days not money (simpler books). IAP – store-owned; support copy links the store's refund page and we honor whatever `EXPIRATION` says.  
**\*\*Disputes/chargebacks:\*\*** provider dispute webhook → sub to `past\_due` (freeze upside, don't nuke data) → alert you with an auto-assembled evidence pack (signup IP/UA, `audit\_log` trail, `usage\_daily` showing actual product use, invoices) – usage evidence wins the winnable ones and takes the worker 0 minutes to compile. Two disputes on one owner → account flagged, manual review before any new subscription.

## ## 10. Entitlement propagation (closing the loop)

Single choke point = Part 4 §4.1 resolver · Redis cache 60 s · **\*\*bust triggers\*\*** (exhaustive): any §3 transition · `gym\_members` insert/removal · `plans.entitlements` edit (admin) · `pending\_plan\_id` application. Clients read `GET /v1/me/entitlements` (ETag; returns effective doc + `source: own|org|free` + `renewsAt`) to *\*shape UI\**; the server re-derives on every gated request regardless – client entitlements are a hint, never an authority. Grace semantics restated once: `past\_due` grants, `expired` doesn't, and the 60-second cache is why a gym owner's payment fixing itself feels instant to 150 members.

## ## 11. Invoicing & tax hooks (engineering now, CA later)

Every `ACTIVATED`/`RENEWED`/proration charge → worker renders **\*\*our own invoice PDF\*\*** (org logo-free, sequential `invoice\_no` per Indian FY from `invoice\_counters` – `2026-27/000041`), stores to R2 `invoices/`, emails it, lists it on Billing (Part 3 §4.6). `billing\_profiles` captures legal name/GSTIN/address at first org checkout (optional for consumers). `invoices.tax\_minor` + a `tax\_note` line on the PDF exist from day one so enabling GST is a **\*\*data + template change, not a schema change\*\***. ⚠ *\*Not tax advice:* GST registration thresholds, SAC codes, place-of-supply for the USD book, and export-of-services treatment are questions for your CA before the first paid invoice – the schema above is deliberately shaped so whatever they answer slots in.

## ## 12. Abuse & safety rails

Server-side pricing only (client sends `plan\_code`, nothing else) · hosted  
checkouts only (no raw card handling, no card-testing surface on our API)  
· webhook raw-body + signature + dedupe (\$4) · pilot codes  
single-redemption, expiring, audit-logged · trial re-abuse (org deletes,  
re-signs for another 7 days): allowed once – second trial for a matching  
owner email/phone requires a pilot code (a soft gate that costs honest  
users nothing) · provider/plan mapping asserted at boot (\$4.1) · all money  
mutations behind `Idempotency-Key` and written to `audit\_log` (v1 \$10) ·  
secrets: separate webhook secrets per provider per environment, rotated  
with the \$0 doctrine that rotation is routine, not incident response.

## ## 13. Finance analytics

Events (v1 \$16 taxonomy, money subset): `paywall\_viewed{trigger}` ·  
`checkout\_started{plan,provider}` · `subscribe\_activated` ·  
`trial\_converted{days\_used}` · `plan\_changed{from,to}` ·  
`payment\_failed/recovered` · `churned{reason}` · `refunded`. **\*\*MRR view\*\***  
(admin panel, next to the v1 \$9.3 cost dashboard – margin on one screen):

```
```sql
```

```
CREATE VIEW mrr AS
```

```
SELECT p.audience, p.currency,
```

```
       sum(CASE p."interval" WHEN 'month' THEN p.price_minor ELSE
```

```
p.price_minor/12 END) AS mrr_minor,
```

```
       count(*) AS subs
```

```
FROM subscriptions s JOIN plans p ON p.id=s.plan_id
```

```
WHERE s.status IN ('active','past_due') GROUP BY 1,2;
```

```
```
```

Weekly finance digest to you: MRR by book, trials in flight, conversion %,  
dunning saves, per-org cost vs revenue outliers.

## ## 14. Test matrix & acceptance criteria (money code graduates only through this gate)

Sandbox suites per provider (Razorpay test mode, Stripe test clocks, RC  
sandbox store): happy path trial→attach→activate→renew ×2 → cancel →  
period end · dunning path with **\*\*fake clock\*\*** (fail day 0 → nudges 0/2/4 →  
recover day 3 ⊥ expire day 5) · duplicate webhook ×5 → one transition  
(dedupe) · **\*\*out-of-order\*\*** (`charged` before `activated`) → correct final  
state via fetch-authoritative · replayed old event after refund → no

resurrection (illegal-transition alert fires) · cross-channel double-pay guard both directions (§5.4) · seat-blocked downgrade → guided flow → succeeds after removals · proration property test (§7) across 1,000 random triples · card-less trial lapse → Part 3 read-only console → reactivation restores history · pilot code full lifecycle · resolver flips within 60 s of every §10 bust trigger (integration-timed) · one-live-sub invariant: concurrent double-checkout race admits exactly one (DB test like Part 4's seat race).

**\*\*Done when:\*\*** every row above is a green CI job against sandboxes · a stranger can pay you in INR and USD on web and via IAP on a test device · you can narrate any subscription's life from `audit\_log` alone · and killing the worker mid-webhook-burst loses nothing (queue + dedupe replay test).

## ## 15. Build mapping & open items

Lands in **\*\*v1 Phase 3\*\*** exactly as scheduled (weeks 6-8): §5 checkouts + §4 adapters + §3 machine + sweeps + §6 trials/pilots are the Phase-3 core; §11 invoicing and §13 digest ride the same phase's worker; §9 dispute pack and annual-org upsell copy can trail by a sprint. **\*\*Open items (external, start now — lead times, not code):\*\*** CA session (GST posture for INR + export treatment for USD book) · Apple Small Business + Google 15% enrollments · Razorpay live-mode KYC for subscriptions · Stripe account activation for export receipts. None block Phase 1-2 engine work.

---

*\*— End of Part 5. Queue: **\*\*Part 6 — Mobile\*\*** (pose-pipeline spike protocol, offline sync, RC integration test plan, store listings & privacy labels) · Part 7 — Retention playbook · Part 8 — Ops runbook.\**

## # Part 6 — Mobile App (Expo / React Native): Full Specification

**\*\*Prerequisites:\*\*** Architecture v1 §12 · Part 2 (PoseFrame §2.1, mirroring §2.2, outputs §2.4, bundle §9.3, perf gate §7.6) · Part 4 (client-generated IDs = sync idempotency) · Part 5 (§4.3 RevenueCat, §5.4 cross-channel guards) · Part 2B (provisional calories §2.2, barcode path §3.5).

**\*\*Scope:\*\*** the developer-workflow change · app architecture & screen parity · the pose pipeline with a pass/fail spike protocol · the offline sync protocol in full · background-GPS Running · push, IAP, auth, assets · EAS build/update pipeline · store listings & privacy labels · testing, sequence, risks.

## ## 0. Amendments & locked context

1. **\*\*Engine thread placement\*\*** (amends v1 §12's "landmarks stay in worklet-land"): the engine runs **\*\*on the JS thread\*\*** in v1 of the app; the worklet ships only the landmark array across. Measured honestly: one `PoseFrame` is 33×4 floats = **\*\*528 bytes\*\***; at 20 Hz that's ~10 KB/s of marshalling — nothing. What worklet-hosting the engine would cost is real: no Sentry, no debugger, a restricted JS runtime, and every engine bugfix becoming a worklet-compat question. Moving the engine into the worklet (via `react-native-worklets-core`) is the documented **\*\*escape hatch if profiling ever shows bridge pressure — not the default.\*\*** The engine's purity (Part 2 invariants) is what makes this a config choice instead of a rewrite.

2. **\*\*Pose-provider candidate order ratified\*\*** (§3.2): `react-native-mediapipe` (MediaPipe Tasks wrapper — full 33-landmark parity with web) first, `react-native-fast-tflite` second, own native module last. The 17-keypoint MoveNet path is **\*\*rejected up front\*\***: it lacks heels and foot-index, which silently kills `knee\_over\_toe`, `ankle\_angle`, and `heel\_elev` — i.e., lunges, calf raises, and parts of T2. Landmark-set parity is not negotiable; it's what lets golden traces and definitions transfer unmodified.

3. **\*\*Android-first ship.\*\*** iOS follows 2-4 weeks later through EAS + TestFlight — **\*\*no Mac required at any step\*\*** (EAS builds iOS in the cloud; you test on a physical iPhone via TestFlight). Apple Individual developer account (\$99/yr) is the only prerequisite; start that enrollment now (review lead times, like Part 5's open items).

What's already decided and merely referenced here: Expo + **\*\*EAS dev-client, not Expo Go\*\*** (v1 §12 — pose needs native modules Expo Go can't load); shared `@app/engine` + `@app/shared`; SQLite offline-first; RC for IAP; Running is mobile-only with web showing the install prompt.

---

## ## 1. The workflow change (read this first – it's the only disruption to how you build today)

Your current loop (Expo Go on your Android) becomes, **\*\*once\*\***:

1. ``eas build --profile development --platform android`` → cloud-builds a **\*\*custom dev client APK\*\*** (~15 min) → install on your phone.
2. Daily loop is unchanged in feel: ``npx expo start --dev-client`` → scan QR → fast refresh exactly as in Expo Go.
3. Rebuild the dev client **\*\*only when native deps change\*\*** (add/upgrade vision-camera, mediapipe, RC, location) – expect a handful of rebuilds during Phase 5, then rarely.

Windows is fine for everything: Android dev needs no Android Studio (EAS builds remotely; ``adb`` optional for logs), and iOS never touches your machine. Build profiles (in ``eas.json``): ``development`` (dev client, internal), ``preview`` (release-mode APK/IPA for testers + the pilot gym), ``production`` (store artifacts). Secrets (RC keys, Sentry DSN, API URL) come from EAS env per profile – never committed, per standing doctrine.

---

## ## 2. App architecture & screen inventory

**\*\*Position in the monorepo:\*\*** ``apps/mobile`` consumes ``packages/engine`` (pose logic, untouched) and ``packages/shared`` (zod types, API client) – the entire point of v1 \$4's layout. State: React Query (server) + zustand (session) matching web; persistence via the \$4 SQLite layer. Navigation: expo-router. Capability flags from one ``platform.ts`` (``runTracking: true``, ``barcodeScan: true``, ``poseAvailable: detected at runtime`` – \$3.6).

**\*\*Tabs:\*\*** Home (today, streak, quick-start) · **\*\*Workout\*\*** (library → ActiveWorkout) · **\*\*Run\*\*** (mobile-only) · Coach · Progress (history, PRs, body log) · Profile (settings, billing, org membership).

|                  |                                                                     |                                                    |         |
|------------------|---------------------------------------------------------------------|----------------------------------------------------|---------|
| Surface          | Parity with web                                                     | Mobile-specific                                    |         |
| ---              | ---                                                                 | ---                                                |         |
| ActiveWorkout    | same engine, same events, same UI logic ported to RN + Skia overlay | camera pipeline \$3; landscape lock per definition | `setup` |
| Library / detail | same (bundle-driven)                                                | demo videos streamed                               | \$9     |

```
| Coach / Nutrition | same endpoints, quotas | **barcode scan** (2B §3.5:
OpenFoodFacts path – mobile finally unlocks it), camera meal capture |
| Progress / Templates / Gamification | same | – |
| Run suite | web shows browse-only + install prompt (v1 $12) | live
tracking $5, planner, schedules |
| Org membership | join-by-code + **"What {org} can see" sheet** +
leaderboard opt-out (Part 3 §2.4/§4.4 obligations) | QR-scan join (poster
QRs) |
| Billing | reads entitlements; **no prices for org plans ever shown
in-app** (Part 5 §2 – B2B stays off-store) | RC paywall for Pro ($7) |
```

---

### ## 3. The pose pipeline (camera → landmarks → engine → UI)

#### ### 3.1 Dataflow & threading

...

VisionCamera (720p, target 30 fps, front cam default)

```
└ frame processor (worklet): pose model → 33 landmarks + confidences
 └ shared value (latest landmarks) → Skia overlay draws at UI
rate (decoupled)
 └ runOnJS @ inference rate (≤ 24 Hz): Float32Array(132) + t +
aspect + mirrored
 └ PoseProvider adapter → PoseFrame (Part 2 §2.1,
mirrored=true for front cam per §2.2)
 └ @app/engine.push() → FrameResult / RepEvent / cues
→ UI + voice
...
```

Rules: the **\*\*overlay never waits for the engine\*\*** (draws the latest shared landmarks – camera feels 60 fps even if inference is 20); the engine consumes every frame it receives and is fps-agnostic by construction (Part 2 time-constant smoothing); inference is frame-dropping (process-latest, never queue – a backlog of stale frames is worse than a lower rate).

#### ### 3.2 The spike (2 days, run before any screen is built) – pass/fail bars

Build one throwaway **\*\*Benchmark screen\*\***: camera + candidate pipeline + counters (inference ms p50/p95, delivered Hz, JS-thread FPS, memory, battery %, thermal state) + a "record trace" button (Part 2 §7.2 format – the recorder ships in the spike because its output *\*is\** the parity evidence).

| Bar                        | Pass                                                                                                                                                                                                                         | Fail action                                                     |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|
| Throughput                 | $\geq 20$ Hz landmarks sustained <b>**20 min**</b> , mid-device (§3.5)                                                                                                                                                       | try `lite` model variant → next candidate                       |
| Thermal                    | no throttle-crash; $\geq 15$ Hz at minute 20                                                                                                                                                                                 | drop camera to 24 fps / 640p → next candidate                   |
| UI health                  | JS thread $\geq 50$ fps during session                                                                                                                                                                                       | move overlay fully to Skia/worklet side (it should already be)  |
| Battery                    | $\leq 20$ % drain / 20-min session                                                                                                                                                                                           | resolution/model step-down                                      |
| <b>**Landmark parity**</b> | replay one recorded video through web MediaPipe <i>*and*</i> the device pipeline → per-landmark RMSE small enough that the three ✓ exercises' golden traces produce <b>**identical rep counts and fault sets**</b> on-device | candidate rejected regardless of speed – parity is the contract |
| Memory                     | stable over 20 min (no creep > 100 MB)                                                                                                                                                                                       | leak hunt or reject                                             |

Candidate order per §0.2. Deliverable: a one-page decision record (numbers per bar per candidate) committed to the repo – the same discipline as definition rationales.

### ### 3.3 Model & camera configuration

BlazePose **\*\*full\*\*** as default, **\*\*lite\*\*** as the automatic step-down (§3.6); 1280×720, front camera, `mirrored=true` end-to-end (the Part 2 §2.2 policy means the engine and definitions never know which camera); landscape-orientation prompt driven by the definition's `setup` block exactly like web. Torch off, exposure auto, focus continuous.

### ### 3.4 Performance budget (per frame, mid-device)

Inference  $\leq 35$  ms (full) /  $\leq 22$  ms (lite) · worklet→JS marshal  $\leq 1$  ms · adapter  $\leq 0.5$  ms · engine  $\leq 1.5$  ms (Part 2 §7.6 gate, unchanged – same number web and mobile) · UI event handling  $\leq 2$  ms. Total JS-side  $\leq 5$  ms/frame keeps the thread breathing at 20 Hz with 10× headroom.

### ### 3.5 Device matrix & support floor

Test trio: your Android · one ₹10-12k 4 GB device (Redmi/Realme class – **\*\*the pilot gym's median phone\*\***; buy one, it's the most valuable ₹11k in this project) · one Android 10 / 3 GB floor unit. Support floor: Android 9+, 3 GB; iOS: iPhone XR+ (trivially clears every bar). `minSdkVersion` 26, but capability detection decides features, not SDK level.

### ### 3.6 Graceful degradation ladder (runtime, automatic, honest)

full model → lite model (delivered Hz < 15 for 10 s) → 640p (still < 15) → **\*\*log-only mode\*\***: pose unavailable on this device → manual rep counting UI + honest copy ("Form checking needs a bit more phone than this one – your workout still counts"). Degradation events → PostHog with device model, so the support floor is data, not guesswork. A user on a weak phone gets a working app, never a slideshow.

---

## ## 4. Offline-first data layer & the sync protocol (in full)

**\*\*Doctrine\*\*** the phone is a *\*client-side ledger\** of things the user did, append-only, eventually delivered. Nothing here is a distributed-systems problem because Part 4 made every entity **\*\*client-ID'd and idempotent\*\*** – there are no conflicts to resolve, only deliveries to complete. Offline never blocks a workout: pose, logging, streak-provisional, and cached history all work in airplane mode; only metered features (coach/meal/route-gen) honestly require a network, and say so.

### ### 4.1 SQLite schema (expo-sqlite; mirrors, not masters)

```
```sql
local_workouts(id TEXT PK, payload TEXT, started_at INT, synced INT
DEFAULT 0);
local_runs(id TEXT PK, payload TEXT, points BLOB, started_at INT, synced
INT DEFAULT 0);
sync_queue(id TEXT PK, kind TEXT, entity_id TEXT, attempts INT DEFAULT 0,
next_attempt_at INT, last_error TEXT, status TEXT DEFAULT
'pending');
```

```
cached_bundle(channel TEXT PK, version INT, sha256 TEXT, json TEXT,
  fetched_at INT);
cached_entitlements(json TEXT, etag TEXT, fetched_at INT);
cached_history(page_key TEXT PK, json TEXT, fetched_at INT);
kv(key TEXT PK, value TEXT);          -- profile, streak-provisional,
flags
```

```

### ### 4.2 Write path

`engine.endSet()` → SetSummaries accumulate in the session → workout finish → **\*\*one transaction\*\***: insert `local\_workouts` (id = client UUID, Part 4 §3.5) + enqueue `sync\_queue(kind:'workout')` → UI shows the workout instantly with a subtle "pending 0" badge. Runs identical (`kind:'run'`, GPS points crash-safe per §5.3). App-kill mid-set loses at most the in-progress set — an accepted, stated bound (summaries exist only at set end by design).

### ### 4.3 Delivery engine

Triggers: connectivity regained (NetInfo) · app foreground · immediately post-enqueue · a 15-min background fetch (best-effort; OS-throttled — treated as bonus, never relied on). Per item: `POST /v1/workouts/sync` (or `/runs/sync`) → **\*\*any 2xx or 409 ⇒ delivered\*\*** (server `ON CONFLICT DO NOTHING` returns canonical state; a retry of a delivered item is a no-op by construction) → mark `synced`, reconcile server-computed fields (kcal per 2B §2.2, streak, achievements) into caches. Failures: network/5xx → exponential backoff with jitter (1 m → 2 h cap), forever (append-only data never expires from the queue); 4xx validation → park item `status:'rejected'`, surface in Profile → "sync issues" with a *\*send-report\** action (this is a bug in our code, not the user's problem — the report is for you). Battery courtesy: batch flushes, never hold a wakelock for sync.

### ### 4.4 Read path & caches

History/progress/leaderboards: server truth, cursor-cached in `cached\_history`, stale-while-revalidate; pending locals overlay on top with the badge. **\*\*Definition bundle\*\*** boot uses cache instantly; background `GET /v1/exercise-definitions` with `If-None-Match` (Part 2

§9.3); new bundle → applied on next session start (never mid-set);  
`minEngineVersion` filter as specced. **\*\*Entitlements offline:\*\*** last-known  
doc honored up to **\*\*72 h\*\*** stale for UI gating; past that, gate as  
free-tier for anything metered (which is offline-dead anyway) –  
pose/logging never gate. Clock skew: client timestamps stand; server  
clamps `started\_at` > now+24 h and flags `quality\_flags:['clock\_skew']`  
(Part 4 field).

### ### 4.5 Housekeeping & multi-device

Synced locals pruned after 90 days; queue > 200 pending → gentle "you've  
been offline a while" note. Two phones, one account: append-only makes it  
safe; streaks/achievements are **\*\*server-computed at sync\*\*** (authoritative)  
with client-provisional display – the only place a second device can  
"surprise" you is a streak looking better after sync, which is the good  
direction.

---

## ## 5. Running (mobile-only; the feature web advertises but doesn't have)

### ### 5.1 Permissions (the compliance-critical flow – Play rejects apps that fumble this)

Foreground-only first: start a run → `ACCESS\_FINE\_LOCATION` request with  
one prior in-app screen stating exactly why. Background is asked **\*\*only\*\***  
when the user enables "keep tracking with screen off" – preceded by Play's  
required **\*\*prominent disclosure\*\*** screen (our copy: "AI Home Gym collects  
location data to record your running route even when the screen is off or  
the app is closed. Routes are private to you – never shared with your gym  
or anyone else."), then the OS "Allow all the time" step. iOS mirrors:  
WhenInUse → Always upgrade with  
`NSLocationAlwaysAndWhenInUseUsageDescription` in the same words. **\*\*Play's  
background-location review\*\*** requires a screen-recording of this exact  
flow + a policy declaration – deliverable in §12, prepared before  
submission, not after rejection.

### ### 5.2 Tracking mechanics

expo-location + TaskManager background task · accuracy High,  
`distanceInterval: 5 m` / `timeInterval: 3 s` · **\*\*Android foreground service\*\*** with persistent notification (pause/stop actions) and  
`foregroundServiceType: 'location'` declared (Android 14 requirement) ·  
auto-pause when speed < 0.7 m/s for 30 s (resume on movement) · GPS-loss:  
keep timing, flag the gap, straight-line the *\*display\** only — distance  
uses measured points, honesty per Trust doctrine (no invented meters).

### ### 5.3 Data & save

Points stream to `local\_runs.points` (crash-safe: a killed app recovers  
the partial run on next launch and offers save/discard) → on stop:  
polyline-encode, splits computed on device (port of `scoring.py`, v1 §12),  
**\*\*pace-based MET\*\*** kcal (running rows from the Compendium keyed by avg  
pace; provisional on device, server recompute at sync stamps  
`kcal\_calc\_version` — the Part 2B pattern, third use) → route name via  
server geo cache at sync (cached reverse-geocode; does **\*\*not\*\*** consume the  
`route\_gen` quota — that quota is the planner's). Battery target ≤ 8 %/30  
min tracked.

### ### 5.4 Planner & privacy

Route planner (ORS loop generation) is the `route\_gen`-quota'd feature,  
cached per v1 §6.1. Privacy: GPS data is org-invisible (Part 4 §3.9), and  
**\*\*share cards trim 200 m from both route ends\*\*** by default — the map thumb  
shows your run, not your front door.

---

## ## 6. Push notifications

expo-notifications → token → `POST /v1/push-tokens` (Part 4 table).  
Android channels: `workout-reminders`, `streaks`, `org`, `billing`  
(user-mutable in OS settings — respect it, don't re-ask). All  
send-decisions, caps, and quiet hours are **\*\*server-side\*\*** (v1 §15); the  
app only renders and deep-links (`aihg://workout/...`,  
`aihg://org/join?code=`). Notification taps land logged-in or route  
through auth first — deep links never dead-end on a login wall.

## ## 7. IAP (RevenueCat — mechanics; policy already in Part 5)

Configure RC with **\*\*our user UUID\*\*** at login (Part 5 §4.3 – deterministic restores/transfers) · entitlement id `pro` · payroll placements: locked-exercise tap, metered-quota hit, Progress-history gate – each records `paywall\_viewed{trigger}` (Part 5 §13) · Restore Purchases on login + payroll · the §5.4 cross-channel guards verbatim (web-Pro hides purchase; IAP-Pro shows "managed in your app store" on web) · sandbox test list: purchase, renewal (accelerated), billing-issue grace, cancel, restore on second device, PRODUCT\_CHANGE may – each asserting our `subscriptions` row lands correctly via the RC webhook, because the app never writes entitlements (Part 5 §10).

## ## 8. Auth on mobile

Refresh token in **\*\*SecureStore\*\*** (Keychain/Keystore), access token in memory only · Google Sign-In via `@react-native-google-signin` (dev-client native module) → ID token → existing `auth\_identities` exchange – same table, same backend path as web (Part 4 §3.1) · session revocation honored (401 + refresh-family reuse → local logout everywhere) · optional biometric app-lock deferred to a fast-follow (flagged, not built).

## ## 9. Assets (the 58-demo problem)

Bundling 58 GIFs would be a 150+ MB app – instead: **\*\*convert once\*\*** GIF → MP4 (H.264) + WebP poster (~80-90 % smaller), upload to R2 `app-assets/` behind Cloudflare CDN with immutable cache headers → app bundles only T1 posters (~1 MB) and streams the rest with expo-image/av caching. Library works offline for anything ever viewed; never for never-viewed T3 videos – acceptable and stated. The conversion script lives in `infra/` and runs in CI when assets change.

---

## ## 10. Observability

Sentry RN with EAS source maps, every event tagged `{engineVersion, bundleVersion, deviceModel, poseTier: full|lite|logonly}` · performance transactions on pose sessions (delivered Hz, dropped frames, degradation events) · PostHog RN mirroring the v1 §16 taxonomy + mobile-only events (`pose\_degraded{step}`, `offline\_workout`, `sync\_delivered{age\_s}`, `bg\_location\_granted`) · a hidden **\*\*Diagnostics screen\*\*** (7 taps on

version): delivered Hz live, model tier, bundle version, queue depth, last sync, and **"run device parity check"** – replays the bundled squat golden trace through the installed engine and asserts rep/fault equality (the Part 2 harness, shipped in the app; when a gym owner's phone "counts wrong," this button answers *\*engine or camera?\** in 30 seconds).

## ## 11. EAS pipeline & release trains

``runtimeVersion: { policy: 'fingerprint' }`` – OTA updates apply only to compatible native builds, mechanically (the classic OTA-crash footgun deleted by config) · channels: ``preview`` (pilot gym testers) and ``production``, mapped per build profile · **\*\*release trains:\*\*** JS-only changes ship OTA weekly (definitions already bypass releases entirely via the bundle – Part 2's design doing its job); native changes batch into a monthly store release; store submits via ``eas submit`` both platforms · versioning ``1.x.y`` app / build numbers auto-incremented by EAS · rollout: Android staged 10 % → 50 % → 100 % with a 24 h Sentry-crash gate between steps; halt = one dashboard toggle.

## ## 12. Store listings, privacy labels & review-proofing (what on-device processing has earned)

**\*\*The two sentences that define the listing (and are architecturally true):\*\*** *"Your camera is processed entirely on your phone – workout video never leaves your device."* and *"Meal photos are analyzed and immediately discarded – never stored."*

**\*\*Google Play Data Safety (exact rows):\*\*** Location (precise) – collected, not shared, optional, users can delete · Photos – collected (meal photos are transmitted for analysis), **\*\*ephemeral processing\*\***, not stored, not shared · Health & fitness (workout activity) – collected, not shared, deletable · Personal info (name, email) – collected, account ·

**\*\*Camera/pose video – NOT declared as collected, because it never leaves the device\*\*** (on-device processing is exempt; this row's absence *\*is\** the selling point) · No data sold; no ads; encrypted in transit; deletion available in-app (Part 4 §5.2).

**\*\*Apple privacy nutrition label:\*\*** Health & Fitness, Contact Info, Location – linked to identity, App Functionality only · Photos – App Functionality, not linked beyond the meal log it creates · **\*\*Tracking: none → no ATT prompt.\*\***

**\*\*Permission strings (same words both platforms, honesty doctrine):\*\***

Camera – "AI Home Gym checks your exercise form on your device. Video never leaves your phone." · Location Always – the §5.1 disclosure copy · Notifications – standard.

**\*\*Review-proofing checklist:\*\*** background-location screen-recording + policy declaration (§5.1) ready **\*\*at first submission\*\*** · Android 14 FGS-type declaration · demo account with seeded data for reviewers · health-content disclaimer ("general fitness guidance, not medical advice") in listing + onboarding – doubly load-bearing given the clinic segment's positioning boundary (Part 2B §7) · age rating: no restricted content; IAP declared · target API level current.

## **## 13. Testing**

Layered exactly like the rest of the system: **\*\*engine\*\*** – untouched, Node golden traces already gate it (Part 2 §7) · **\*\*device parity\*\*** – §10's on-device trace replay, run in CI on one real device via EAS + Maestro cloud lane, and manually on the §3.5 trio before any store release · **\*\*E2E (Maestro)\*\*** – flows: signup→first workout offline→airplane-off→sync verified server-side · join-by-code + visibility sheet · paywall→sandbox purchase→entitlement flip · run with backgrounding→save→route named · degradation ladder forced via debug flag · **\*\*sync engine\*\*** – unit-tested with a mock network: dedupe on 409, backoff schedule, park-on-4xx, crash-recovery of a partial run · **\*\*beta ramp\*\*** – internal (you) → `preview` channel to the pilot gym's members (v1 Phase 5's closed Android test) with the §10 diagnostics screen as your remote eyes.

## **## 14. Build sequence (Phase 5, weeks 10–14 – expanded to working granularity)**

**\*\*W10:\*\*** dev-client stands up (§1) · spike days 1–2 (§3.2) with decision record · Benchmark screen kept as the Diagnostics seed. **\*\*W11:\*\*** ActiveWorkout on-device (camera→engine→overlay→voice) · SQLite layer + write path · auth + SecureStore. **\*\*W12:\*\*** sync engine + reconciliation · library/progress/coach/nutrition screens on shared client · bundle caching · barcode scan. **\*\*W13:\*\*** Running end-to-end (§5, permissions flow filmed for Play) · push · RC paywall + sandbox suite. **\*\*W14:\*\*** asset pipeline · store listings/labels/review pack (§12) · Maestro flows green · `preview` build to pilot-gym testers · staged production rollout begins.

**\*\*Acceptance (v1 Phase 5 line, now fully expanded):\*\*** a member on the ₹10-12k device completes a **\*\*full offline workout at  $\geq 20$  Hz pose\*\***, thermally stable for 20 minutes, and it syncs cleanly later (409-idempotent) · the three  exercises pass the on-device parity check byte-for-byte with web · a real outdoor run records through screen-off, survives an app kill, saves with splits + cached route name, and never appears to any org · sandbox IAP flips `subscriptions` via webhook with both cross-channel guards proven · background-location review pack accepted by Play on first submission · crash-free sessions  $\geq 99.5\%$  across the beta week.

## ## 15. Risks & mitigations

| Risk                                                          | Mitigation                                                                                                                                                          |
|---------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pose perf on low-end Android                                  | \$3.6 ladder + log-only floor; support floor is telemetry-driven, not hoped                                                                                         |
| `react-native-mediapipe` maintenance risk (community wrapper) | adapter isolates it (PoseProvider interface); fallback candidates pre-ordered \$0.2; worst case = own thin native module over MediaPipe Tasks (bounded, documented) |
| Play background-location rejection                            | \$5.1 disclosure + \$12 review pack prepared pre-submission; feature degrades to screen-on tracking if ever forced                                                  |
| OTA/native mismatch crashes                                   | fingerprint runtimeVersion (\$11) makes the mismatch impossible to ship                                                                                             |
| GIF/app-size bloat                                            | \$9 pipeline; app stays < 60 MB installed                                                                                                                           |
| Two-store release overhead (solo dev)                         | Android-first (\$0.3); iOS rides the same EAS pipeline when Android is proven                                                                                       |

---

*\*— End of Part 6. Queue: **\*\*Part 7 — Retention & Growth Playbook\*\*** (exact mechanics, triggers, and copy for the v1 \$21 set: Form Score™, streak freezes, TV-mode leagues, WhatsApp share cards, referral, programs, recap engine) · Part 8 — Ops runbook.\**

## # Part 7 — Retention & Growth Playbook: Full Specification

**\*\*Prerequisites:\*\*** Architecture v1 §21 (the ranked feature set – this document is its mechanics) · Part 2 (message keys, F-families, committed scores) · Part 2B (display standard, recommender progression/recovery terms) · Part 3 (boards/TV/poster/at-risk – built there, referenced here) · Part 4 (streaks/achievements/challenges DDL) · Part 5 (entitlements, `share\_watermark`) · Part 6 (push channels, deep links, offline-provisional reconciliation).

**\*\*Scope:\*\*** exact mechanics, triggers, and copy (EN message-key seeds; HI/AS are translation passes) for Form Score™, streak freezes, challenges & the City League, share cards & referral, programs, PR/badges, the weekly recap & resurrection ladder, localization/voice, the QoL trio – plus the global notification budget, per-feature kill criteria, and DDL addendum B.

## ## 0. Doctrine (four rules every mechanic below obeys)

1. **\*\*Honest numbers retain; fake ones churn\*\*** – the Trust Layer applies to gamification too. Deltas display only when both sides are real (§2.2); calorie totals in recaps are ranges (2B §2.3); streaks are server-truth (Part 6 §4.5).

2. **\*\*Notification restraint is a retention feature.\*\*** One engagement push per user per day, hard cap, server-enforced (v1 §15) – every send in this playbook fits inside that budget (§13) or doesn't ship.

3. **\*\*No dark patterns – enumerated, not implied.\*\*** Banned: purchasable streak freezes (monetizing anxiety), paywalls on the streak-loss screen (kicking someone who's down), fake urgency ("only 2 spots!"), guilt copy, opt-out-hostile emails. The cancel flow already set this bar (Part 5 §9); growth features don't get an exemption.

4. **\*\*Every mechanic ships with a metric and a kill criterion\*\*** (§14). A retention feature that doesn't move retention is UI debt.

**\*\*The loop these features assemble:\*\*** workout → *\*visible\** progress (Form Score) → identity (streak, mastery, badges) → social proof (boards, challenges) → share (cards → referral → growth) → return trigger (recap, freeze save, program day). Each section below is one arc of that loop.

## ## 1. What each feature is for (metric map)

| Feature | Primary metric | Guardrail |  |
|---------|----------------|-----------|--|
| ---     | ---            | ---       |  |

| Form Score™ §2 | W1→W4 retention (progress visibility) | score inflation (distribution drift alarm) |  
| Streaks + freezes §3 | DAU/WAU ratio, D7 | churn spike after streak loss |  
| Challenges / League §4 | WAU, org adoption % | gaming flags (v1 §14) |  
| Share cards + referral §5 | K-factor, activated referees | fake-account rate |  
| Programs §6 | D30/D60 (the finish-line effect) | abandonment week ≥ 2 |  
| PR + badges §7 | session frequency | notification fatigue |  
| Recap + resurrection §8 | resurrection rate, recap CTR | unsubscribe rate |

---

## ## 2. Form Score™ (the proprietary metric – feature #1)

### ### 2.1 Definition (no new math – packaging of what the engine already commits)

- **\*\*Set score:\*\*** `SetSummary.avgFormScore` (Part 2 §3.8, committed rep scores only – live frame-quality never persists, so the score can't be gamed by posing between reps).
- **\*\*Weekly Form Score:\*\*** mean of set scores in the user's week (user TZ), **\*\*integer\*\***, shown only at ≥ 5 scored sets that week (Part 3 §3.2's gate, reused); otherwise "—" with copy `form.week.notyet`: *"A few more sets and your weekly Form Score appears."*
- **\*\*Per-exercise mastery states\*\*** (unifies with 2B §4.2's progression term – one ladder everywhere): `Learning` (< 10 scored sets lifetime) → `Developing` (avg < 70) → `Solid` (70–84) → `Mastered` (avg ≥ 85 over the last 3 sessions of that exercise). Mastered is exactly the recommender's promote-trigger, so the library badge, the "ready to progress" chip, and the program auto-swap prompt (§6.4) can never disagree.

### ### 2.2 Display rules (Trust doctrine applied)

Weekly delta (`84 → 88`) renders only when **\*\*both\*\*** weeks pass the 5-set gate; a partial week shows the current number alone – a delta against noise is a lie with an arrow on it. Method sheet `form.score.how` (2B §6 table row, one paragraph): scored per rep from your joint angles against the exercise's standards, on your device. Surfaces: Home hero card ·

exercise detail (mastery ring) · recap (\$8) · share cards (\$5) · org member drawer & boards (Part 3 – min-set gates already aligned) · TV mode.

### ### 2.3 Integrity & the ™

Global boards show verified users only (v1 \$14 – unchanged); org boards are socially self-policing. Distribution telemetry per definition (Part 2 \$9.4) doubles as the inflation alarm: a threshold tweak that shifts the population +10 shows up before users feel "everyone's a 95 now." On the ™: use it as branding freely; **\*\*registered\*\*** protection is a trademark filing – one conversation with an IP attorney when revenue justifies it, noted and parked.

---

## ## 3. Streaks & freeze tokens (feature #2)

### ### 3.1 What keeps a streak alive

≥ 1 synced **\*\*workout, run, or F12 mobility session\*\*** in a calendar day, user's timezone (`users.timezone`, captured from the device at login – Part 4 left it nullable for exactly this). F12 counting is deliberate: stretch days are streak-safe rest days, which makes the streak compatible with recovery instead of at war with it (v1 \$21 #14's "guilt-free" mechanic, delivered through content rather than exceptions).

### ### 3.2 Freeze tokens (earn-only – Doctrine 3)

Earn **\*\*1 per 7 consecutive active days\*\***, bank cap **\*\*3\*\*** (Part 4 `freezes\_available`). On the first missed day the server **\*\*auto-spends\*\*** one at the day-close sweep – the user wakes to a saved streak, not to a decision: push `streak.freeze.used` – *"❄️ Streak freeze used – your 12-day streak is safe. 2 left."* Earning: quiet in-app moment + `streak.freeze.earned` toast – *"You earned a streak freeze. Banked for a busy day."* Never purchasable, never gifted by support as a habit (one goodwill exception per user per year, admin-tooled, so kindness exists but can't be farmed).

### ### 3.3 The loss moment (where most apps churn people)

No freeze left → streak resets. Copy `streak.lost` is compassionate and forward-looking: *"14 days was real – nobody can take that. Day 1 starts whenever you do."* `longest` is displayed forever next to `current` (the achievement survives the reset). **\*\*Banned on this screen:\*\*** any paywall, any freeze upsell (Doctrine 3). Measured option, not built: monthly "streak repair" – ships only if \$14 shows streak-loss-week churn > 1.5× baseline.

### ### 3.4 The about-to-break nudge (the one push this feature gets)

19:00 user TZ, only if `current ≥ 3` and no qualifying activity today and no freeze available to make it moot – wait, freezes make it *\*low-stakes\**, so: send regardless of freeze balance but let copy reflect it. With freeze: *"No workout yet today – your freeze has you covered, but a 10-minute stretch keeps it clean."* Without: *"Your 8-day streak ends at midnight. Even a short session counts."* Class: engagement (fits the 1/day budget, \$13); auto-suppressed if the app was opened after 17:00 (they know).

### ### 3.5 Mechanics & edges

Server-authoritative at sync (Part 6 §4.5): the day-close sweep (per-TZ batches, same worker pattern as Part 4 §4.3) evaluates yesterday → keep / auto-freeze / reset, then earn-check. Client shows provisional state instantly; server reconciliation only ever *\*improves\** it (offline workouts arriving late can restore a "lost" streak retroactively at sync – the sweep re-runs affected days idempotently). Timezone travel: a day is kept if it qualifies in **\*\*either\*\*** the stored-at-the-time TZ or the new one – travel can never eat a streak. Milestones 7 / 30 / 100 / 365 → badge (\$7) + share-card prompt (\$5).

---

## ## 4. Challenges & the City League (feature #4)

### ### 4.1 Challenge templates (config, not code – Part 4

`challenges.template\_code` + `config`)

| Template | Scope | Win condition (computed from SetSummaries) | Notes |
|----------|-------|--------------------------------------------|-------|
| ---      | ---   | ---                                        | ---   |

```
| `squat30` "30-Day Squat Challenge" | user | daily rep ladder (day N
target in config), 25/30 days | the classic; T1-only variant for free tier
|
| `consistency20` | user | ≥ 20 active days in 30 | pairs with streaks,
forgiving |
| `formfocus` | user | avg Form Score ≥ 80 on chosen exercise, ≥ 12 sets |
mastery on-ramp |
| `org_reps` "Gym Total" | org | sum of member reps in month, target from
config | the notice-board number |
| `org_active` "Adoption Sprint" | org | $\geq X\%$ members active this month |
secretly an owner-retention feature |
| `league_city` | league | $4.3 | gated |
```

Join: one tap from Home/console  $\rightarrow$  `challenge\_participants`; progress: Redis increments at sync + nightly true-up worker (same dual pattern as org counters, Part 3 §3.2); completion  $\rightarrow$  badge + share card + `challenge.done` push (celebration class). Copy seeds: `challenge.joined` *"You're in. Day 1 starts now."*  $\cdot$  `challenge.behind` (day 10+,  $< 60\%$  pace, max once per challenge): *"You're 40 squats behind pace – one solid session catches you up."*

### ### 4.2 Org challenges (console Slice B/C)

Owner creates from templates (Part 3 §2.3 gating: clinics off), appears on member Home + TV mode; progress bar is the lobby-screen centerpiece for the month. No prizes handled by us – the gym supplies its own (a shaker bottle moves mountains); we supply the scoreboard.

### ### 4.3 The City League (B2B growth infrastructure – **\*\*gate: $\geq 5$ paying orgs in one city\*\***, same discipline as the clinic gate)

Season = calendar month  $\cdot$  entry = console opt-in, free  $\cdot$  **\*\*score = avg workouts per participating member\*\*** (min 10 participants to qualify) – per-capita by design so a 60-member Jorhat gym can beat a 300-member Guwahati one; total-volume boards would just rank size. Standings on TV mode + console + a public city page (org names only, no member data – §Part 3 §2.4 boundary applies to leagues too). Winner: digital trophy on their TV mode for the next month + a print-ready *"Jorhat's Most Active Gym – July 2026"* poster asset. Why it exists: owners recruit **\*\*each other\*\*** (a league needs opponents) – the only B2B feature that markets

itself. Anti-gaming: verified summaries only; a gym spiking on flagged accounts (v1 \$14) is quietly excluded from standings with a private note to you, never a public shaming.

---

## ## 5. Share cards & referral (feature #5 – the growth loop)

### ### 5.1 Card generator

Server-side worker renders PNG (1080×1920 story + 1080×1080 square) from templates: **\*\*workout summary\*\*** (exercises, reps, Form Score, streak) · **\*\*PR card\*\*** · **\*\*streak milestone\*\*** · **\*\*challenge complete\*\*** · **\*\*program certificate\*\*** (\$6.5) · **\*\*run card\*\*** (map thumb, 200 m end-trim per Part 6 \$5.4). Org members' cards carry the org logo (Part 3 branding – the gym markets itself every time a member shares; owners will notice). Free tier: small wordmark + referral QR (`entitlements.share\_watermark` – the watermark *is* the growth mechanic); Pro: clean, QR optional. Locale-aware text. Flow: post-workout "Share" → `POST /v1/share-cards {type, entity\_id}` → R2 `share-cards/` (7-day lifecycle, v1 \$7.3) → signed URL → native share sheet, WhatsApp first-class. Rate: 20/day (Redis, quota module).

### ### 5.2 Referral program (give-a-month / get-a-month)

- Every user gets `users.referral\_code` (7-char, ambiguity-free); link `https://<domain>/r/{code}` + the same code typed manually on mobile signup (install-referrer attribution where available, manual entry as the reliable path – first-touch wins, 30-day window).
- **\*\*Reward trigger = referee activation, not signup:\*\*** referee completes **\*\*3 workouts within 14 days\*\*** → both sides receive **\*\*+30 days Pro credit\*\***. Signup-triggered rewards farm fake accounts; activation-triggered rewards farm *users*.
- **\*\*Credits mechanics (DDL addendum B):\*\*** `entitlement\_credits(user\_id, plan\_id-pro, days, source, expires\_unused\_at)`; the Part 4 \$4.1 resolver gains a third UNION arm (rank-10 candidate while a credit window is open). Credits **\*\*bank while a paid sub is active\*\*** and start counting only when no paid plan grants – a paying referrer's credit extends their coverage after cancellation rather than fighting the provider's billing period (no

proration surgery, no Part 5 state-machine exceptions). Bank cap: 180 days/rolling year per referrer.

- Anti-abuse: referee must be a new account · self-referral blocked (email/device heuristics) · device-fingerprint soft-flag → admin queue past 3 same-device referrals · no payment method required (activation gate is the fraud filter) · all grants in `audit\_log`.
- Copy: `referral.cta` *"Give a month of Pro, get a month of Pro – when your friend does their first 3 workouts."* (the condition stated up front
- Trust doctrine, applied to marketing).
- **\*\*Org-to-org referral:\*\*** owner refers owner → 1 month free appended at the referred org's first paid renewal + the referrer's next renewal. Year one: fulfilled manually via admin credit-note (volume will be tens, not thousands); automated only when that stops being true.

---

## ## 6. Programs (feature #6 – the retention backbone)

### ### 6.1 Model (DDL addendum B)

`programs(id, code, name\_key, level, goal, weeks jsonb, status)` – `weeks` = ordered sessions of `{day, exercise\_slug, target\_sets, target\_reps|hold\_s, optional}` · `program\_enrollments(user\_id, program\_id, started\_at, schedule text[], current\_day, completed\_at, abandoned\_at)` – one active enrollment per user (partial unique, same trick as subscriptions).

### ### 6.2 Launch content (authored like definitions: message keys, versioned, no code)

**\*\*Foundations\*\*** (4 wk, 3×/wk, pure T1 – the free-tier program, `entitlements.programs='starter'`) · **\*\*Home Strength\*\*** (6 wk, 4×/wk, T1+T2) · **\*\*Mobility Reset\*\*** (3 wk, daily 15-min F12/F10 – the streak-safe program, and the clinic segment's natural demo). Pro unlocks all; more programs are content drops via config, zero deploys – the EDS philosophy, third application.

### ### 6.3 Session matching (deliberately fuzzy)

A workout counts toward today's prescribed session when  $\geq 60\%$  of prescribed exercises appear in it. Exact-match adherence punishes substitutions and equipment reality; 60% rewards showing up. This same number is Part 3 §6.1's **\*\*On-plan %\*\*** for studios/clinics – consumer feature and B2B feature are one mechanism.

### ### 6.4 Progression & schedule mercy

When an exercise hits **\*\*Mastered\*\*** (§2.1), the next prescribed instance offers the harder family variant – **\*\*prompted, never silent\*\***

(``program.progress.offer``: *"Your squats have been  $\geq 85$  for three sessions. Swap in jump squats for week 3?"*). Missed days shift forward (a program is a sequence, not a calendar); 14 days inactive  $\rightarrow$  ``abandoned_at`` + a no-guilt resume path (*"Week 2, day 1 is right where you left it."*).

### ### 6.5 Completion

Certificate share card + badge + the recommender's next-program suggestion (2B §4.2 surfaces it with its "why" chip). Completion rate per program is the §14 metric; a program with  $< 15\%$  completion gets redesigned, not promoted.

---

## ## 7. PR celebrations & badges (feature #7)

**\*\*PRs\*\*** (``user_prs``, addendum B): per exercise – most reps in a set · best Form Score ( $\geq 8$ -rep sets only, quality gate) · longest hold (F10/F12) · plus run bests (5k/10k time, longest run). Detected client-side against the local cache for the **\*\*instant confetti\*\*** (from ``RepEvent``, zero latency), confirmed server-side at sync (authoritative; multi-device disagreements reconcile quietly – Part 6 §4.5 pattern). In-app celebration suppresses the push (no double dopamine, §13).

**\*\*Badges:\*\*** ~24 at launch – ``badges.py`` port + streak 7/30/100/365, first workout, first sync-after-offline ("Basement Session"), 1k/10k lifetime reps, all-T1-tried ("Full Body Explorer"), Form 90 Club (ten 90+ sets), program completions, referral activated, challenge winner. **\*\*Positive-only catalog\*\*** – no shame badges, no "comeback from being lazy" framing; the catalog is seed data (Part 4 ``achievements``), additions are content.

## ## 8. The weekly recap & resurrection ladder (feature #8)

**\*\*Recap:\*\*** Sundays 18:00 user TZ (per-TZ worker batches, the Part 4 §4.3 pattern) · push + email, one engagement-class send (§13) · contents, display-standard compliant: workouts vs last week · Form Score delta (§2.2 gates) · streak + freezes banked · top exercise · kcal as a **\*\*banded weekly range\*\*** (2B §2.3 total-then-band rule) · one highlight (PR/badge/challenge) · one honest nudge from the recommender's recovery data (2B §4.2: *"Legs untouched for 9 days – recovered and ready"*) · CTA = program day or recommended workout. Skip rule: 2 consecutive zero-activity weeks → recap pauses, resurrection ladder takes over.

**\*\*Resurrection ladder (lapsed = 0 activity):\*\*** D3 push *"Your streak freeze is still banked ❄"* → D7 email (their own stats: *"Your best week was 5 workouts. It's still in you."*) → D14 what's-new + win-back → D30 monthly digest cadence → **\*\*D90 full stop\*\*** (respect is a retention strategy too). Every rung: one-tap per-channel unsubscribe, honored instantly, tracked as the §14 guardrail.

## ## 9–12. Supporting cast (specced brief because the machinery already exists)

**\*\*§9 Localization (#10):\*\*** all copy above ships as message keys (Part 2 App A conventions); HI/AS = native-speaker passes before T1 pilots (Part 2 DoD, unchanged). **\*\*Voice in Assamese:\*\*** Android TTS coverage for `as` is unreliable → the ~30 workout cues are **\*\*pre-recorded audio files\*\*** (one afternoon with a native speaker, ~1 MB bundled); EN/HI use device TTS. A form-checking app that *\*speaks Assamese\** in a Jorhat gym demo is the moment the trainer smiles – cheap, decisive.

**\*\*§10 Voice coach (#11):\*\*** cue policy is already law (Part 2 §4.9 – one cue/rep, 4 s gap, ≤ 2 repeats, positive reinforcement on fixes); mobile playback via expo-speech/bundled audio with music ducking; persistent mute toggle.

**\*\*§11 Trainer mode (#12):\*\*** stays gated – **\*\*≥ 5 paying orgs asking\*\*** (the clinic-gate discipline). When it opens: assignment = §6 programs + Part 3 §6.1 On-plan (already built by then); live-view = the WebSocket that finally earns its way back (v1 §21). One paragraph on purpose; speccing it now would be speculation wearing a roadmap.

**\*\*§12 QoL trio (#14):\*\*** rest timer (auto-starts post-set; default 60 s / 90 s for F1-F3 heavies; editable; local notification if backgrounded) · plate calculator (offline barbell math, kg-first with lb toggle) · workout

note (`workouts.note`, addendum B – private, **\*\*org-invisible\*\***, the \$Part 3 \$2.4 boundary extended by default to anything new; that's now the standing rule for every future field).

### **## 13. The global notification budget (every push in this playbook, one table)**

| Push                        | Class              | Trigger                              | Cap                         |
|-----------------------------|--------------------|--------------------------------------|-----------------------------|
| ---                         | ---                | ---                                  | ---                         |
| Streak nudge \$3.4          | engagement         | 19:00, streak $\geq 3$ , no activity | the 1/day slot              |
| Recap \$8                   | engagement         | Sun 18:00                            | weekly (owns Sunday's slot) |
| Freeze used \$3.2           | transactional-lite | morning after                        | 1/event                     |
| PR / badge / challenge-done | celebration        | on event, suppressed if seen in-app  | 2/week combined             |
| Challenge behind-pace \$4.1 | engagement         | once per challenge                   | 1/challenge                 |
| Resurrection rungs \$8      | resurrection       | D3/D7/D14/D30                        | ladder only, then stop      |
| Org "we miss you" (Part 3)  | org                | owner-initiated                      | 1/member/7d (already set)   |

Server-side arbitration (v1 \$15): engagement class shares one daily slot – if two qualify, priority = streak > recap > challenge; losers drop silently (never queue-and-flood tomorrow). Quiet hours 21:30-08:00 user TZ. In-app-seen suppression everywhere.

### **## 14. Measurement & kill criteria (the monthly 30-minute retention review)**

Per feature: adoption %, target metric (\$1), guardrail, and a pre-committed action – e.g., streaks: if streak-loss-week churn > 1.5× baseline → build repair (\$3.3); recap: unsubscribe > 3 %/month → halve frequency; referral: fake-flag > 10 % of grants → tighten activation gate to 5 workouts; programs: < 15 % completion → redesign. A/B where volume allows via `feature\_flags` + PostHog cohorts; below A/B volume, before/after cohorts with honesty about noise (the Trust doctrine applies to *\*your\** dashboards too). The ritual: first Monday monthly, 30 minutes, D7/D30 curves by signup cohort + this table – decisions get one line in a `RETENTION\_LOG.md`.

```

15. DDL addendum B (migration `0003_growth`)

```sql
ALTER TABLE users ADD COLUMN referral_code text UNIQUE, ADD COLUMN
referred_by uuid REFERENCES users(id);
ALTER TABLE workouts ADD COLUMN note text;
CREATE TABLE entitlement_credits (
    id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    plan_id uuid NOT NULL REFERENCES plans(id), days smallint NOT NULL,
    source text NOT NULL, granted_at timestamptz NOT NULL DEFAULT now(),
    consumed_from timestamptz, consumed_until timestamptz );
CREATE TABLE programs ( id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
code text UNIQUE NOT NULL,
    name_key text NOT NULL, level text, goal text, weeks jsonb NOT NULL,
    status text NOT NULL DEFAULT 'live' CHECK (status IN
('draft','live','retired')) );
CREATE TABLE program_enrollments ( id uuid PRIMARY KEY DEFAULT
gen_random_uuid(),
    user_id uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    program_id uuid NOT NULL REFERENCES programs(id),
    started_at timestamptz NOT NULL DEFAULT now(), current_day int NOT NULL
DEFAULT 1,
    completed_at timestamptz, abandoned_at timestamptz );
CREATE UNIQUE INDEX enroll_one_active_uq ON program_enrollments (user_id)
    WHERE completed_at IS NULL AND abandoned_at IS NULL;
CREATE TABLE user_prs ( user_id uuid NOT NULL REFERENCES users(id) ON
DELETE CASCADE,
    exercise_id uuid REFERENCES exercises(id), kind text NOT NULL,
    value numeric NOT NULL, workout_id uuid, achieved_at timestamptz NOT
NULL,
    PRIMARY KEY (user_id, exercise_id, kind) );
CREATE TABLE challenge_org_scores ( challenge_id uuid NOT NULL REFERENCES
challenges(id) ON DELETE CASCADE,
    gym_id uuid NOT NULL REFERENCES gyms(id), score numeric NOT NULL, rank
int,
    PRIMARY KEY (challenge_id, gym_id) );
```

```

Resolver amendment (extends Part 4 §4.1): third UNION arm selects rank/entitlements from `plans` joined via active `entitlement\_credits` windows; credit consumption (`consumed\_from/until`) is stamped by the day-close sweep only when no paid candidate exists – banked otherwise (§5.2). DPDP cascade (Part 4 §5.2) extends to: credits, enrollments, user\_prs, org\_scores rows.

## ## 16. Build mapping & acceptance

**\*\*Ships with v1 core (Phases 2-4, per v1 §21 sequencing):\*\*** §2 Form Score surfaces · §3 streaks/freezes · §5 share cards + referral · §7 PR/badges · §8 recap. ✓ *\*Done when:* freeze auto-save fires correctly across a TZ-travel test · delta gates verified (no delta on 4-set weeks) · referral end-to-end on sandbox (activation gate, credit banking while paid, credit consumption after cancel) · recap renders banded kcal and honors the Sunday slot arbitration · share card generates < 5 s and WhatsApp-shares with working QR.

**\*\*v1.1 (with console Slice B):\*\*** §4 personal + org challenges. **\*\*Gated:\*\*** City League (5 orgs/city) · Trainer mode (5 orgs asking). **\*\*Content track (parallel, no code):\*\*** 3 launch programs · badge catalog · HI/AS passes + Assamese cue recordings.

---

*\*– End of Part 7. Final queue item: **\*\*Part 8 – Ops Runbook\*\*** (deploy, monitoring, backup drills, incident basics, launch checklist).\**

## # Part 8 – Ops Runbook (Solo-Operator Edition)

**\*\*Prerequisites:\*\*** v1 §17-§20 (observability, deploy, scale triggers) · Part 2 §9.3-9.4 (definition rollout/telemetry) · Part 4 (migration discipline, sweeps, drill mandate) · Part 5 (money alerts, sweeps) · Part 6 §11 (mobile release trains).

**\*\*Scope:\*\*** deploy procedures for all four release surfaces · the alert catalog with thresholds and first actions · backup/restore drills · incident playbooks and break-glass commands · the routine ops calendar · security & cost ops · the launch checklist that ties Parts 1-8 together.

## ## 0. Doctrine for a team of one

1. **\*\*You are not on call – your defaults are.\*\*** You will be asleep, in a gym demo, or on a bus to Guwahati when things break. Therefore: every failure mode degrades to a *\*safe standing state\** without you (quotas fail-open/closed by design, ``past_due`` grants, sync retries forever, pose runs on-device regardless of any server fire). The alert's job is to summon you to *\*restore\**, never to *\*prevent harm\** – harm prevention that requires a human awake is a design bug, and the earlier parts were written so there are none.
2. **\*\*One command per emergency, rehearsed before it's needed.\*\*** Everything below is a subcommand of a thin ``infra/ops`` CLI (wrapper scripts, nothing clever): ``ops deploy``, ``ops rollback``, ``ops flag``, ``ops drill``, ``ops breakglass ...``. If a procedure isn't a command, it's a blog post – and you don't read blog posts at 2 a.m.
3. **\*\*Boring on purpose.\*\*** No infra change lands the same week as a product launch; no Friday-evening deploys; no clever migrations. The runbook's KPI is how rarely it's interesting.
4. **\*\*Write-downs beat memory:\*\*** five living files, referenced throughout – ``RUNBOOK/`` (one page per alert/playbook), ``INCIDENTS.md``, ``DRILL_LOG.md``, ``DECISIONS.md``, ``CHANGELOG-OPS.md``. Standing rule: any prod change that would surprise future-you gets one line in ``DECISIONS.md`` the same day.

## ## 1. Topology & the "if my laptop dies" inventory

|                      |                                                      |                                                                                                    |  |
|----------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------------|--|
| Piece                | Where                                                | Access recovery                                                                                    |  |
| --- ---              | --- ---                                              | --- ---                                                                                            |  |
| API + worker + Caddy | Hetzner VPS (Docker Compose, <code>`infra/`</code> ) | Hetzner account (2FA + recovery codes in escrow); rebuildable from repo (\$4.4 game day proves it) |  |
| Postgres             | Neon (prod + <code>`staging`</code> branch)          | Neon 2FA; PITR + our dumps                                                                         |  |
| Redis                | Upstash                                              | rebuildable by design (v1 \$7.2; \$5.3 playbook)                                                   |  |
| Web + console        | Vercel                                               | instant rollback built-in                                                                          |  |
| Objects/CDN          | Cloudflare R2                                        | bucket versioning on                                                                               |  |
| Mobile               | EAS + Play + App Store                               | Part 6 \$11                                                                                        |  |
| Repo/CI              | GitHub (Actions)                                     | hardware-key 2FA                                                                                   |  |
| Money                | Razorpay, Stripe, RevenueCat                         | 2FA everywhere; webhook secrets per env                                                            |  |
| Alerting             | Better Stack (uptime/status) + Telegram bot          |                                                                                                    |  |
|                      | phone-independent email fallback                     |                                                                                                    |  |

**\*\*Escrow doc\*\*** (encrypted, stored twice – password manager + offline): every vendor account, its 2FA recovery codes, every secret's name → where used → rotation command → last-rotated date. This single document is the difference between a lost laptop being an afternoon and being an incident.

## ## 2. Deploy procedures (four release surfaces, one rhythm)

**\*\*Deploy window:\*\*** 11:00–15:00 IST (gyms peak 06–09 and 17–21 IST – never deploy into their rush), never Friday after 15:00.

### ### 2.1 API + worker

PR → CI gates (typecheck · unit · **\*\*golden traces\*\*** · Part 5 money suite · Gitleaks · build) → merge → auto-deploy **\*\*staging\*\*** → smoke suite (health, auth round-trip, sync echo, one sandbox charge) → `ops deploy promote` (the one manual click) → post-deploy watch: 15 min on Sentry release + 5xx rate; the deploy isn't "done" until that window closes clean.

**\*\*Rollback:\*\*** `ops rollback api <previous-tag>` – always safe because migrations are expand-then-contract (Part 4 §1): destructive steps (drops/renames-away) ship only in release N+1 after N has soaked a week. Code can therefore always step back one version against the live schema, by construction.

### ### 2.2 Web/console (Vercel)

Preview per PR → promote; rollback is a dashboard click. The console is inside `apps/web` (Part 3 §3.1), so one surface, one rhythm.

### ### 2.3 Mobile

Part 6 §11 verbatim: OTA weekly for JS (fingerprint runtimeVersion makes native mismatch unshippable), native monthly, staged 10→50→100 with 24 h Sentry gates; `ops mobile halt` toggles the rollout.

### ### 2.4 Definition bundles & config (the deploys that don't look like deploys)

Exercise definitions follow Part 2 §9.3 (draft → beta soak → live; **\*\*rollback = `ops defs rollback <bundle>`\*\***, a repoint, seconds). `plans.entitlements` and `feature\_flags` edits are production changes: made via admin panel only, audit-logged automatically, one line in `CHANGELOG-OPS.md` – a mispriced plan is an outage with better manners.

## ## 3. Monitoring & the alert catalog

**\*\*Severities:\*\*** **\*\*P1\*\*** = Telegram + phone-call escalation, act now (money broken, API down, backup failed, data-loss risk). **\*\*P2\*\*** = Telegram, same day; quiet-hours (22:00-08:00 IST) P2s batch to morning. **\*\*P3\*\*** = weekly review queue. Every alert message ends with a link to its `RUNBOOK/` page – the alert *is* the table of contents.

| Alert | Condition                  | Sev | First action                                                                                                              |
|-------|----------------------------|-----|---------------------------------------------------------------------------------------------------------------------------|
|       |                            |     |                                                                                                                           |
|       | API down                   |     | /health fails 2 consecutive checks   P1   `ops status` → VPS/Neon/Upstash triage page                                     |
|       | 5xx spike                  |     | > 1 % of requests, 5 min   P1   recent deploy? → rollback first, diagnose second                                          |
|       | Backup failed              |     | nightly dump job non-zero / missing artifact   P1   rerun now; a night without a backup is not acceptable risk            |
|       | Illegal money transition   |     | Part 5 §3 alert fires   P1   freeze nothing; read audit trail; likely adapter bug                                         |
|       | Webhook backlog            |     | pending > 50 or any `failed` (Part 4 table)   P2   `ops queue drain webhooks`; check provider status page                 |
|       | Sync success               |     | < 99.9 % over 1 h   P2   Sentry group; clients retry forever (Part 6) – no user harm, fix at leisure of hours not minutes |
|       | p95 latency                |     | > 250 ms for 30 min   P2   v1 §20 Stage-1 trigger playbook                                                                |
|       | Queue age                  |     | oldest BullMQ job > 15 min   P2   worker logs; `ops worker restart`                                                       |
|       | Per-gym cost breaker       |     | > ₹800/gym/month (v1 §9.3)   P2   cost dashboard → which feature; quotas already capped the bleeding                      |
|       | Provider spend             |     | daily > 2× 30-day median   P2   per-feature cost events; suspect abuse or a retry loop                                    |
|       | Definition anomaly         |     | rage-quit % or fault-rate shift post-publish (Part 2 §9.4)   P2   `ops defs rollback`, then investigate with traces       |
|       | Mobile crash-free          |     | < 99.5 % on latest version   P2   halt staged rollout (Part 6)                                                            |
|       | Neon storage/conns         |     | > 80 %   P2   v1 §20 ladder page                                                                                          |
|       | VPS disk/CPU               |     | > 80 % sustained   P2   resize command in playbook (one Hetzner CLI line + reboot)                                        |
|       | Vendor quota               |     | Groq/ORS/Resend/Upstash at 70 % of tier   P2   raise tier or tighten cache – decided per page                             |
|       | Rollup/report/sweep missed |     | expected job absent by +2 h   P2   rerun (all idempotent by construction – Parts 4/5/7)                                   |

| India synthetic | TTFB from Mumbai probe > 1.5 s | P3 | weekly trend review |

| Flagged accounts (v1 \$14) | queue > 10 | P3 | weekly review ritual |

**\*\*The morning glance (one saved dashboard, 5 minutes):\*\*** uptime · last night's backup ✓ · sync success · Sentry new issues · queue depths · MRR & new subs · yesterday's actives · cost-vs-revenue outliers. If all green, close the laptop — that's the point.

---

## ## 4. Backups & restore drills ("an untested backup is a rumor" — v1 \$17, now a procedure)

### ### 4.1 What is backed up, where

Neon PITR (continuous, 7–30 d window) · **\*\*nightly `pg\_dump` → R2\*\*** (encrypted, 35-day rolling + first-of-month kept 12 months) · R2 bucket versioning (assets, reports, invoices) · repo on GitHub · secrets in the \$1 escrow · Redis **\*\*deliberately not backed up\*\*** (rebuildable by design — \$5.3 is the rebuild) · mobile artifacts live in EAS/store history.

### ### 4.2 The monthly drill (`ops drill restore`, ~20 min, scripted end-to-end)

1) Restore last night's dump into a fresh Neon scratch branch. 2) Verification script: row counts vs prod within tolerance · canary user's workouts/meals/subscription intact · migrations apply cleanly on top · Part 4 \$4 canonical queries return sane values. 3) One line in `DRILL\_LOG.md` (date, dump size, restore minutes, ✓/✗). A red drill is a **\*\*P1 on the backup pipeline\*\***, treated exactly like an outage.

### ### 4.3 Quarterly + annual

Quarterly: PITR drill — pick a timestamp 3 days back, branch-restore to it, verify a known-changed row shows its old value (this is the ransomware/fat-finger recovery path; rehearse it before you need it). Annually: **\*\*total-loss game day\*\*** — new VPS from `infra/` scripts, restore DB, point staging DNS, stopwatch running; target < 2 h, gaps become `infra/` fixes the same week.

## ## 5. Incident response (solo edition)

### ### 5.1 The loop

**\*\*Detect\*\*** (alert) → **\*\*Stabilize\*\*** (break-glass command — restore a safe state *before* diagnosis) → **\*\*Communicate\*\*** (status page + in-app banner flag; org owners get the email template only for > 30 min money/console impact) → **\*\*Fix\*\*** → **\*\*Postmortem-lite\*\***: 30 blameless minutes into `INCIDENTS.md` (timeline · impact · cause · 1-3 actions with dates). Five incidents with skipped postmortems = the same incident five times.

### ### 5.2 Break-glass commands (each rehearsed once on staging — \$9 requires it)

`ops breakglass maintenance on` (writes 503 with friendly copy; reads still served) · `ops breakglass metered off` (coach/meal/route-gen honestly disabled; **\*\*pose, logging, sync untouched\*\*** — the on-device architecture means your worst server day still counts every rep) · `ops breakglass grace-extend 48h` (payment-provider outage: nobody degrades for a failure that isn't theirs) · `ops queue pause|drain <name>` · `ops defs rollback <bundle>` · `ops rotate <provider>` (scripted per \$7) · `ops tv revoke-all` · `ops flag <key> <value>`.

### ### 5.3 Playbooks (one `RUNBOOK/` page each; triggers → numbered steps)

**\*\*DB degraded/down:\*\*** Neon status → if theirs, maintenance-on + wait (their SLO beats your heroics); if ours (conn saturation), restart pool, check the \$3 ladder. **\*\*Redis down:\*\*** by design — quotas fail-open (cheap) / fail-closed (metered) exactly per v1; leaderboards restore from `leaderboard\_snapshots` + resync; geo cache rewarms; document says *\*do nothing but restart it\**. **\*\*Payment provider outage:\*\*** webhooks queue (Part 5 doorbell doctrine), grace-extend if > 12 h, reconcile-on-recovery is automatic (fetch-authoritative). **\*\*LLM outage:\*\*** Groq → OpenRouter fallback is automatic (v1 \$6.1); both down → honest error, **\*\*no quota consumed\*\*** on failures (standing rule). **\*\*Bad deploy / mass sync failures:\*\*** rollback first; clients retry forever (Part 6 \$4.3) so the queue self-heals — your job is the fix, not the data. **\*\*Leaked secret:\*\*** `ops rotate <provider>` (exact console paths per provider in the page), then audit-log sweep for misuse window; you have already done this once — the runbook page is that experience, written down. **\*\*Store emergency (bad native build):\*\*** halt rollout; JS-fixable → OTA within the hour; native → expedited-review request template in the page. **\*\*Suspicious admin access:\*\*** revoke-all sessions, rotate JWT secrets, audit trail review — 15 minutes, scripted.

## ## 6. Routine ops calendar (the whole job, time-boxed)

**\*\*Daily (5 min):\*\*** morning glance (§3) · Telegram triage. **\*\*Weekly (45 min):\*\*** Renovate dependency PRs (patch/minor auto-merged by CI policy; majors reviewed) · Sentry to zero-or-ticketed · parked 4xx sync rejects (Part 6 – those are *\*your\** bugs) · flag-queue review (v1 §14) · finance digest read (Part 5 §13). **\*\*Monthly:\*\*** restore drill (§4.2) · retention review (Part 7 §14, 30 min) · rotate one provider's secrets (staggered so everything rotates quarterly, and rotation stays routine, not archaeology) · invoice/GST export to your CA · access review (the answer should be "just me, 2FA everywhere" – verify, don't assume). **\*\*Quarterly:\*\*** PITR drill · k6 load test: `/workouts/sync` + entitlement reads at 10× current peak (the number goes in `DRILL\_LOG.md`; regressions are Stage-trigger evidence, v1 §20) · unit-economics snapshot (cost dashboard vs plans – the Part 1 margins, re-verified against reality). **\*\*Annually:\*\*** game day (§4.3) · dependency major-version window · pricing review.

## ## 7. Security & compliance ops

Gitleaks in CI (standing) + quarterly manual scan of infra repo · npm audit gate: fail CI on high-severity in production deps · 2FA + recovery codes escrowed for every vendor (§1) · rate-limit tuning is a P3 review item, ban tooling lives in the admin panel with audit-logging · **\*\*DPDP ops:\*\*** deletion/export are automated (Part 4 §5.2) – the ops job is monitoring the day-14 job and a monthly count in the compliance note; breach response outline: assess → contain (rotate/revoke) → document timeline → notify per DPDP obligations – *\*confirm notification thresholds and timelines with a lawyer once, write the answer into this page\** · clinic-copy linter (Part 3 §2.2) runs in CI – positioning compliance is a build gate, not a memo.

## ## 8. Cost & capacity ops

The margin lives on one screen (v1 §9.3 cost dashboard + Part 5 MRR view, side by side – already specced; this section just says *\*look at it weekly\**). Capacity is not a judgment call: the §3 alerts **\*\*are\*\*** v1 §20's triggers, each mapped to a pre-planned action page (resize VPS → one command; Stage 1 second API node → the LB page written now, executed later; read replica → Stage 2 page). You never decide architecture during an incident – you executed a decision made in daylight.

## ## 9. The launch checklist (Parts 1-8, tied off)

**\*\*Gates (all must be green):\*\*** ☐ P1 acceptance per part – v1 \$22 phase criteria · Part 2 engine DoD + parity traces · Part 3 Slice A · Part 4 \$9 (incl. the zero-PII deletion scan) · Part 5 \$14 money matrix · Part 6 acceptance (incl. Play background-location pack) · Part 7 v1-core set. ☐  
**\*\*Security pass:\*\*** all secrets rotated post-audit (the v1 \$18 + Part 2 \$0 lists), Gitleaks clean, headers scan, admin 2FA. ☐ **\*\*Legal/copy:\*\*** privacy policy (on-device + never-stored-photos claims verbatim true) · ToS · org agreement with the fair-use quota table (v1 \$9.2) · clinic terms boundary (2B \$7) · health disclaimer everywhere Part 6 \$12 says. ☐ **\*\*Ops live:\*\*** every P1/P2 alert **\*\*test-fired once\*\*** (an alert that never fired is a rumor, same as backups) · drill ✓ in `DRILL\_LOG.md` · break-glass rehearsed on staging · status page up · escrow doc current. ☐ **\*\*Business ready:\*\*** pilot gym on `preview` with real members · support channel (email + WhatsApp Business number) with a canned-answers doc · Part 5 external items closed (CA, store programs, Razorpay KYC) · dashboards saved · the five living files created with their first entries.  
**\*\*Launch day:\*\*** deploy freeze from T-1 15:00 IST · morning glance × 3 (08:00/13:00/19:00) · rollback criteria pre-agreed in writing (5xx > 1 % or crash-free < 99 % ⇒ roll back, no debate) · one person (you) watching, everything else automated – which is the whole design.  
**\*\*Week 1:\*\*** glance twice daily · error budgets reviewed Friday · first-cohort funnel (signup → first workout → D7) against Part 7 \$1 targets · first `RETENTION\_LOG.md` entry.

## ## 10. The document map (the series, closed)

**\*\*Specs (immutable, amended forward):\*\*** Part 1 Architecture v1 → Part 2 Engine/EDS + 2B Trust Layer → Part 3 Org Console → Part 4 Database & Migration → Part 5 Billing → Part 6 Mobile → Part 7 Retention → Part 8 Ops. Amendments always travel forward (Part 3 \$0, Part 4 \$0, Part 6 \$0 pattern) – never edit an earlier part; supersede it and cite. **\*\*Living files:\*\*** `RUNBOOK/\*` · `INCIDENTS.md` · `DRILL\_LOG.md` · `RETENTION\_LOG.md` · `DECISIONS.md` · `CHANGELOG-OPS.md` · the encrypted escrow.  
**\*\*Acceptance for Part 8 itself:\*\*** the `ops` CLI exists with every \$2/\$5.2 subcommand · each alert fired once on purpose · one restore drill logged · break-glass rehearsal logged · game day on the calendar.

**\*\*What to do first, tomorrow morning:\*\*** the standing items with lead times – secrets rotation if any remain, Apple/Google program enrollments,

Razorpay KYC, the CA session, the ₹11k test phone – then v1 \$22 **\*\*Phase 0, week 1.\*\*** Everything after that is written down.

*\*– End of Part 8, and of the specification series. Build order: v1 \$22. –\**